

Teoría de la Información

CODIFICACIÓN DE FUENTE

2026

La Teoría de la Información de Shannon

La Teoría de la Información de Shannon es clave para entender cómo la información se transmite a través de distintos medios de comunicación.

RO by Raúl Oscar Klenzi

Last edited about 2 hours ago



Introducción a Claude Shannon

Vida Temprana

Claude Shannon nació en Petoskey, Michigan en 1916. Su fascinación por la electrónica lo llevó a estudiar ingeniería eléctrica en el MIT.

Principales Aportes

Shannon es conocido por su trabajo en la teoría de la información, así como en el desarrollo de circuitos digitales y la criptografía.

Legado

La obra de Shannon ha tenido un impacto en muchos campos, incluyendo la informática, la ingeniería eléctrica, las telecomunicaciones y la criptografía.

Comunicación y Señales



Canales de Comunicación

Shannon definió la información como la medida de libertad que se obtiene de un canal. Los canales de comunicación pueden ser analógicos o digitales.



Señales

Las señales son las perturbaciones que se transmiten a través de un canal. Una señal puede ser continua o discreta, y puede ser analógica o digital.



Codificación de las Señales

Shannon demostró cómo las señales pueden ser codificadas para optimizar su transmisión y reducir la cantidad de errores.

recibo x Unive x Recib x (sin a x Mi un x La Tec x La Tec x COM x COM x Gráfico x Introc x Estad x Wolfr x +

https://gamma.app/docs/La-Teoria-de-la-Informacion-de-Shannon-951wm144uf8skw7mode=doc

GOOGLE Gmail: correo electr... Descargar y convert... INCUCAI - Sistema... ¿Quieres jugar con L... Convierte imagen a... Singularity Hub | Th... INCUCAI - Sistema... gMapToGPX

Present

Conceptos Fundamentales de la Teoría de la Información

Información

La información es un concepto fundamental en la teoría de Shannon. Se define como la reducción de incertidumbre que se obtiene al recibir un mensaje.

Entropía

La entropía es una medida de la cantidad de información en un mensaje. Cuanto mayor es la entropía, mayor es la incertidumbre y, por lo tanto, mayor es la cantidad de información que se transmite.

Ruido

El ruido es cualquier perturbación que afecta la transmisión del mensaje. El ruido puede ser causado por factores como la interferencia electromagnética o la atenuación de las señales.

Made with Gamma

USD/MXN -0,57%

Búsqueda

ESP LAA 14:16 22/8/2023

Entropía y su Importancia en la Teoría de la Información

Importancia de la Entropía

La entropía es fundamental para determinar la cantidad de información que se puede transmitir a través de un canal de comunicación y cómo se debe codificar el mensaje para optimizar su transmisión.

1

Cálculo de la Entropía

Shannon desarrolló una fórmula para calcular la entropía de un mensaje, que está determinada por la probabilidad de cada símbolo que se utiliza.

2

Limitaciones

La entropía tiene limitaciones en cuanto a su capacidad para representar la complejidad de la información. Otros factores, como la redundancia y el contexto, también son importantes en la transmisión de la información.

3

Canales de Comunicación y Capacidad de Información

Canal de Comunicación

Un canal de comunicación es cualquier medio a través del cual se transmite información. Los canales pueden ser físicos, como el cableado o el aire, o virtuales, como un protocolo de red.

Capacidad de Información

La capacidad de información es la cantidad máxima de bits que se pueden transmitir por segundo a través de un canal de comunicación. Shannon demostró cómo se puede calcular la capacidad de un canal en función de la tasa de error y la relación señal-ruido.

Codificación de la Información



Codificación de Fuente

La codificación de fuente se refiere a la manera en que se codifica la información antes de ser transmitida por el canal de comunicación. La codificación de fuente puede mejorar la eficiencia de la transmisión de datos y reducir la cantidad de errores.



Codificación de Canal

La codificación de canal se refiere a la manera en que se transmiten los datos a través del canal de comunicación. La codificación de canal puede mejorar la integridad de los datos y reducir la cantidad de errores.



QR Code

El código QR es un ejemplo de codificación de fuente que se utiliza para almacenar información en un formato que puede ser leído por un escáner de código QR. El código QR es muy eficiente en cuanto a la cantidad de información que puede almacenar y la velocidad a la que se puede leer.



Aplicaciones Prácticas de la Teoría de la Información

1 Compresión de Datos

La teoría de la información se utiliza en la compresión de datos para reducir la cantidad de espacio de almacenamiento necesario para un archivo sin perder información.

2 Redes de Comunicación

La teoría de la información se aplica en el diseño y optimización de redes de comunicación, incluyendo redes de computadoras y sistemas de telefonía celular.

3 Teoría de los Códigos

La teoría de los códigos se utiliza en la criptografía y la seguridad informática para proteger la información confidencial de ser interceptada o modificada por terceros.

EXTREMO TRANSMISOR

FUENTE

CODIFICACION
DE LA
INFORMACION
COMPACTACION

ENMASCARAMIENTO
DE LA
INFORMACION
ENCRIPCACION

CODIFICACION
DE
CANAL
CONTROL DE ERROR

TEORÍA DE INFORMACIÓN

FUENTE

DECODIFICACION
DE LA
INFORMACION
DESCOMPACTACION

DESENMASCARAMIENTO
DE LA
INFORMACION
DESENCRIPCACION

DECODIFICACION
DE
CANAL
CONTROL DE ERROR

C
A
N
A
L

D
E

C
O
M
U
N
I
C
A
C
I
O
N

RUIDO

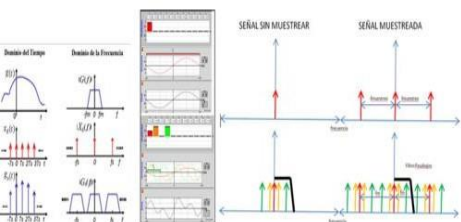
EXTREMO RECEPTOR

BIBLIOGRAFÍA DE REFERENCIA PARA LA ASIGNATURA

[BIBLIOGRAFIA/T.S.I.C/Stefan M Moser, Po-Ning Chen-A student's guide to coding and information theory-Cambridge University Press \(2012\).pdf](#)

[BIBLIOGRAFIA/T.S.I.C/Algorithmic Information Theory - Mathematics of Digital Information Processing Signals and Communication Technology.pdf](#)

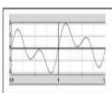
TEOREMA DEL MUESTREO DE SHANNON



RECUPERACIÓN DE LA SEÑAL ORIGINAL DESDE LAS MUESTRAS

Muestras de la señal

7

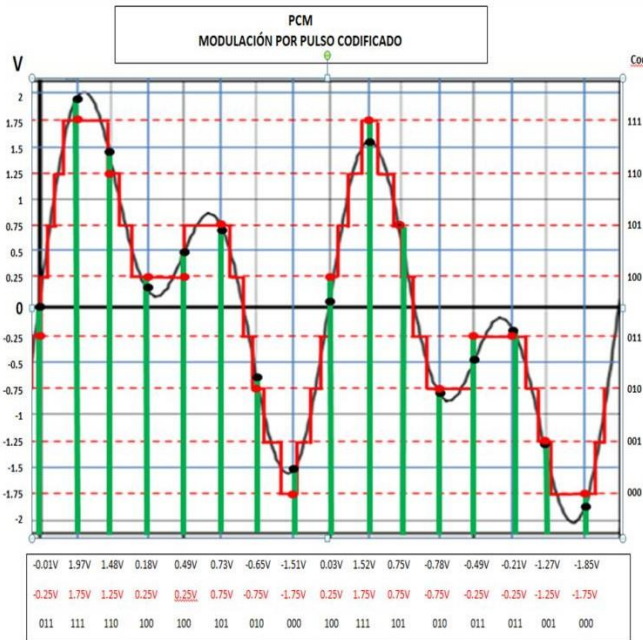


LA FRECUENCIA DE MUESTREO ES MAYOR O IGUAL QUE DOS VECES EL MÁXIMO CONTENIDO ARMÓNICO DE LA SEÑAL ORIGINAL!!!!!!

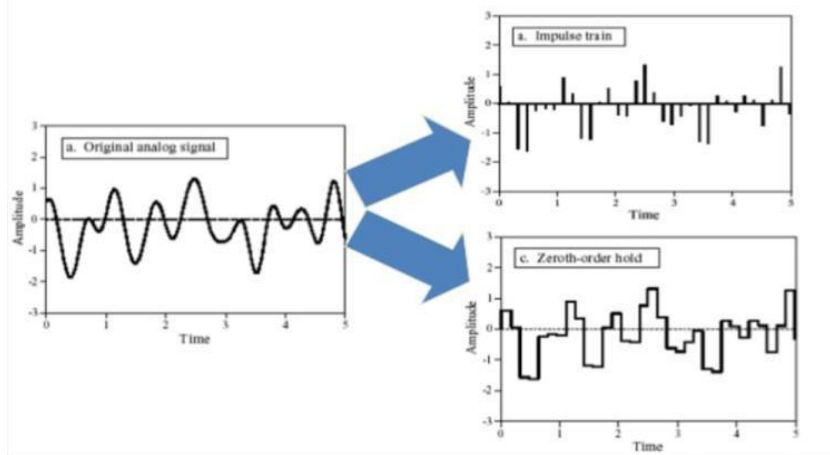
f_m mayor o igual que $2 \cdot f_{max}$

O EL PERIODO DE MUESTREO T_m SEA MENOR O IGUAL QUE LA MITAD DEL PERIODO DE LA SEÑAL DE MAYOR FRECUENCIA QUE CONSTITUYE LA SEÑAL ORIGINAL

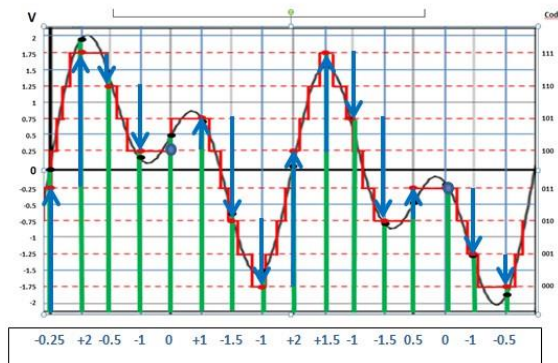
T_m menor o igual que $T_{max}/2$



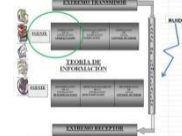
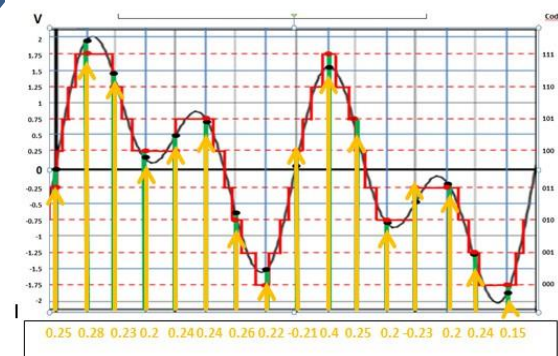
Muestreo



DPCM MODULACIÓN POR PULSO CODIFICADO DIFERENCIAL

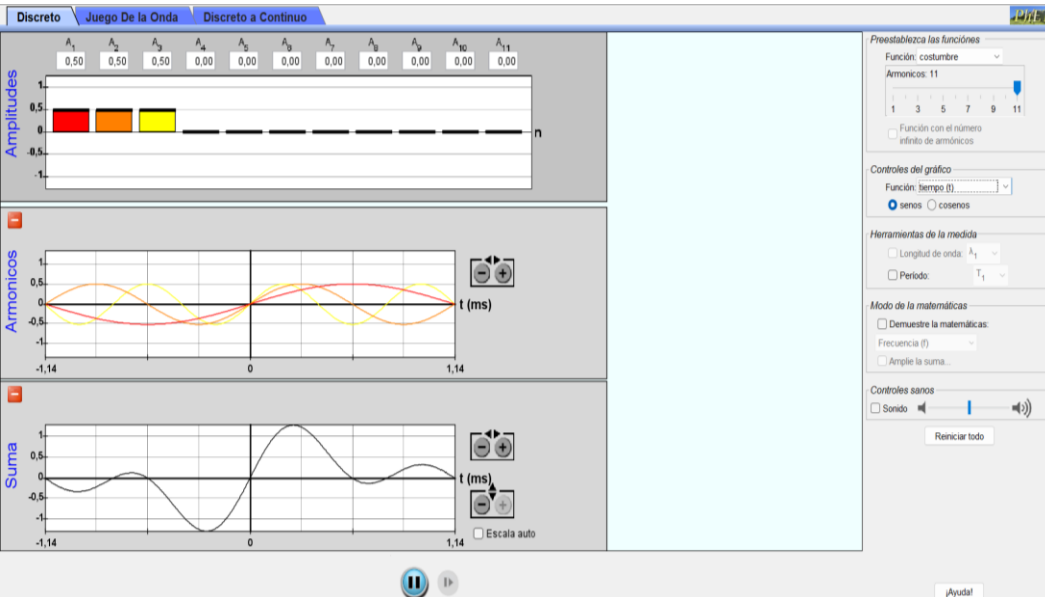


ADPCM MODULACIÓN POR PULSO CODIFICADO DIFERENCIAL CON PREDICTO



ONDAS

DISCRETIZACION TEMPORAL LIMITE DE NYQUIST



Para recrear exactamente los gráficos del simulador PhET Fuera de su interfaz, utilizaremos la formulación matemática indicada en el panel: tres senoides con amplitudes $A_1 = 0,5$, $A_2 = 0,5$, $A_3 = 0,5$ y frecuencias $f_1 = 438,59$ Hz, $f_2 = 877,19$ Hz y $f_3 = 1315,78$ Hz.

La ecuación de la señal temporal resultante es:

$$x(t) = 0,5 \sin(2\pi \cdot 438,59 \cdot t) + 0,5 \sin(2\pi \cdot 877,19 \cdot t) + 0,5 \sin(2\pi \cdot 1315,78 \cdot t)$$

Representación de las Señales

1) Componentes y Señal Temporal Completa ($x(t)$)

- Armónico 1 (Rojo): $x_1(t) = 0,5 \sin(2\pi \cdot 438,59 \cdot t)$ con periodo $T_1 \approx 2,28$ ms.
- Armónico 2 (Naranja): $x_2(t) = 0,5 \sin(2\pi \cdot 877,19 \cdot t)$ con periodo $T_2 \approx 1,14$ ms.
- Armónico 3 (Amarillo): $x_3(t) = 0,5 \sin(2\pi \cdot 1315,78 \cdot t)$ con periodo $T_3 \approx 0,76$ ms.
- Señal Total (Suma): En el intervalo de $t = -1,14$ ms a $t = 1,14$ ms, la onda sumada alcanza su pico positivo máximo de $+1,25$ cerca de $t = +0,3$ ms y un valle mínimo de $-1,25$ cerca de $t = -0,3$ ms.

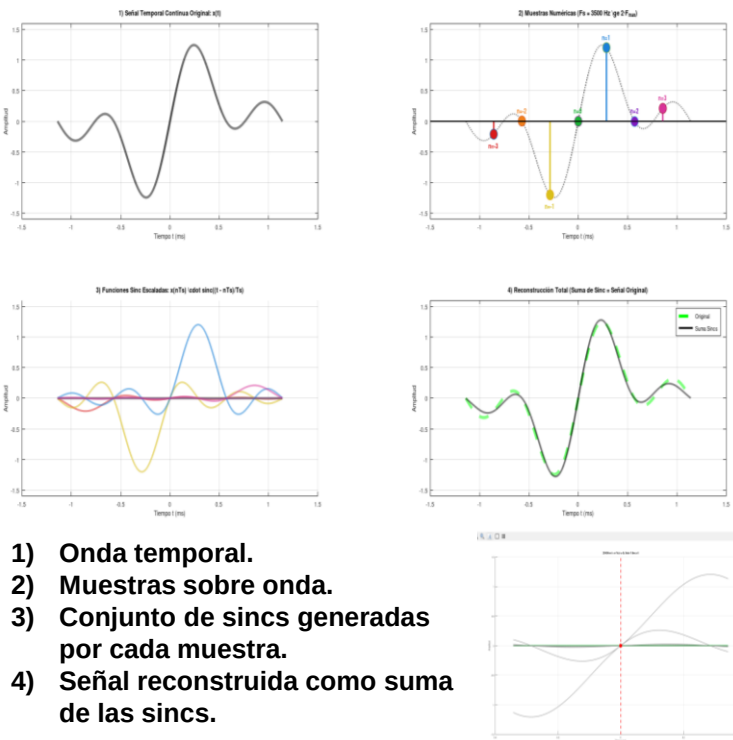
SE OBSERVAN QUE DESDE LOS PUNTOS DE MUESTRAS TOMADOS EQUIDISTANTEMENTE SOBRE LA ONDA CADA T_M (PERIODO DE MUESTRA) MEDIANTE ONDAS SINC SE RECONSTRUYE LA SEÑAL ORIGINAL SI SE RESPETAN LAS CONDICIONES DE NYQUIST.

$F_M \geq 2 \cdot F_{MAX}$ EN ESTE CASO $2631,57$ HZ = $2 \cdot 1315,78$ HZ

ESTO DA UN PERIODO DE MUESTREO DE $T_M = 1/F_M = 0,38$ MSEG. SE APRECIAN POR ELLO PARA EL TIEMPO DE REPRESENTACION DE LA ONDA $2,28$ MSG QUE APARECEN 7 MUESTRAS O SEIS ESPACIOS ENTRE MUESTRAS.

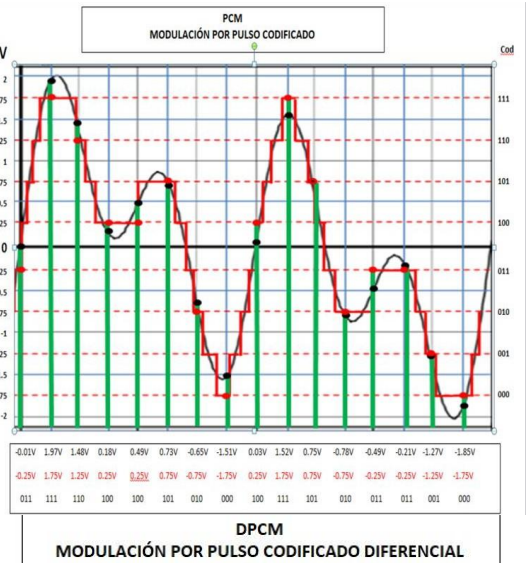
DESDE LAS MUESTRAS Y CONFORME LAS SEÑALES SINC QUE SE GENERAN POR CADA MUESTRA, SE RECONSTRUYE LA SEÑAL ORIGINAL!!

SE DESTACA COMO LO MUESTRA EL ULTIMO CUADRO UE EN CADA PUNTO DE MUESTRA SOLO HAY UNA SINC DISTINTA DE CERO DADO QUE LAS RESTANTES, EN ESE PUNTO, COINCIDEN TODAS EN UNO DE SUS CRUCES POR CERO



- 1) Onda temporal.
- 2) Muestras sobre onda.
- 3) Conjunto de sincs generadas por cada muestra.
- 4) Señal reconstruida como suma de las sincs.

DISCRETIZACIÓN EN AMPLITUD



COMO SE APRECIA DE LA SUCESIÓN DE IMÁGENES, **PCM** ES UNA PRIMER INSTANCIA DE DISCRETIZAR LA AMPLITUD DE LA SEÑAL ENVIANDO CADA MUESTRA (TOMADAS CADA T_m) CODIFICADA EN BINARIO SEGÚN EL VALOR MEDIO DE LOS SEGMENTOS CONSIDERADOS. (EN ESTE CASO: 8 SEGMENTOS, 3 BITS DE CÓDIGO) ESTA DIERENCIA ENTRE EL VALOR REAL DE LA SEÑAL CONTINUA Y SU APROXIMACIÓN AL VALOR MEDIO, ES UN ERROR PERSISTENTE AUN EN LA ETAPA DE RECUPERACIÓN O DECODIFICACIÓN, ATENUADA EN LA MEDIDA QE MAYOR CANTIDAD DE BITS O SEGMENTOS SE UTILICEN.

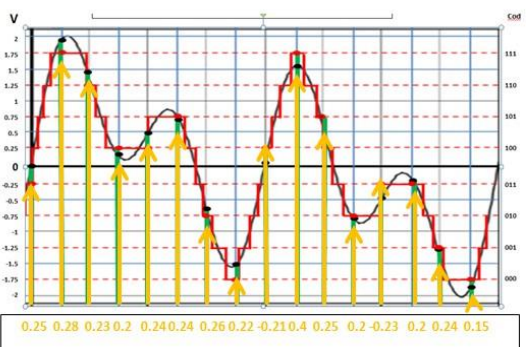
EN LA SEGUNDA IMAGEN SE CONSIDERA LA CODIFICACION **DPCM** QUE EN LUGAR DE CODIFICAR CADA MUESTRA, CODIFICA LA DIFERENCIA ENTRE MUESTRAS. DADO QUE EL RANGO DINÁMICO DE LA DIFERENCIA ENTRE MUESTRAS, SI SE RESPETA EL TEOREMA DE NYQUIST, DEBE SER MENOR QUE EL DE LA PROPIA MUESTRA MANTENIENDO EL ERROR DE LA APROXIMACIÓN PCM, SE PUEDE CODIFICAR CON MENOR CANTIDAD DE BITS.



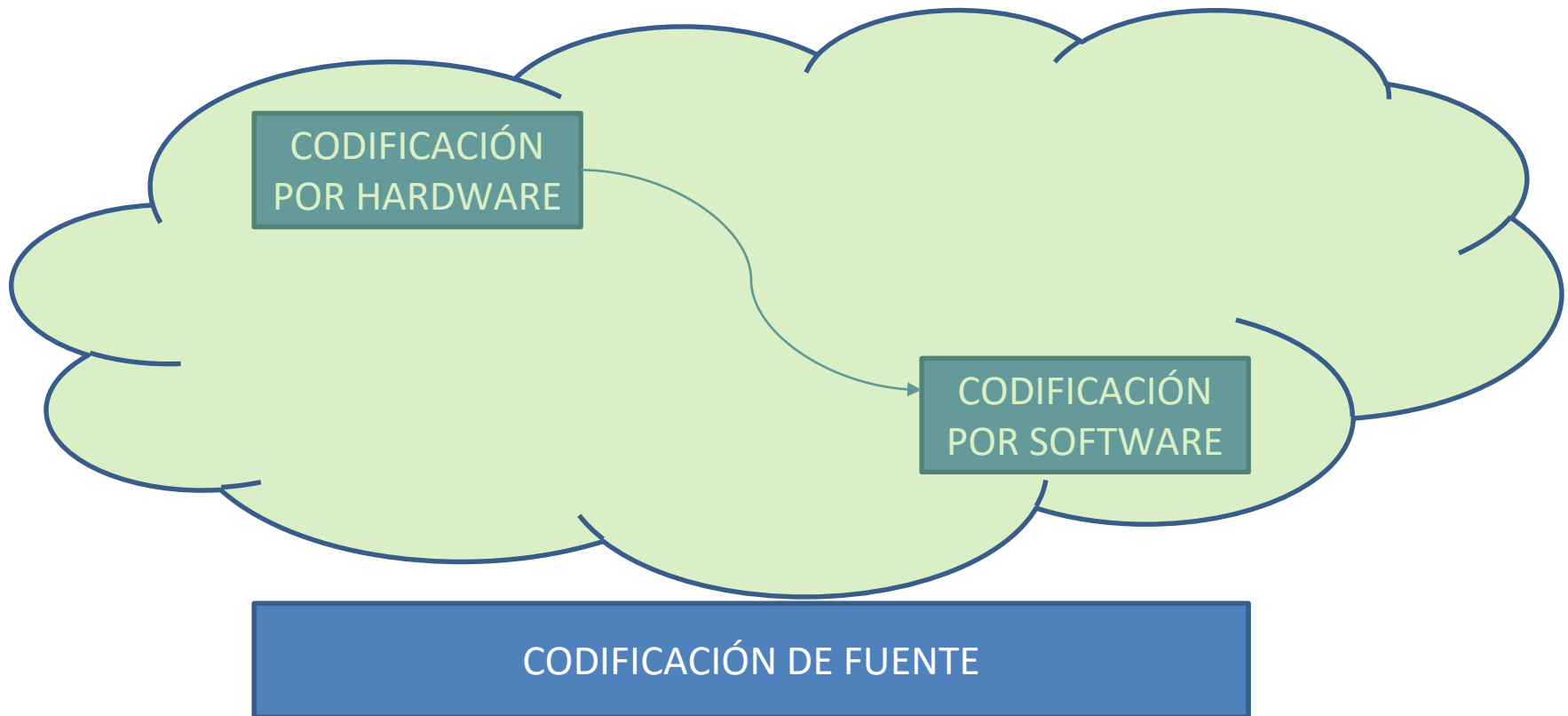
ANTE LA POSIBILIDAD DE UNA ERROR E LA RECEPCIÓN DE UNA MUESTRA, LO QUE PRODUCIRÍA UN ERROR PERMANENTE DESDE ESA MUESTRA ERRÓNEA EN LA RECUPERACIÓN DE LA SEÑAL EN EL EXTREMO RECEPTOR, SE ENVIA, A MODO DE SINCRONIZACIÓN Y A EECTOS DE ATENUAR ESA RECUPERACION ERRÓNEA, CADA CIERTA CANTIDAD DE CÓDIOS DE DIFERENCIAS UN CÓDIGO PCM COMPLETO.

LA TERCER IMÁGEN PRESENTA LA CODIFICACIÓN **ADPCM** DONDE SE CONSIDERA LA POSIBILIDAD DE UTILIZAR UN PREDICTOR (EL MISMO EN EMISOR Y RECEPTOR) DE LA SIGUIENTE MUESTRA. SI EL PREDICTOR ES BUENO, UNA VEZ MÁS EL RANGO DINÁMICO DE LA DIERENCIA ENTRE LA PREDICCIÓN Y LA MUESTRA SERÁ MENOR QUE EN PCM Y TAMBIÉN QUE LA DIFERENCIA ENTRE MUESTRAS DPCM POR LO QUE UNA VEZ MÁS SE ESTARÁ EN CONDICIONES DE REDUCIR LA CANTIDAD DE BITS DE CODIFICACIÓN MANTENIENDO EL ERROR DE DISCRETIZACIÓN POR DEBAJO DE UN UMBRAL. LA A EN **ADPCM** ES UE LA PREDICCIÓN UEDE TENER CARACTERÍSTICAS DE ADAPTABILIDAD CONFORME LA VARIACIÓN DE LA SEÑAL.

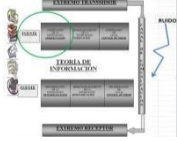
ESTA ÚLTIMA ALTERNATIVA SE APLICA A TIPOS DE SEÑALES CONOCIDAS COMO AUDIO O VIDEO.



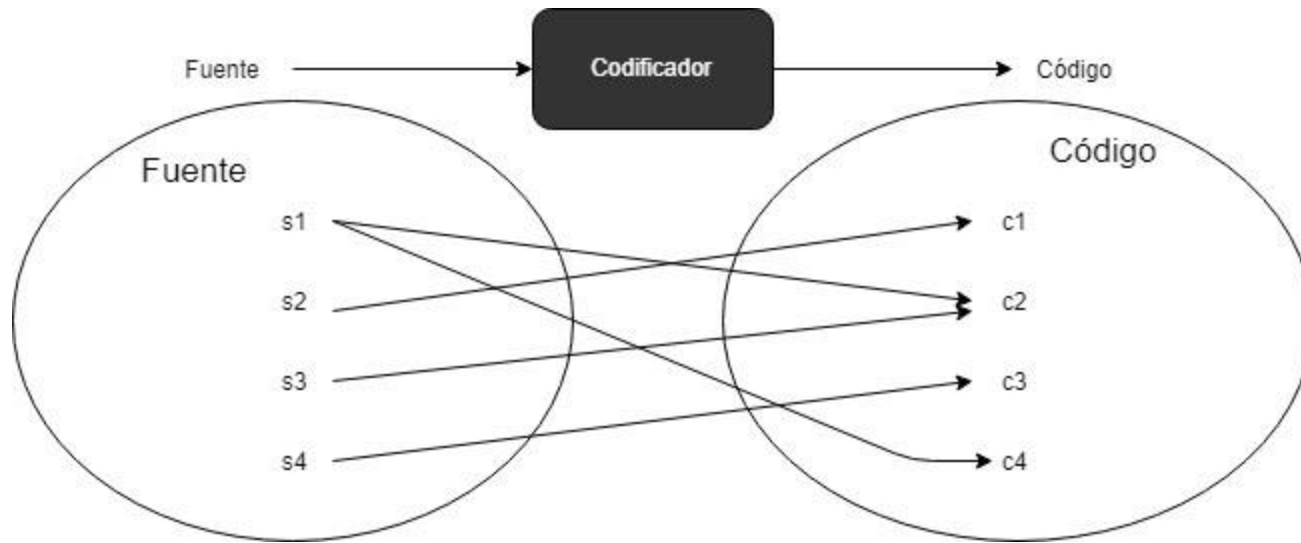
HABIENDO VISUALIZADO Y TOMADO CONOCIMIENTO DE LAS TECNICAS DE DISCRETIZACIÓN Y PRIMERAS ALTERNATIVAS DE CODIFICACIÓN Y REDUCCIÓN DE BITS FÍSICOS ANTERIORES (QUE CONFORME SON PROCESOS QUE SE REALIZAN EN PLACAS DE VIDEO O SONIDO - HARDWARE-), COMENZAREMOS LA SIGUIENTE SECCIÓN QUE LLAMAREMOS CODIFICACIÓN O COMPACTACIÓN POR SOFTWARE ATENDIENDO A QUE, YA DISPONIBLE LA FUENTE ORIGINAL EN SU FORMATO DISCRETO Y BINARIO INTENTAREMOS UNA VEZ MÁS, REDUCIR LOS BITS FÍSICOS UTILIZADOS EN EL PROCESO DE CODIFICACIÓN SIN PERDER INFORMACIÓN!!!



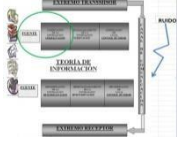
Definición de Código



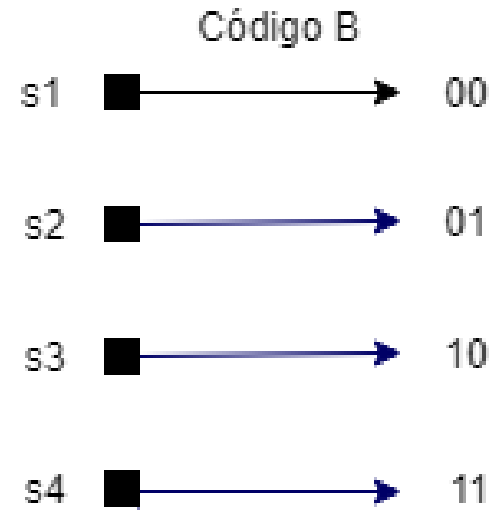
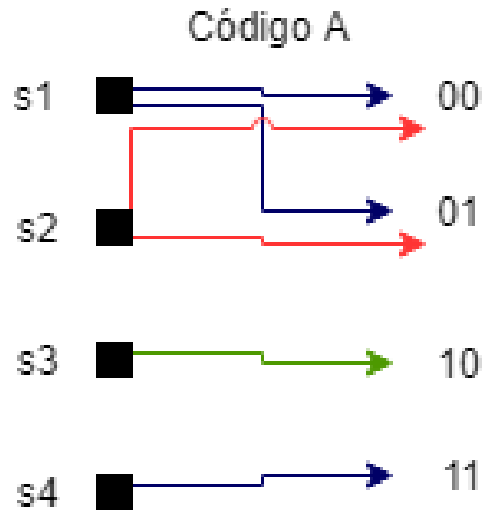
Un código es una **correspondencia** que existe entre un **conjunto de símbolos** o secuencia de símbolos posibles de **generar por una fuente** (letras del alfabeto, colores de una imagen, expresiones gestuales, etc.), llamada alfabeto fuente, y **un conjunto de símbolos** o secuencia de símbolos **pertenecientes a otro alfabeto**, este último se denomina **alfabeto código**.



Código Bloque



Código bloque: asigna a cada símbolo del alfabeto fuente una secuencia de longitud fija y determinada de símbolos del alfabeto código (palabra código).



Problema: cada símbolo, admite múltiples palabras código

Código no singular: es un código donde todas las palabras código son distintas.



Bastará con pedir que todas las palabras código sean distintas?

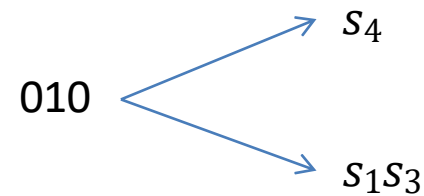
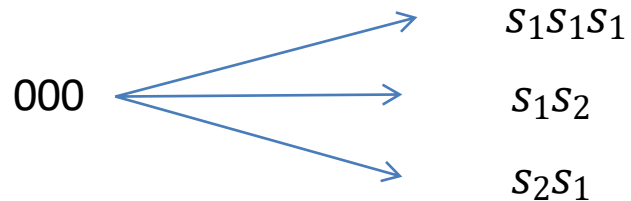
Código no bloque

$s_1 s_1 s_1$

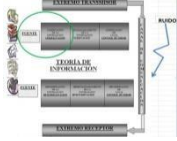
Código no bloque: es un código donde no todas las palabras código tienen la misma longitud.

Símbolo	Palabra código
s_1	0
s_2	00
s_3	10
s_4	010

Dadas las secuencias recibidas:



Extensión de orden n



Recordando de la unidad anterior, la extensión de orden n , de un código es la correspondencia que existe entre una secuencia de n símbolos del alfabeto fuente y una secuencia de n palabras código.

Código no bloque

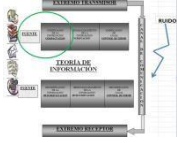
Símbolo	Palabra código
s_1	0
s_2	00
s_3	10
s_4	010

Extensión de orden 2

σ_i	Código	σ_i	Código
s_1s_1	00	s_3s_1	100
s_1s_2	000	s_3s_2	1000
s_1s_3	010	s_3s_3	1010
s_1s_4	0010	s_3s_4	10010
s_2s_1	000	s_4s_1	0100
s_2s_2	0000	s_4s_2	01000
s_2s_3	0010	s_4s_3	01010
s_2s_4	00010	s_4s_4	010010

Para lograr una decodificación unívoca (única) se exigirá que no sólo las palabras código sean distintas, sino también las palabras código de las extensiones del código.

Parámetros para evaluar y construir códigos

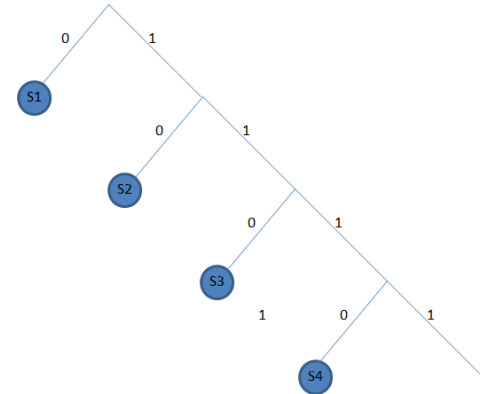


- Eficiencia: siendo la variedad de una fuente, $V = q$, r la base de codificación, y n la cantidad de bits, n debe cumplir con la siguiente inecuación : $n \leq \log_r q$.
- Seguridad: el código debe proteger la información permitiendo la detección y/o corrección de errores.

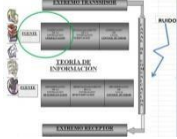
	0
	01
	011

Código B

Símbolo	Código
s_1	0
s_2	10
s_3	110
s_4	1110



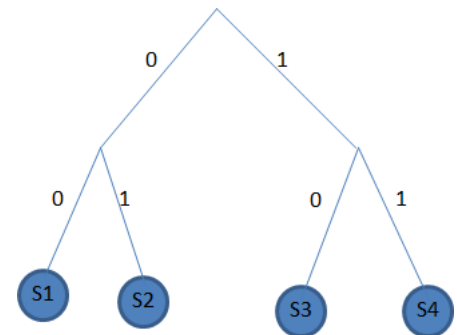
Un problema en la transmisión de información (1)



Con el objeto de exponer algunas ideas sobre codificación y su relación la medida de la información, se expondrá el siguiente problema:

Problema: se desea establecer un sistema de comunicación entre San Juan y Buenos Aires. El sistema debe transmitir a intervalos regulares datos sobre el estado del tiempo, debiendo hacer uso de un sistema binario para tal cometido. Para simplificar el problema, el estado del tiempo en San Juan, se clasificará en una de cuatro condiciones climáticas: soleado, nublado, lluvioso y frío. Estas cuatro condiciones constituyen los símbolos la fuente, en primer momento se va suponer la equiprobabilidad de los mismos.

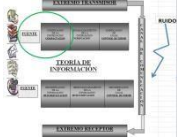
Código A			
Mensaje	Símbolo	$p(s_i)$	Código
Soleado	s_1	$1/4$	00
Nublado	s_2	$1/4$	01
Lluvioso	s_3	$1/4$	10
Frío	s_4	$1/4$	11



Con este código se pretende enviar el mensaje: «soleado, frío, frío, nublado», se codificará de la forma «00111101».

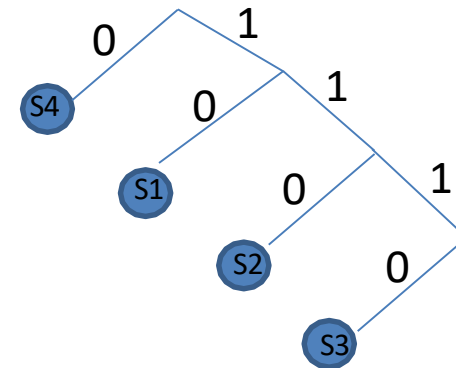
El código A, puede transmitir dicho mensaje sin ambigüedad alguna, además necesita dos dígitos binarios por símbolo.

Un problema en la transmisión de información (2)



Ahora, se verá el caso donde los mensajes no son equiprobables.

Código B			
Mensaje	Símbolo	$p(s_i)$	Código
Soleado	s_1	$1/4$	10
Nublado	s_2	$1/8$	110
Lluvioso	s_3	$1/8$	1110
Frío	s_4	$1/2$	0

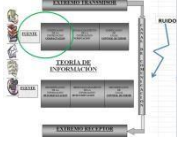


Si se utiliza el código A para transmitir información, la solución será igual, pero no mejor. Si se manda el mismo mensaje, es decir «solado, frío, frío, nublado», pero ahora con el código B, se transmitirá «1000110». Al igual que en el código A, la secuencia es única, se puede deducir el mensaje enviado sin ambigüedades, pero ahora el código es más corto.

$$L = \sum_{i=1}^q p(s_i) \times l(s_i) = \frac{1}{4} \times 2 + \frac{1}{8} \times 3 + \frac{1}{8} \times 4 + \frac{1}{2} \times 1 = 1,875 \text{ bits por símbolo}$$

La longitud del código se reduce de 2 bits por símbolo a 1,875 bits por símbolo.

Inecuación de Kraft



Sea una fuente S capaz de generar q símbolos distintos, $S = \{s_1, s_2, \dots, s_q\}$, r la base de codificación del sistema, y l_i las longitudes de cada palabra código.

Todo código instantáneo deberá satisfacer, respecto de las longitudes individuales de cada palabra código la siguiente expresión, denominada **Inecuación de Kraft**:

$$\sum_{i=1}^q r^{-l_i} \leq 1$$

Con el conjunto de longitudes que cumpla la inecuación se puede construir un código unívoco, se verá un procedimiento en general y luego en particular.

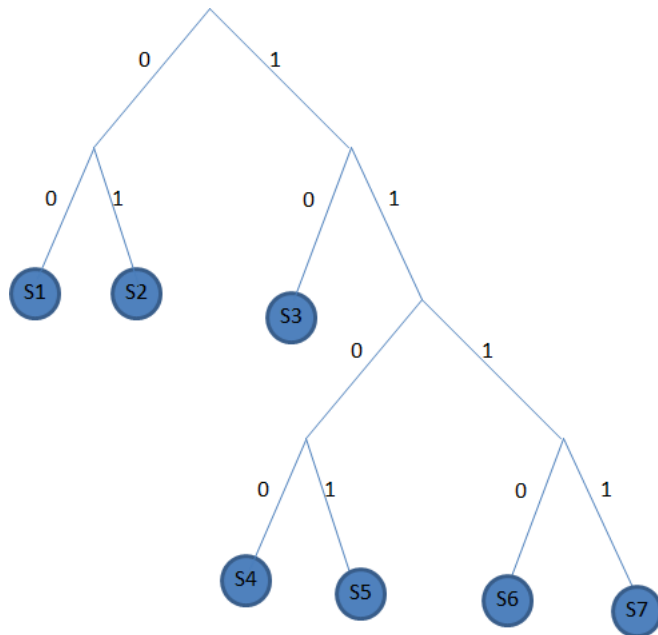
Dadas las siguientes longitudes de palabras código:

Código	Longitudes de palabras código
A	2, 2, 2, 4, 4, 4
B	1, 1, 2, 3, 3
C	1, 1, 2, 2, 2, 2

- a) Puede construirse un código binario instantáneo con alguna de ellas?
- b) Puede construirse un código ternario instantáneo con alguna de ellas?

Código	Longitudes de palabras código
A	2, 2, 2, 4, 4, 4
B	1, 1, 2, 3, 3
C	1, 1, 2, 2, 2, 2

Código A en base 2



Si las longitudes de un código, bajo la condición de evitar palabras códigos prefijo, se puede implementar en una base n también podrá realizarse en base $n+1$, $n+2$

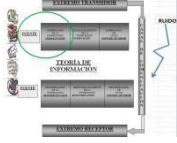
En este caso:

El código A puede implementarse en Base 2 y 3

El código B puede implementarse en Base 3

El código C no puede implementarse ni en Base 2 ni en Base 3

Construcción de un código instantáneo a partir de la inecuación de Kraft (1)



Sea una fuente $S = \{s_1, s_2, \dots, s_q\}$, $V = q$

La construcción del código instantáneo se hará a través de un código con base r .

Las longitudes del código propuesto, debe cumplir la inecuación de Kraft:

$$\sum_{i=1}^q r^{-l_i} \leq 1 \quad (1)$$

Se puede construir un código con:

Cantidad de palabras código	Longitud (l_i)
n_1	1
n_2	2
$\vdots$	$\vdots$
n_l	l

De modo que $\sum_{i=1}^l n_i = q$

Luego, se puede reescribir la inecuación (1) de la siguiente forma:

$$\sum_{i=1}^l n_i r^{-i} \leq 1 \quad (2)$$

The diagram illustrates the system architecture. At the top is the 'EXTENSIVE DECLASSIFIER'. Below it is a 'TECHNICAL INFORMATION' block, which is divided into two sections: 'TECHNICAL INFORMATION' and 'TECHNICAL INFORMATION'. Below this is another 'TECHNICAL INFORMATION' block, also divided into two sections: 'TECHNICAL INFORMATION' and 'TECHNICAL INFORMATION'. At the bottom is the 'EXTENSIVE RECEPTOR'. On the left side, there is a vertical column of icons representing various data sources, with 'GATE' labels indicating input points. On the right side, there is a vertical column of icons representing various data sinks, with 'GATE' labels indicating output points. A blue arrow points from the right side towards the bottom right corner.

Desarrollando la inecuación 2:

$$n_1r^{-1} + n_2r^{-2} + n_3r^{-3} + \cdots + n_{l-2}r^{-(l-2)} + n_{l-1}r^{-(l-1)} + n_lr^{-l} \leq 1$$

Multiplicando ambos miembros de la desigualdad por r^l :

$$\begin{aligned} n_1 r^{l-1} + n_2 r^{l-2} + n_3 r^{l-3} + \cdots + n_{l-2} r^{-l+2} r^l + n_{l-1} r^{-l+1} r^l + n_l r^{-l} r^l &\leq r^l \\ n_1 r^{l-1} + n_2 r^{l-2} + n_3 r^{l-3} + \cdots + n_{l-2} r^2 + n_{l-1} r^1 + n_l &\leq r^l \end{aligned} \quad (3)$$

Despejando n_l :

$$n_l \leq r^l - n_1 r^{l-1} - n_2 r^{l-2} - n_3 r^{l-3} - \dots - n_{l-2} r^2 - n_{l-1} r^1$$

Dividiendo ambos miembros por r :

$$n_l r^{-1} \leq r^{l-1} - n_1 r^{l-2} - n_2 r^{l-3} - n_3 r^{l-4} - \dots - n_{l-2} r^1 - n_{l-1}$$

Luego:

$$n_{l-1} \leq r^{l-1} - n_1 r^{l-2} - n_2 r^{l-3} - n_3 r^{l-4} - \dots - n_{l-2} r^1$$

Dividiendo ambos miembros por r :

$$n_{l-1}r^{-1} \leq r^{l-2} - n_1r^{l-3} - n_2r^{l-4} - n_3r^{l-5} - \dots - n_{l-2}$$

Luego:

$$n_{l-2} \leq r^{l-2} - n_1 r^{l-3} - n_2 r^{l-4} - n_3 r^{l-5} - \dots - n_{l-3} r^1$$

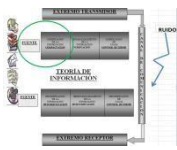
$$n_3 \leq r^3 - n_1 r^2 - n_2 r^1$$

$$n_2 \leq r^2 - n_1 r^1$$

$$n_1 \leq r$$

$$\sum_{i=1}^l n_i r^{-i} \leq 1 \quad (2)$$

Aplicación del método para crear un código instantáneo



Se toman n_1 palabras de longitud 1, de modo que $n_1 \leq r$, quedan $r - n_1$ prefijos de longitud 1, y pueden formarse $(r - n_1)r = r^2 - n_1r$ palabras de longitud 2.

Se toman n_2 palabras de longitud 2, de modo que $n_2 \leq r^2 - n_1r$, quedan $(r^2 - n_1r) - n_2$ prefijos de longitud 2, y pueden formarse $(r^2 - n_1r - n_2)r = r^3 - n_1r^2 - n_2r$

Se toman n_3 palabras de longitud 3, de modo que $n_3 \leq r^3 - n_1r^2 - n_2r$...

El proceso continúa hasta formar el código buscado.

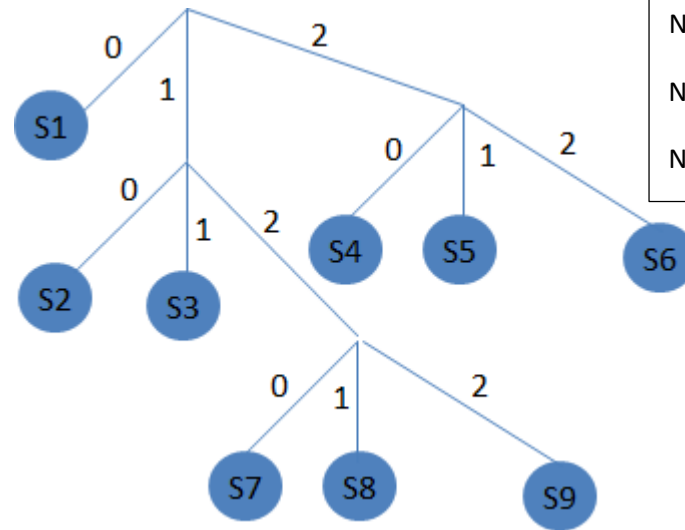
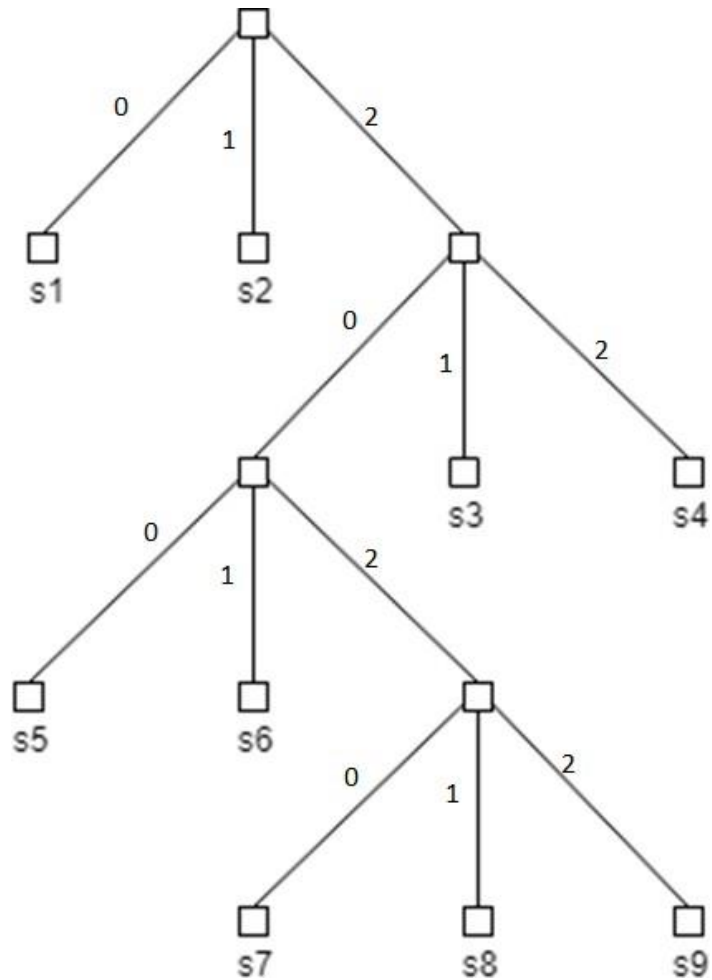
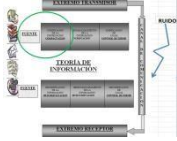
Ejemplo:

Dada una fuente $S = \{s_1, s_2, s_3, \dots, s_9\}$, $V = q = 9$

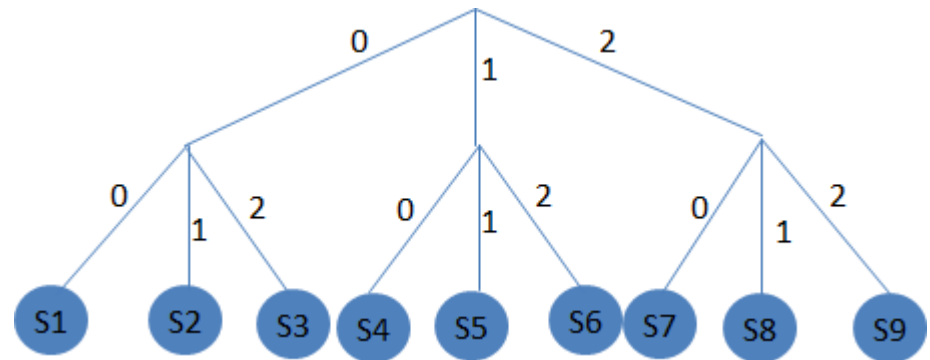
Se pretende construir un código instantáneo para los 9 símbolos, con una base $r = 3$.

Se toman $n_1 \leq 3$ palabras de longitud 1, por ejemplo $n_1 = 2$, quedan $3^2 - 2 \times 3$ posibles prefijos de longitud 2, $n_2 \leq 3^2 - 2 \times 3 = 3$, se toman $n_2 = 2$ prefijos de longitud 2, quedan $3^3 - 2 \times 3^2 - 2 \times 3$ posibles prefijos de longitud 3, $n_3 \leq 3^3 - 2 \times 3^2 - 2 \times 3 = 3$, se toman $n_3 = 2$ prefijos de longitud 3, quedando $3^4 - 2 \times 3^3 - 2 \times 3^2 - 2 \times 3$ posibles prefijos de longitud 4, se tomará la igualdad ya que con los tres prefijos restantes se completa el código.

Posible código ternario

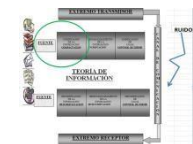


$N1 \leq 3$	$N1=1$
$N2 \leq 3^2 - N1 \cdot 3 = 9 - 3$	$N2=5$
$N3 \leq 3^3 - N1 \cdot 3^2 - N2 \cdot 3$	$N3=3$



$N1 \leq 3$	$N1=0$
$N2 \leq 3^2 - N1 \cdot 3 = 9$	$N2=9$

Longitud Promedio (1)



Dada una determinada codificación de la fuente, se busca calcular la longitud promedio de cada símbolo codificado, para lograrlo la metodología es la siguiente:

$$S = \{A, E, I, O\}$$

Secuencia de la fuente: ...AEEEEEEEOOIIAAOOOEEEA...

La secuencia tiene 20 símbolos.

s_i	Longitud de codificación	Cantidad en la secuencia
A	$l(A)$	4
E	$l(E)$	9
I	$l(I)$	2
O	$l(O)$	5
Total		20

La definición de promedio indica que se obtiene como la sumatoria de cantidad de símbolos, multiplicada por la longitud de cada uno de ellos, dividido por la cantidad total de símbolos, para este ejemplo:

$$L = (4 \times l(A) + 9 \times l(E) + 2 \times l(I) + 5 \times l(O)) / 20$$

Distribuyendo el denominador en cada sumando:

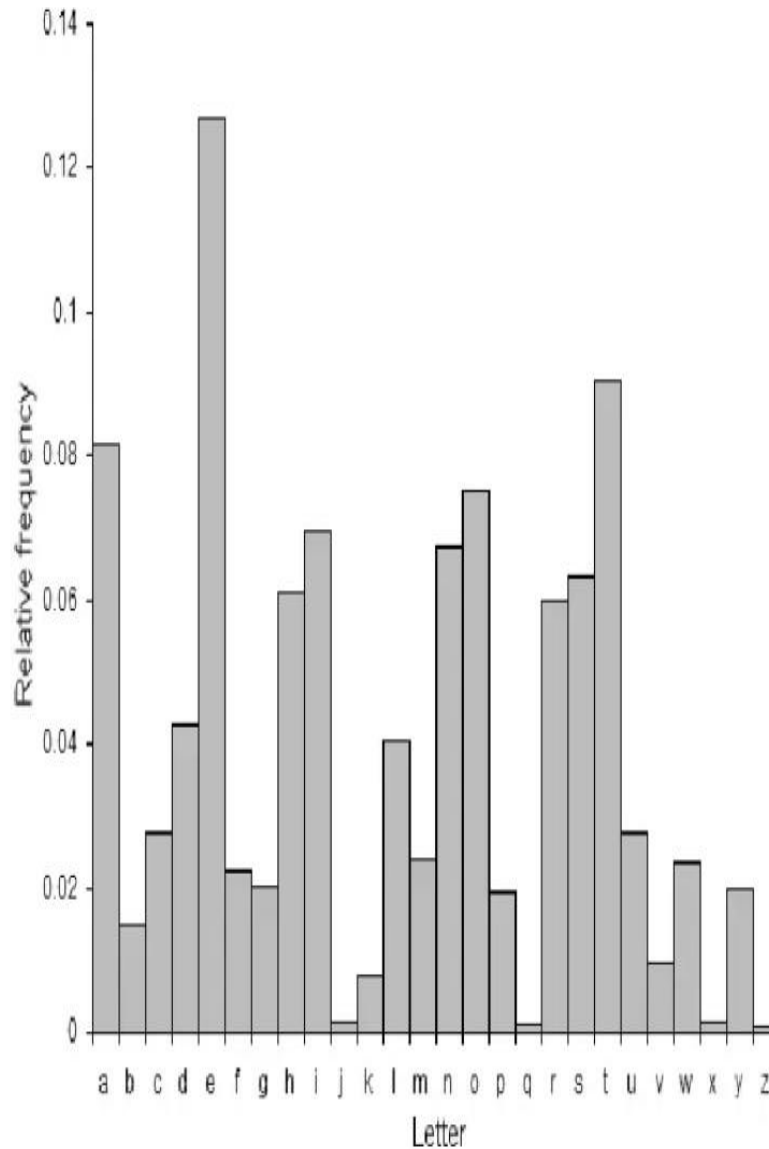
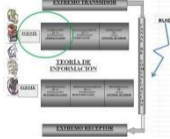
$$L = 4/20 \times l(A) + 9/20 \times l(E) + 2/20 \times l(I) + 5/20 \times l(O)$$

Y dado que $p(A) = 4/20$, $p(E) = 9/20$, $p(I) = 2/20$, $p(O) = 5/20$, la expresión general quedará:

$$L = \sum_{i=1}^q p(s_i) \times l(s_i)$$

Tabla de Frecuencias del idioma Inglés

Código Morse

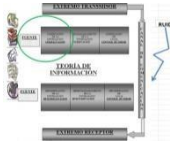


A ● -
 B - ● ● ●
 C - ● - ●
 D - ● ●
 E ●
 F ● ● - ●
 G - - ●
 H ● ● ● ●
 I ● ●

J ● - - -
 K - ● -
 L ● - ● ●
 M - -
 N - ●
 O - - -
 P ● - - ●
 Q - - ● -
 R ● - ●

S ● ● ●
 T -
 U ● ● -
 V ● ● ● -
 W ● - -
 X - ● ● -
 Y - ● - -
 Z - - ● ●

Longitud Promedio (2)

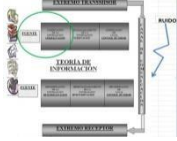


SEAN q SIMBOLOS QUE UNA FUENTE GENERA CON LAS PROBABILIDADES QUE SE EXPRESAN SEGUIDAMENTE Y DONDE SE PROPONEN DOS ALTERNATIVAS DE CODIFICACIÓN UNA CON L DE CODIFICACIÓN PROMEDIO Y DONDE A LOS SÍMBOLOS MÁS PROBABLES SE ASIGNAN LOS CÓDIGOS MÁS CORTOS Y OTRA L^* QUE NO RESPETA ESTA CONDICIÓN

	S_1	S_2	S_3	S_4		S_q	
	$P_1 \geq$	$P_2 \geq$	$P_3 \geq$	$P_4 \geq$		$P_q \geq$	
	$l_1 \leq$	$l_2 \leq$	$l_3 \leq$	$l_4 \leq$		$l_q \leq$	
L	$P_1 * l_1$	$+ P_2 * l_2$	$+ P_3 * l_3$	$+ P_4 * l_4$	$+ \dots$	$P_q * l_q$	
L^*	$P_1 * l_1$	$+ P_2 * l_3$	$+ P_3 * l_2$	$+ P_4 * l_4$	$+ \dots$	$P_q * l_q$	
$L - L^*$		$P_2 * l_2 - P_2 * l_3 +$	$P_3 * l_3 - P_3 * l_2$				
$L - L^*$		$P_2(l_2 - l_3) + P_3(l_3 - l_2)$					
$L - L^*$		$(P_2 - P_3)(l_2 - l_3) \leq 0$					

$$L \leq L^*$$

Longitud Promedio (3)



El mejor código es aquel que brinde menor longitud promedio, además, debería verificar la igualdad de la inecuación de Kraft.

A medida que la inecuación de Kraft, para un determinado código, se aleja de la unidad, si bien seguirá siendo unívoco, cada vez tendrá mayor valor la longitud promedio.

Relación entre Teoría de la Información y Teoría de la Codificación:

$$H \leq L$$

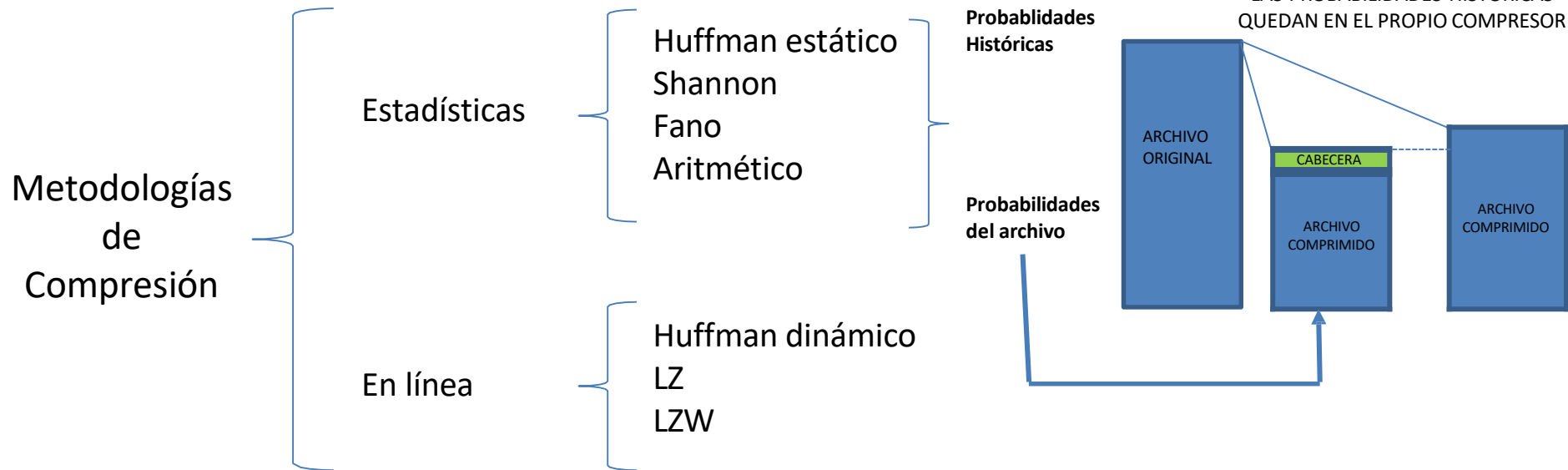
Indica que la cota inferior de cualquier longitud promedio, proveniente de la codificación de una fuente, es la entropía. Toda codificación que perfore esta cota, es una codificación con pérdida de información.

Códigos de longitud variable

El código ASCII asigna una longitud fija de 8 bits a cada uno de los 256 símbolos que lo componen (números, letras, signos de puntuación, símbolos de control, etc.)

En general, los símbolos aparecen con distinta frecuencia, por lo que es coherente buscar estrategias de codificación que contemplen la asignación de palabras código más cortas a símbolos más frecuentes, con el objeto de reducir la longitud promedio de codificación, reduciendo de este modo el tamaño del archivo.

Las metodologías de compresión reducen el tamaño del archivo.



Otra clasificación: con pérdida de información, **sin pérdida de información**

The diagram illustrates the Information Processing Model. It shows a flow from **EXTERNAL PROCESSING** to **EXTERNAL RECEPTION**. The process involves **INPUT** and **OUTPUT** stages. A feedback loop labeled **ERROR** connects the output back to the input. The central part of the diagram is labeled **THEORY OF INFORMATION**.

Es la metodología de compresión de menor longitud promedio, es decir es la que más reduce el tamaño del archivo.

La metodología sigue el siguiente proceso:

Se ordenan las probabilidades de los símbolos de mayor a menor.

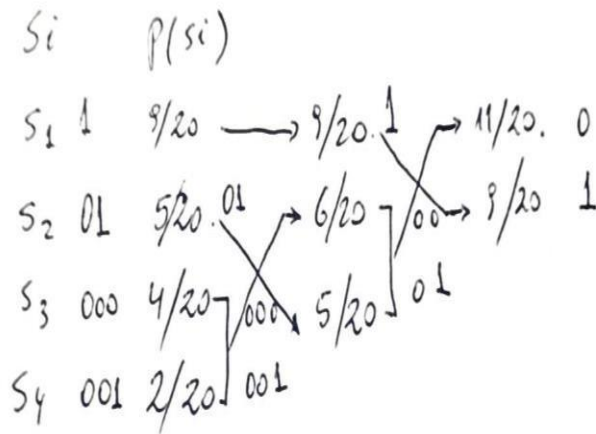
Luego, repite el proceso de asociar los r símbolos de menor probabilidad, en un nuevo símbolo, cuya probabilidad es la suma de las probabilidades de los símbolos combinados, se reordena la lista de probabilidades hasta que queden sólo r símbolos.

A los últimos r símbolos, se les asigna cada uno de los símbolos de base de codificación.

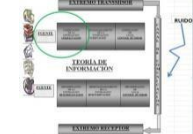
Por cada símbolo combinado, se arrastra el símbolo que resultó de la combinación, para luego volver a asignar cada uno de los símbolos de la base de codificación, el proceso se repite hasta volver a los símbolos de la fuente

Ejemplos:

s_i	$p(s_i)$
s_1	9/20
s_2	5/20
s_3	4/20
s_4	2/20

[illegible]

Huffman (2)



$$L = 9/20 \times 1 + 5/20 \times 2 + 4/20 \times 3 + 2/20 \times 3 = 1,85 \text{ bits/símbolo}$$

$$H(S) = \frac{9}{20} \log_2 \frac{20}{9} + \frac{5}{20} \log_2 \frac{20}{5} + \frac{4}{20} \log_2 \frac{20}{4} + \frac{2}{20} \log_2 \frac{20}{2} = 1,81 \text{ bits/símbolo}$$

$$H(S) \leq L$$

Codificación de la misma fuente de longitud fija:

s_i	$p(s_i)$	Código
s_1	9/20	00
s_2	5/20	01
s_3	4/20	10
s_4	2/20	11

Comparación:

- 20 símbolos a 2 bits / símbolo = 40 bits
- 20 símbolos a 1,85 bits/símbolo = 37 bits.

Eficiencia de un código, está dada por la siguiente fórmula: $\eta = \frac{H(S)}{L} \times 100\%$, y la

redundancia $R = 1 - \eta = L - H(S)/L$

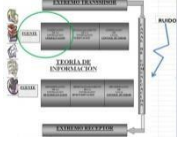
En el ejemplo: $\eta = 1,81/1,85 \times 100\% = 97,83\%$, $R = 1 - 1,81/1,85 \times 100\% = 2,16\%$

Comparado con un código de longitud fija, donde $L = 2$, $\eta =$

$1,81/2 \times 100\% = 90,5\%$, $R = 9,5\%$

De los números, $\eta = 97,83\%$ y $\eta = 90,5\%$, se deduce que la primera instancia de codificación es mucho más eficiente que la segunda, por lo que el primer código es más compacto.

Código compacto



El código más compacto de todos, debería tener una eficiencia del 100%.

Esto requiere que $H(S) = L$

Para que esto ocurra, las probabilidades de los símbolos tengan la forma:

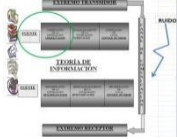
$p(s_i) = 1/r^{l_i} = r^{-l_i}$, esto implica que $l_i = \log_r 1/p(s_i)$, para $i = 1, 2, \dots, q$, siendo l_i un número entero. Este tipo de fuente es una fuente especial.

Fuente A		
s_i	$p(s_i)$	$l_i = \log_2 \frac{1}{p(s_i)}$
s_1	1/8	3
s_2	1/4	2
s_3	1/2	1
s_4	1/8	3

Fuente B		
s_i	$p(s_i)$	$l_i = \log_3 \frac{1}{p(s_i)}$
s_1	1/9	2
s_2	1/9	2
s_3	1/3	1
s_4	1/27	3
s_5	1/27	3
s_6	1/9	2
s_7	1/9	2
s_8	1/27	3
s_9	1/9	2

AÚN CUANDO NO CAMBIA L SE PREFIEREN CÓDIGOS BALANCEADOS

Teorema de la Codificación de Shannon



La ecuación $H(S) \leq L$, establece que la longitud promedio de codificación de una fuente, tiene como límite inferior la entropía, además la longitud promedio tiene como límite superior la entropía más una unidad de información, un bit en el caso de la base binaria.

Teorema de Shannon:

La longitud promedio L , de un código compacto r -ario para una fuente S , entonces:

$$H(S) \leq L < H(S) + 1$$

Sea $S = \{s_1, s_2, \dots, s_q\}$ una fuente con variedad $V = q$, se considerará un esquema de codificación sub óptima, donde la longitud de cada palabra código se elegirá de la siguiente forma:

$$\log_2 \frac{1}{p_i} \leq l_i < \log_2 \frac{1}{p_i} + 1 \quad (1)$$

Ya se vio el caso especial del código más eficiente, donde $l_i = \log_r 1/p_i$ (l_i es un entero), con probabilidades que tienen la forma $p_i = 1/r^{l_i}$, en este caso $H(S) = L$.

Ahora, cuando $p_i \neq 1/r^{l_i}$, l_i no es un número entero, por lo tanto es válido asignarle el entero inmediato superior, que corresponde al esquema propuesto por la ecuación (1).

Multiplicando cada miembro de la desigualdad por p_i :

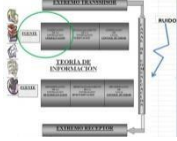
$$p_i \log_2 \frac{1}{p_i} \leq p_i l_i < p_i \log_2 \frac{1}{p_i} + p_i$$

Aplicando la sumatoria sobre todos los valores de i :

$$\sum_{i=1}^q p_i \log_2 \frac{1}{p_i} \leq \sum_{i=1}^q p_i l_i < \sum_{i=1}^q p_i \log_2 \frac{1}{p_i} + \sum_{i=1}^q p_i$$
$$H(S) \leq L < H(S) + 1$$

Esto indica que la longitud promedio de un código compacto no será mas que una unidad más grande que la longitud media del código 100% eficiente.

Código de Shannon



El código de Shannon hace uso de la inecuación (1), para asignar las longitudes de las palabras código.

Ejemplo sea la fuente, $S = \{s_1, s_2, s_3, s_4\}$, con una distribución de probabilidades: $p(s_1) = 9/20$, $p(s_2) = 5/20$, $p(s_3) = 4/20$, $p(s_4) = 2/20$.

Nº decimal			decim					
	0,7	*2		0,9	*2	pot 2		
*2	1,4	1	*2	1,8	1	-1	0,5	0,5
rest. a 0	0,4		rest. a 0	0,8				
*2	0,8	0	*2	1,6	1	-2	0,25	0,25
	1,6	1	rest. a 0	0,6				
rest. a 0	0,6		*2	1,2	1	-3	0,125	0,125
*2	1,2	1	rest. a 0	0,2				
rest. a 0	0,2		*2	0,4	0	-4	0,0625	0
*2	0,4	0	*2	0,8	0	-5	0,03125	0
*2	0,8	0	*2	1,6	1	-6	0,015625	0,015625
*2	1,6	1	rest. a 0	0,6				
rest. a 0	0,6		*2	1,2	1	-7	0,0078125	0,007813
*2	1,2	1	rest. a 0	0,2				
			*2	0,4	0	-8	0,0039063	0
								0,898438
0,7	10110011		0,9	11100110				

$$\log_2 20/9 \leq l_1 < \log_2 20/9 + 1, \therefore l_1 = 2$$

$$\log_2 20/5 \leq l_2 < \log_2 20/5 + 1, \therefore l_2 = 2$$

$$\log_2 20/4 \leq l_3 < \log_2 20/4 + 1, \therefore l_3 = 3$$

$$\log_2 20/2 \leq l_4 < \log_2 20/2 + 1, \therefore l_4 = 4$$

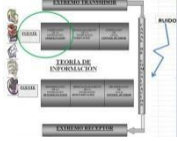
s_i	$p(s_i)$	FA	Decimal	Binario	Longitud	Código
s_1	9/20	0	0	0	2	00
s_2	5/20	9/20	0,45	0.01	2	01
s_3	4/20	14/20	0,70	0.101	3	101
s_4	2/20	18/20	0,90	0.1110	4	1110

$$L_{SH} = \frac{9}{20} \times 2 + \frac{5}{20} \times 2 + \frac{4}{20} \times 3 + \frac{2}{20} \times 4 = 2.4 \text{ bits/símbolo}$$

Recordando , $L_H = 1,85 \text{ bits/símbolo}$. $L_H \leq L_{SH}$

$$\eta = H(S)/L_{SH} = 1,81/2,4 \times 100\% = 75,41\%$$

Código de Fano



La codificación de Fano, se basa en la idea de que los símbolos equiprobables deberían corresponderse con palabras código de igual longitud.

Fano es un método para diseñar códigos r – *arios* instantáneos, la base de codificación tiene r símbolos de la forma x_i .

Sea $S = \{s_1, s_2, \dots, s_q\}$, una fuente con variedad $V = q$, con una distribución de probabilidades $P = \{p(s_1), p(s_2), \dots, p(s_q)\}$.

Se ordenan los símbolos de acuerdo a las probabilidades de mayor a menor, $p(s_1) \geq p(s_2) \geq \dots \geq p(s_q)$.

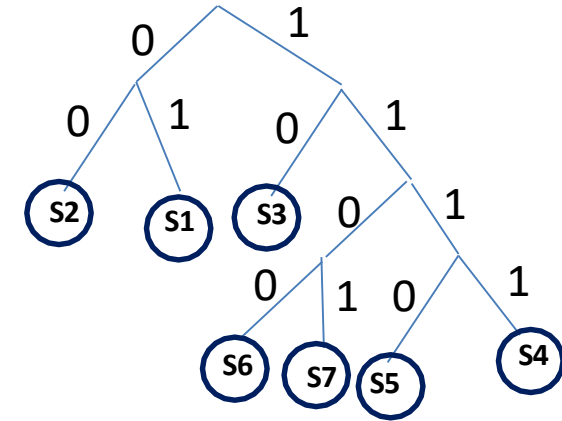
En el método de Fano, los q símbolos se dividen en r grupos equiprobables.

A cada símbolo del i -ésimo grupo se le asigna la palabra código x_i , esto se hace para todos los grupos, $i = 1, 2, \dots, r$.

Cada uno de los r grupos, se subdivide en r subgrupos asignando la palabra código x_i al i – *ésimo* subgrupo, el proceso se repite hasta que no se pueden dividir los grupos.

Código de Fano (2)

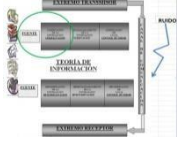
Cod.	S	Prob						
01	S1	9/27					1	0
00	S2	7/27					0	1
10	S3	5/27					0	3
1111	S4	2/27	1					
1110	S5	2/27	0	5	1		1	
1100	S6	1/27	0			4	1	
1101	S7	1/27	1	6	0			



$$L = \frac{9}{27} \times 2 + \frac{7}{27} \times 2 + \frac{5}{27} \times 2 + \frac{2}{27} \times 4 + \frac{2}{27} \times 4 + \frac{1}{27} \times 4 + \frac{1}{27} \times 4 = \frac{66}{27} = 2,44 \text{ bits/símbolo}$$

$$H_F(S) = 2,392 \text{ bits/símbolo}, \eta = 2,392/2,444 \times 100\% = 97,87\%$$

Entonces....Sobre cada porción probabilística considerada, se suma desde los menos probable Hasta lograr la mayor aproximación a la mitad de la porción considerada (se por exceso o por defecto) se asigna un 0 a una porción y un 1 a la otra. Así, recursivamente, se repite el proceso hasta finalizar obteniendo un código unívoco e instantáneo.

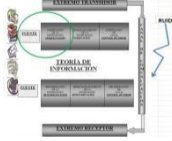


Relación entre longitudes promedio Huffman-Fano-Shannon

$$L_{Huffman} \leq L_{Fano} \leq L_{Shannon}$$

La igualdad se da cuando las probabilidades de los símbolos son de la forma: $p(s_i) = \frac{1}{r^{l_i}}$

Compresión de una extensión de orden n



Sea $S^n = \{\sigma_1^n, \sigma_2^n, \dots, \sigma_{q^n}^n\}$, la extensión de orden n de la fuente $S = \{s_1, s_2, \dots, s_q\}$, donde $\sigma_i^n = s_{i1}s_{i2} \dots s_{in}$.

Se considera el código compacto de S^n con longitud media L_n , recordando el primer teorema de Shannon: $H(S) \leq L < H(S) + 1$, luego se tiene:

$$H(S^n) \leq L_n < H(S^n) + 1 \quad (1)$$

Como se vio anteriormente $H(S^n)$, se puede considerar como la entropía de n fuentes independientes, ya que: $p(\sigma_i^n) = p(s_{i1}) \times p(s_{i2}) \times \dots \times p(s_{in})$.

Por la fórmula de la entropía de una extensión de orden n de la fuente S : $H(S^n) = nH(S)$.

Dividiendo la inecuación (1) por n :

$$\begin{aligned} \frac{H(S^n)}{n} &\leq \frac{L_n}{n} < \frac{H(S^n)}{n} + \frac{1}{n} \\ \frac{nH(S)}{n} &\leq \frac{L_n}{n} < \frac{nH(S)}{n} + \frac{1}{n} \\ H(S) &\leq \frac{L_n}{n} < H(S) + \frac{1}{n} \end{aligned} \quad (2)$$

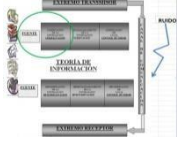
L_n/n es la longitud media de las palabras código por símbolo s_i cuando se codifica la extensión de orden n de la fuente S .

$L_n \neq L$, sin embargo L_n/n puede usarse como longitud media de un esquema de codificación de una fuente S (basada en la codificación de la n – ésima extensión de la fuente).

Como puede verse en la inecuación (2) la longitud media de un código compacto para S basado en la codificación de la n – ésima extensión de la fuente S difiere en $1/n$ unidades respecto de la longitud media de un código 100% eficiente y esta sobrecarga disminuye con extensiones más grandes.

Intuitivamente, se espera que la codificación de la extensión de orden n de la fuente proveerá códigos más eficientes, logrando el 100% de eficiencia cuando $n \rightarrow \infty$.

Teorema de la codificación de Shannon



Sea L la longitud media de un código compacto para la fuente S , luego:

$$H(S) \leq L < H(S) + 1$$

Sea L_n la longitud media de un código compacto para la n – *ésima* extensión de la fuente S , luego:

$$H(S) \leq \frac{L_n}{n} < H(S) + \frac{1}{n}$$

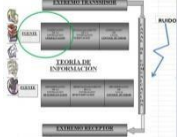
Entonces:

$$\lim_{n \rightarrow \infty} \frac{L_n}{n} = H(S)$$

Esto es, en el límite, el código es 100% eficiente.

Por lo tanto, **es conveniente codificar extensiones de código.**

Ejemplo – codificación de la extensión de orden 2 de una fuente



Sea la fuente $S = \{s_1, s_2\}$, con una distribución de probabilidades $P = \{p(s_1), p(s_2)\} = \{8/10, 2/10\}$

s_i	$p(s_i)$	Código
s_1	8/10	0
s_2	2/10	1

σ_i	$p(\sigma_i)$	Código
s_1s_1	64/100	0
s_1s_2	16/100	11
s_2s_1	16/100	100
s_2s_2	4/100	101

σ_i	$p(\sigma_i)$	Código
$s_1s_1s_1$	512/1000	0
$s_1s_1s_2$	128/1000	100
$s_1s_2s_1$	128/1000	101
$s_2s_1s_1$	128/1000	110
$s_1s_2s_2$	32/1000	11100
$s_2s_1s_2$	32/1000	11101
$s_2s_2s_1$	32/1000	11110
$s_2s_2s_2$	8/1000	11111

$$L = 1, H(S) = 0,722 \text{ bits/símbolo}$$

$$\eta = H(S)/L \times 100\% = 72,20\%$$

Si se codifica la extensión de orden 2, $\sigma_i = s_i s_j$, con $p(\sigma_i) = p(s_i) \times p(s_j)$ para $i, j = 1, 2$

$$L_2 = 1,56 \text{ bits/símbolo}$$

$$H(S^2) = 2 \times H(S) = 1,444 \text{ bits/símbolo}$$

$$\eta = \frac{H(S^2)/2}{L_2/2} \times 100\% = \frac{0,722}{1,56/2} \times 100\% = 92,56\%$$

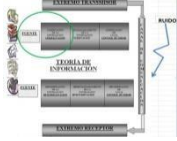
Puede observarse del ejemplo que 92,56% es una mejora, respecto del 72,20%.

$$L_3 = 2,184 \text{ bits/símbolo}$$

$$H(S^3) = 3 \times H(S) = 2,1666 \text{ bits/símbolo}$$

$$\eta = \frac{H(S^3)}{L_3} \times 100\% = \frac{2,1666}{2,184} \times 100\% = 99,17\%$$

Ejemplo – codificación de la extensión de orden 2 de una fuente



σ_i	$p(\sigma_i)$	Código
s_1	5/10	0
s_2	3/10	10
s_3	2/10	11

$$L = \frac{5}{10} * 1 + \frac{3}{10} * 2 + \frac{2}{10} * 1,5, \frac{\text{bits}}{\text{símbolo}}, H(S) = 1,485 \text{ bits de inf promedio}$$

$$\eta = H(S)/L \times 100\% = 99,0316\%$$

Si se codifica la extensión de orden 3, $\sigma_i = s_i s_j s_l$, con $p(\sigma_i) = p(s_i) \times p(s_j) \times p(s_l)$ para $i, j, l = 1, 2, 3$

σ_i	$p(\sigma_i)$	Código	σ_i	$p(\sigma_i)$	Código	σ_i	$p(\sigma_i)$	Código
$s_1 s_1 s_1$	125/1000	?	$s_2 s_1 s_1$	75/1000		$s_3 s_1 s_1$	50/1000	
$s_1 s_1 s_2$	75/1000	?	$s_2 s_1 s_2$	45/1000		$s_3 s_1 s_2$	30/1000	
$s_1 s_1 s_3$	50/1000		$s_2 s_1 s_3$	30/1000		$s_3 s_1 s_3$	20/1000	
$s_1 s_2 s_1$	75/1000		$s_2 s_2 s_1$	45/1000		$s_3 s_2 s_1$	30/1000	
$s_1 s_2 s_2$	45/1000		$s_2 s_2 s_2$	27/1000		$s_3 s_2 s_2$	18/1000	
$s_1 s_2 s_3$	30/1000		$s_2 s_2 s_3$	18/1000		$s_3 s_2 s_3$	12/1000	
$s_1 s_3 s_1$	50/1000		$s_2 s_3 s_1$	30/1000		$s_3 s_3 s_1$	20/1000	m
$s_1 s_3 s_2$	30/1000		$s_2 s_3 s_2$	18/1000		$s_3 s_3 s_2$	12/1000	
$s_1 s_3 s_3$	20/1000		$s_2 s_3 s_3$	12/1000		$s_3 s_3 s_3$	8/1000	

$$L(S_3) = \text{????}; H(S_3) = 4,455$$

s_1 s_1 s_1	125/1000	[1,1] = 0 1 1
s_1 s_1 s_2	75/1000	[1,2] = 0 0 1 0
s_1 s_2 s_1	75/1000	[1,3] = 0 0 1 1
s_2 s_1 s_1	75/1000	[1,4] = 0 1 0 0
s_1 s_1 s_3	50/1000	[1,5] = 1 0 1 1
s_1 s_3 s_1	50/1000	[1,6] = 1 1 0 0
s_3 s_1 s_1	50/1000	[1,7] = 1 1 0 1
s_1 s_2 s_2	45/1000	[1,8] = 1 1 1 0
s_2 s_1 s_2	45/1000	[1,9] = 1 1 1 1
s_2 s_2 s_1	45/1000	[1,10] = 0 0 0 0 0
s_1 s_2 s_3	30/1000	[1,11] = 0 1 0 1 1
s_1 s_3 s_2	30/1000	[1,12] = 1 0 0 0 0
s_2 s_1 s_3	30/1000	[1,13] = 1 0 0 0 1
s_2 s_3 s_1	30/1000	[1,14] = 1 0 0 1 0
s_3 s_1 s_2	30/1000	[1,15] = 1 0 0 1 1
s_3 s_2 s_1	30/1000	[1,16] = 1 0 1 0 0
s_2 s_2 s_2	27/1000	[1,17] = 1 0 1 0 1
s_1 s_3 s_3	20/1000	[1,18] = 0 0 0 1 0 0
s_3 s_1 s_3	20/1000	[1,19] = 0 0 0 1 0 1
s_3 s_3 s_1	20/1000	[1,20] = 0 0 0 1 1 0
s_2 s_2 s_3	18/1000	[1,21] = 0 0 0 1 1 1
s_2 s_3 s_2	18/1000	[1,22] = 0 1 0 1 0 0
s_3 s_2 s_2	18/1000	[1,23] = 0 1 0 1 0 1
s_2 s_3 s_3	12/1000	[1,24] = 0 0 0 0 1 0 0
s_3 s_2 s_3	12/1000	[1,25] = 0 0 0 0 1 0 1
s_3 s_3 s_2	12/1000	[1,26] = 0 0 0 0 1 1 0
s_3 s_3 s_3	8/1000	[1,27] = 0 0 0 0 1 1 1

```

prob =

Columns 1 through 18:

    1.2500e-01    7.5000e-02    7.5000e-02    7.5000e-02    5.0000e-02    5.0000e-02    5.0000e-02    4.5000e-02    4.5000e-02
    4.5000e-02    3.0000e-02    3.0000e-02    3.0000e-02    3.0000e-02    3.0000e-02    3.0000e-02    2.7000e-02
    2.0000e-02

Columns 19 through 27:

    2.0000e-02    2.0000e-02    1.8000e-02    1.8000e-02    1.8000e-02    1.2000e-02    1.2000e-02    1.2000e-02    8.0000e-03

>> huffmandict (1, prob, 0, true)
ans =

```

$$L = (125 \cdot 3 + (75 \cdot 3 + 50 \cdot 3 + 45 \cdot 2) \cdot 4 + (45 + 30 \cdot 6 + 27) \cdot 5 + (20 \cdot 3 + 18 \cdot 3) \cdot 6 + (12 \cdot 3 + 8) \cdot 7) / 1000$$

$$4,487 = L$$

$$\eta = \frac{H(S^3)}{L_3} \times 100\% = \frac{4,455}{4,487} \times 100\% = 99,28682\%$$

Advertencia: Si bien , se pone de manifiesto un mejor rendimiento al codificar la extensión de orden 3 por sobre los símbolos individuales, la mejora no es lo suficientemente atractiva, para esta distribución inicial, atento a la necesidad de agregar un encabezado (al relevar la estadística local) en el producto comprimido que haga perder lo ahorrado al codificar extensiones.

Sea una fuente que genera vocales en la siguiente secuencia: a"aeiouaaaeouaiaaeoua"a

Un código de longitud fija demanda tres bits por símbolo y ocuparía 60 bits la secuencia

CODIFICACIÓN ARITMÉTICA

p(a)=8/20
p(e)=3/20
p(i)=2/20
p(o)=4/20
p(u)=3/20

LH=44/20

HUFFMAN



Para la secuencia planteada mediante huffman y para el total de la cadena se utilizan 44bits (dos menos por cada a)

Si quisieramos especificar hasta la primer letra u de la secuencia, cuantos bits se utilizaron?, sería: 13bits

1	a	1	a	aa	0.76	a	aea	0.706	a	aeia	0.7060	a	aeioa	0.70360	0.70270	*2	suma potencias de 2
	a		a	aa		a	aea		a	aeia		a	aeioa		1.40540	1	0.5
	a		a	aa		a	aea		a	aeia		a	aeioa		0.40540		
	a		a	aa		a	aea		a	aeia		a	aeioa		0.81080	0	
	a		a	aa		a	aea		a	aeia		a	aeioa		1.62160	1	0.125
	a		a	aa		a	aea		a	aeia		a	aeioa		0.62160		
	a		a	aa		a	aea		a	aeia		a	aeioa		1.24320	1	0.0625
	a		a	aa		a	aea		a	aeia		a	aeioa		0.24320		
0.60	o	0.84	o	ao	0.736	o	aeo	0.7036	o	aeio	0.70312	o	aeioo	0.48640	0		
	o		o	ao		o	aeo		o	aeio		o	aeioo	0.97280	0		
	o		o	ao		o	aeo		o	aeio		o	aeioo	1.94560	1	0.0078125	0,6953125
	o		o	ao		o	aeo		o	aeio		o	aeioo	0.94560			
0.40	e	0.76	e	ae	0.724	e	aeie	0.7024	e	aeioe	0.70288	e	aeioee	1.89120	1	0,00390625	
	e		e	ae		e	aeie		e	aeioe		e	aeioee	0.89120			
	e		e	ae		e	aeie		e	aeioe		e	aeioee	1.78240	1	0,001953125	0,701171875
0.25	u	0.70	u	au	0.715	u	aeu	0.7015	u	aeiou	0.70270	u	aeiouu	0.70270			
	u		u	au		u	aeu		u	aeiou		u	aeiouu	1.56480	1	0,0009765625	0,7021484425
	u		u	au		u	aeu		u	aeiou		u	aeiouu	0.56480			
0.1	i	0.64	i	ai	0.706	i	aei	0.7006	i	aeioi	0.70252	i	aeioii	1.12960	1	0,00048828125	0,70263672375
0	i	0.60	i	ai	0.700	i	aei	0.7000	i	aeioi	0.70240						
0.05		0.02			0.003				0.0003		0.0006						

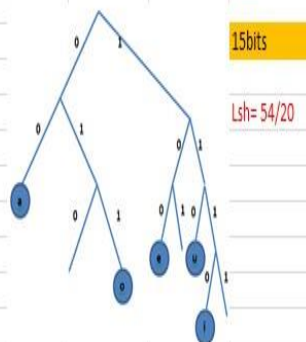
SHANNON

	FR	FA	FA decimal	FA base2	CANT. INF.	li	CÓDIGO
a	8/20	0	0	0.000	1.321	2	00
o	4/20	8/20	0.4	0.01100	2.321	3	011
e	3/20	12/20	0.60	0.1001	2.736	3	100
u	3/20	15/20	0.75	0.1100	2.736	3	110
i	2/20	18/20	0.90	0.11100	3.321	4	1110

conversión a base dos

	*2
0.4	
0.8	0
1.6	1
0.6	
1.2	1
0.2	
0.4	0
0.8	0

lectura



15bits

Lsh= 54/20

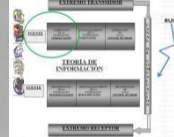
La idea de la codificación aritmética es codificar una cadena de caracteres con una secuencia de bits que es un valor numérico decimal, codificado en binario, que representa un valor que se encuentra entre la cota inferior y superior de la probabilidad de esa secuencia de símbolos. así:
Cualquier valor decimal codificado en binario que se encuentre entre 0.60 y 1 será interpretado como la letra a
Cualquier valor decimal codificado en binario que se encuentre entre 0.70 y 0.76 será interpretado como la secuencia ae
Cualquier valor decimal codificado en binario que se encuentre entre 0.70 y 0.706 será interpretado como la secuencia aei
Cualquier valor decimal codificado en binario que se encuentre entre 0.7024 y 0.7036 será interpretado como la secuencia aeio
para finalmente:
Cualquier valor decimal codificado en binario que se encuentre entre 0.70252 y 0.70270 será interpretado como la secuencia aeioi
Entonces para lograr el código traduciremos a base dos la cota superior del segmento considerado (el 0.70270) y dado que la aproximación binaria a cualquier base es por defecto (se va aproximando al valor buscado, sin alcanzarlo, conforme mayor cantidad de dígitos tiene la aproximación) nos detendremos cuando hayamos superado la cota inferior!!!

cota superior 0.70270

sec. Símbolos
aeiou
Código Binario
10110011111
0,70263672375

cota inferior 0.70252

se usan 11 bits



Sea la siguiente secuencia de tres letras "a" 3 a { a a b a c a b b c b c c a a a }

Cod. pseudo		probabilidades	Cod. Huffman(1)
a	00	$p(a) = 7/15$	a = 011
b	01	$p(b) = 4/15$	b = 10
c	10	$p(c) = 4/15$	c = 11
EOF	11		

secuencia a analizar

$$H = \frac{7}{15} \times \log_2 \frac{15}{7} + 2 \times \frac{4}{15} \log_2 \frac{15}{4} = 1.53 \text{ bit/prom}$$

archivo en código ASCII: a a b a c a b b c b c c a a a = 30 bits $L = 2 \text{ bit promedio}$
archivo comprimido: 0010011010101101100000 = 23 bits $L = \frac{23}{15} = 1.53 \text{ bits promedio}$

ahora: ¿cómo enviamos los códigos al extremo receptor a los efectos de que pueda decodificar?

Por cada símbolo original enviamos: pseudo ASCII + long. N° código + N° código

pseudo a	00	pseudo b	01	pseudo c	10
long. N° código	01(1)	long. N° código	10(2)	long. N° código	10(2)
N° código	0	N° código	10	N° código	11

arch comp: 000100110101010110010010110101011101111000
 a b c a a b a c a b b c b c c a a a

cabecera

archivo (código original) comprimido

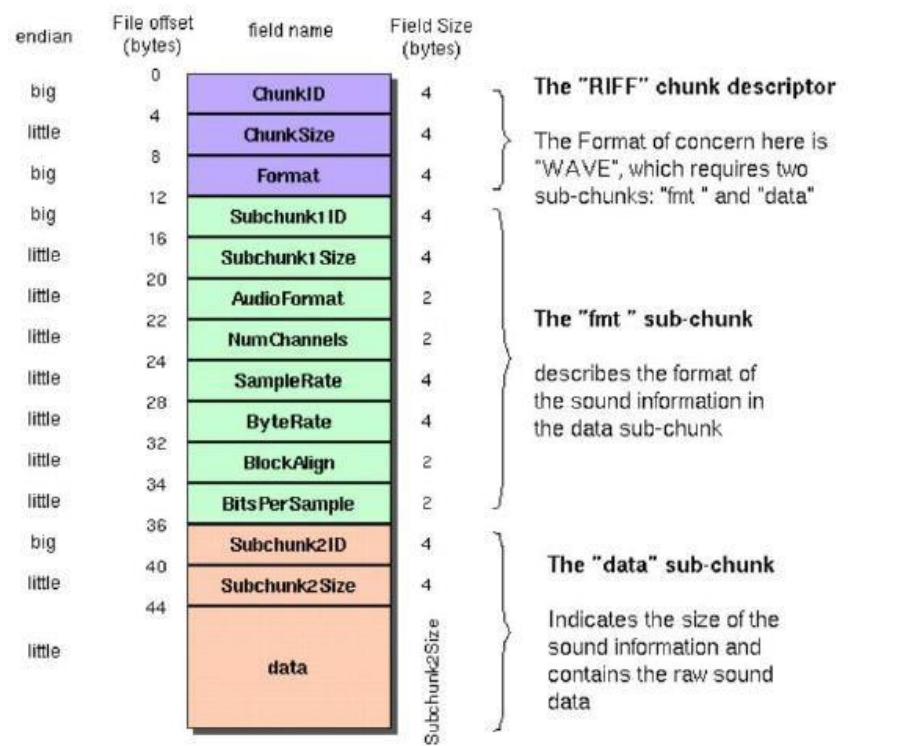
ahora: cómo distinguir entre la cabecera y los datos del archivo original?
se debe colocar como parte de la cabecera: 1) la variedad 0 que especifica los símbolos codificados y la long. del archivo original en una cantidad de bits predeterminados
ej: con un pseudobyte podemos especificar "0" o 11 que se leerá como "3"
y con 3 pseudobytes por ej; especifica la long. del archivo original 001111(15)

0011111100010011010101011001001101010111011110000
a EOF EOF EOF a b a EOF b b b b c b a EOF b b b EOF b EOF c a

leído como 5a; 7 EOF, 10 b, 2c Entropía: $\frac{5}{24} \log_2 \frac{24}{5} + \frac{10}{24} \log_2 \frac{24}{10} + \frac{7}{24} \log_2 \frac{24}{7} + \frac{2}{24} \log_2 \frac{24}{2} = 1.813 \text{ bits promedio}$

Las cabeceras de los archivos poseen información necesaria y relevante, que permite al reproductor o decodificador del formato, rescatar la totalidad del mensaje codificado originalmente. Esto sucede en todo tipo de ormato y archivo. En este caso se presenta la cabecera asociada a un archivo de audio tipo WAV.

The Canonical WAVE file format



OFFSET	SIZE	NAME	DESCRIPTION
The canonical WAVE format starts with the RIFF header:			
0	4	ChunkID	Contains the letters "RIFF" in ASCII form (0x52494646 big-endian form).
4	4	ChunkSize	36 + SubChunk2Size, or more precisely: 4 + (8 + SubChunk1Size) + (8 + SubChunk2Size) This is the size of the rest of the chunk following this number. This is the size of the entire file in bytes minus 8 bytes for the two fields not included in this count: ChunkID and ChunkSize.
8	4	Format	Contains the letters "WAVE" (0x57415645 big-endian form)

The "WAVE" format consists of two subchunks: "fmt" and "data":			
The "fmt" subchunk describes the sound data's format:			
12	4	Subchunk1ID	Contains the letters "fmt" (0x66667420 big-endian form)
16	4	Subchunk1Size	16 for PCM. This is the size of the rest of the Subchunk which follows this number
20	2	AudioFormat	PCM = 1 (i.e. Linear quantization) Values other than 1 indicate some form of compression
22	2	NumChannels	Mono = 1, Stereo = 2, etc
24	4	SampleRate	8000, 44100, etc
28	4	ByteRate	$\text{SampleRate} * \text{NumChannels} * \text{BitsPerSample} / 8$
32	2	BlockAlign	$\text{NumChannels} * \text{BitsPerSample} / 8$ The number of bytes for one sample including all channels. I wonder what happens when this number isn't an integer?
34	2	BitsPerSample	8 bits = 8, 16 bits = 16, etc
	2	ExtraParamSize	if PCM, then doesn't exist
	x	ExtraParams	space for extra parameters

The "data" subchunk contains the size of the data and the actual sound:			
36	4	Subchunk2ID	Contains the letters "data" (0x64617461 big-endian form)
40	4	Subchunk2Size	$\text{NumSamples} * \text{NumChannels} * \text{BitsPerSample} / 8$ This is the number of bytes in the data. You can also think of this as the size of the rest of the subchunk following this number.
44	*	Data	The actual sound data

MARKOV

Sea una fuente que genera tres letras a, b y c q=3 y la secuencia relevada es: **a-aabbccacbaaaaba-a**

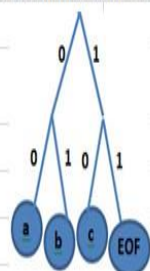
Como fuente Markoviana:

Grafo de Transición Entre Estados

MARKOV

Consideremos un código pseudo ascii de igual longitud para codificar tres cosas distintas

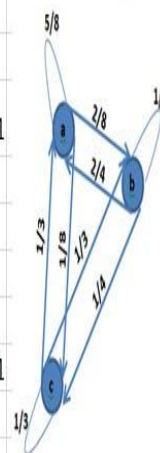
Símbolo	Código
a	00
b	01
c	10
EOF	11



aabbccacbaaaaba
Pseudo Ascii **000001011010001001000000000100**

Matriz de Transición Entre Estados

	a	b	c	t+1
t	a	p(a/a)	p(b/a)	p(c/a)
	b	p(a/b)	p(b/b)	p(c/b)
	c	p(a/c)	p(b/c)	p(c/c)

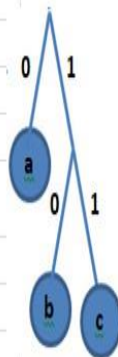


La probabilidad de que la fuente genere la secuencia por ej **p(abcabc)** es:

$$p(a)*p(a/a)*p(b/a)*p(c/b)*p(a/c)*p(b/a)*p(c/b) + p(b)*p(a/b)*p(b/a)*p(c/b)*p(a/c)*p(b/a)*p(c/b) + p(c)*p(a/c)*p(b/a)*p(c/b)*p(a/c)*p(b/a)*p(c/b) = (p(aa)+p(ba)+p(ca))*p(b/a)*p(c/b)*p(a/c)*p(b/a)*p(c/b) = 0.00199$$

Como fuente independiente:

Símbolo	Cód. Huffman
a	0
b	10
c	11



Cód. Huffman **0010101111011100000100**

leído en ascii **accEOFEOFbEOFaaba**

H(S)=1.4564 bits de inf prom

Entropía en fuente Markoviana de orden M: $H(S^M) = \sum_{j=1}^m p(s_{j1}, s_{j2}, \dots, s_{jm}) \sum_{i=1}^q p(s_i/s_{j1}, s_{j2}, \dots, s_{jm}) \log_2 \frac{1}{p(s_i/s_{j1}, s_{j2}, \dots, s_{jm})}$

$H(SM) = 8/15*(5/8*\log(8/5)+2/8*\log(8/2)+1/8*\log(8))+4/15*(2/4*\log(2)+2*(1/4*\log(4))+3/15*(3*1/3*\log(3)) = 0.9429 \text{ bit}$

Obtendré por cada estado una codificación

Estado a	Cod a tras a 0	Estado b	Cod a tras b 1	Estado c	Cod a tras c 0
	Cod b tras a 10		Cod b tras b 00		Cod b tras c 10
	Cod c tras a 11		Cod c tras b 01		Cod c tras c 11

$$I_a = 5/8*1 + 2/8*2 + 1/8*2 = 11/8$$

$$I_b = 2/4*1 + 1/4*(2+2) = 6/4$$

$$I_c = 1/3*(1+2+2) = 5/3$$

$$LM = p(a)*I_a + p(b)*I_b + p(c)*I_c = 8/15*11/8 + 4/15*6/4 + 3/15*5/3 = 22/15$$

LMarkov ≤ Lind

Desde la Matriz de transición y basado en las siguientes ecuaciones, se pueden encontrar los valores de las p(Si) desde las p(Sj/Si)

$$\begin{aligned} p(a)t+1 &= p(a)t*p(a/a) + p(b)t*p(a/b) + p(c)t*p(a/c) \\ p(b)t+1 &= p(a)t*p(b/a) + p(b)t*p(b/b) + p(c)t*p(b/c) \\ p(c)t+1 &= p(a)t*p(c/a) + p(b)t*p(c/b) + p(c)t*p(c/c) \\ 1 &= p(a)t + p(b)t + p(c)t \end{aligned}$$

Sistema de tres ecuaciones con tres incógnitas desde donde se despejan los valores de p(Si)t que si el proceso es estacionario

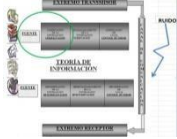
$$\begin{aligned} p(Si)t &= p(Si)t+1 \\ p(a) &= 8/15 \\ p(b) &= 4/15 \\ p(c) &= 3/15 \end{aligned}$$

Otra forma de obtener las probabilidades independientes desde las condicionales es elevar la Matriz de Transición a una potencia n lo suficientemente grande, que ya no admita modificaciones en su resultado.

Esta Matriz se denomina Matriz Estacionaria, y se caracteriza por tener en cada columna los valores de pas p(Si)

$$MT^n = \begin{bmatrix} 5/8 & 2/8 & 1/8 \\ 2/4 & 1/4 & 1/4 \\ 1/3 & 1/3 & 1/3 \end{bmatrix}^n = \begin{bmatrix} 8/15 & 4/15 & 3/15 \\ 8/15 & 4/15 & 3/15 \\ 8/15 & 4/15 & 3/15 \end{bmatrix} = ME$$

Codificación de Fuentes de Markov(1)



Dada la fuente $S = \{A, E, I, O\}$, y la siguiente secuencia generada a partir de dicha fuente, por una fuente de Markov de orden uno:

...AEEEEEEEOOIIAAOOOEEEA|A...

Cuya matriz de transición de estados será:

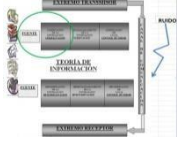
$$\begin{array}{c|cccc} & A & E & I & O \\ \hline A & 2/4 & 1/4 & 0 & 1/4 \\ E & 1/9 & 7/9 & 0 & 1/9 \\ I & 1/2 & 0 & 1/2 & 0 \\ O & 0 & 1/5 & 1/5 & 3/5 \end{array}$$

A efectos de comparación se genera un código bloque de longitud fija para los cuatro símbolos:

s_i	Código
s_1	00
s_2	01
s_3	10
s_4	11

$$L = 2, H(S) = 1,8149$$

Codificación de Fuentes de Markov(2)



Se propone considerar a la fuente como una fuente de Markov de orden $m = 1$.

Se generará un código para cada símbolo, que es función del estado anterior en que se encontraba la fuente, por lo tanto se tendrán cuatro códigos distintos para símbolo.

A modo de ejemplo, se utilizará para la codificación de las transiciones, al código de Huffman:

Estado actual A		
$p(A/A)$	2/4	1
$p(E/A)$	1/4	00
$p(I/A)$	1/4	10
$p(O/A)$	0	

Estado actual I		
$p(A/I)$	1/2	0
$p(E/I)$	0	-
$p(I/I)$	1/2	1
$p(O/I)$	0	-

Estado actual E		
$p(A/E)$	1/9	10
$p(E/E)$	7/9	0
$p(I/E)$	0	-
$p(O/E)$	1/9	11

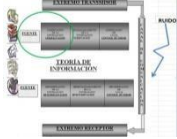
Estado actual O		
$p(A/O)$	0	-
$p(E/O)$	1/5	10
$p(I/O)$	1/5	11
$p(O/O)$	3/5	0

La compresión requiere el estado inicial, por lo tanto se codifica con el código bloque el primer símbolo, en este caso la letra A con el código bloque propuesto 00.

Luego se parte del estado actual y se codifica la transición de A a E, usando el código correspondiente de la primer tabla, esto es 00.

Estando en el estado E, se codifica la transición del estado E al estado E, haciendo uso del código de la segunda tabla, esto es 0, y así siguiendo.

Codificación de Fuentes de Markov(3)



A	E	E	E	E	E	E	O	O	I	I	A	A	O	O	O	E	E	E	A
00	00	0	0	0	0	0	11	0	11	1	0	1	01	0	0	10	0	0	10

Tamaño del archivo original: 20 *símbolos*, 2 *bits por símbolo*, 40 *bits*.

Tamaño del archivo comprimido Huffman Indep.: 27 *bits*.

Cálculo de la longitud promedio, para ello se utiliza, para cada estado, la fórmula:

$$L_M(s_i) = \sum_{j=1}^{q^m} l(s_j) p(s_j/s_i)$$

$$L_M(A) = l(A)p(A/A) + l(E)p(E/A) + l(I)p(I/A) + l(O)p(O/A) = 1,5 \text{ bits/símbolo}$$

$$L_M(E) = 1,22 \text{ bits/símbolo}$$

$$L_M(I) = 1 \text{ bit/símbolo}$$

$$L_M(O) = 1,4 \text{ bits/símbolo}$$

$$L_M = \sum_{i=1}^{q^m} p(s_i) L_M(s_i) = 1,3 \text{ bits/símbolo} = \frac{4}{20} * 1,5 + \frac{9}{20} * 1,22 + \frac{2}{20} * 1 + \frac{5}{20} * 1,4 = 25,98/20 = 1,299$$

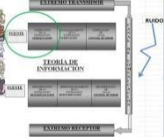
$$H(S^M) = \sum_{j=1}^M p(s_{j1}, s_{j2}, \dots, s_{jm}) \sum_{i=1}^q p(s_i/s_{j1}, s_{j2}, \dots, s_{jm}) \log_2 \frac{1}{p(s_i/s_{j1}, s_{j2}, \dots, s_{jm})}$$

$$H(S^M) = p(A) \left[p(A/A) \log_2 \frac{1}{p(A/A)} + p(E/A) \log_2 \frac{1}{p(E/A)} + p(O/A) \log_2 \frac{1}{p(O/A)} \right] + \dots$$

$$+ p(O) \left[p(E/O) \log_2 \frac{1}{p(E/O)} + p(I/O) \log_2 \frac{1}{p(I/O)} + p(O/O) \log_2 \frac{1}{p(O/O)} \right] = 1,186 \text{ bits/símbolo}$$

Para la misma fuente $H(S) = 1,81 \text{ bits/símbolo}$, $\therefore H(S^M) \leq H(S)$

Codificación de Fuentes de Markov(4)



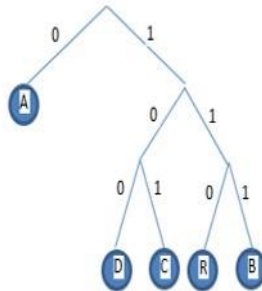
Viendo la fuente como símbolos Dependientes De PRIMER ORDEN Codificar por huffman:

BRA-*ABRACADABRA*-ABR

PSEUDOASCI

A	000
B	001
R	010
C	011
D	100

```
>> hd
hd =
{
  [1,1] = 0
  [1,2] =
    1 1 1
  [1,3] =
    1 1 0
  [1,4] =
    1 0 1
  [1,5] =
    1 0 0
}
```



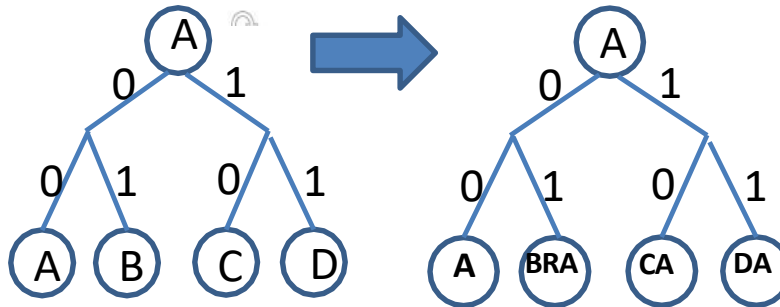
ÁRBOL BALANCEADO

$$L = (5*1 + (2+2+1+1)*3) / 11 = 23/11$$

	A	B	R	C	D
A	1/5	2/5	0	1/5	1/5
B	0	0	1	0	0
R	1	0	0	0	0
C	1	0	0	0	0
D	1	0	0	0	0

COD.

A→B	01	2/5	2/5	3/5	0
A→A	00	1/5	2/5	2/5	1
A→C	10	1/5	1/5		
A→D	11	1/5			



A	A	B	R	A	C	A	D	A	B	R	A	A
	000	001	010	000	011	000	100	000	001	010	000	33 PSEUDOASCI
	0	111	110	0	101	0	100	0	111	110	0	23 HUFFMAN INDEPENDIENTE
	000	01		10		11		01				11 HUFFMAN DEPENDIENTE ORDEN 1

Entrada de matriz

Inserte Matriz Restaurar matriz

☐ Números complejos (más)

Decimal

	A ₁	A ₂	A ₃
1	0.625	0.25	0.125
2	0.5	0.25	0.25
3	0.3333	0.333	0.3333

Limpiar Rellenar celdas vacías con ceros

Calcular

Dimensión de la matriz: 3 Potencia: 2

$$P(A_1) = P(a) = 8/15 = 0.53333$$

$$P(A_2) = P(b) = 4/15 = 0.26666$$

$$P(A_3) = P(c) = 3/15 = 0.2$$

$$P(A_1) = P(A) = 4/20 = 0.2$$

$$P(A_2) = P(E) = 9/20 = 0.45$$

$$P(A_3) = P(I) = 2/20 = 0.1$$

$$P(A_4) = P(O) = 5/20 = 0.25$$

Entrada de matriz

Inserte Matriz Restaurar matriz

☐ Números complejos (más)

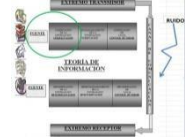
Decimal

	A ₁	A ₂	A ₃	A ₄
1	0.5	0.25	0	0.25
2	0.1111	0.7777	0	0.1111
3	0.5	0	0.5	0
4	0	0.2	0.2	0.8

Limpiar Rellenar celdas vacías con ceros

Calcular

Dimensión de la matriz: 4 Potencia: 2



POTENCIAS SUCESIVAS DE LAS DOS MATRICES DE TRANSICIÓN TRATADAS EN LOS EJEMPLOS

	A ₁	A ₂	A ₃
1	0.5572875	0.2604125	0.1822875
2	0.520825	0.270825	0.208325
3	0.48605139	0.27773889	0.23607639

	A ₁	A ₂	A ₃	A ₄
1	0.277775	0.369425	0.05	0.302775
2	0.14195247	0.65481229	0.02222	0.18083747
3	0.5	0.125	0.25	0.125
4	0.12222	0.27554	0.22	0.38222

	A ₁	A ₂	A ₃
1	0.534799790723375	0.266279174317125	0.198870754160875
2	0.53255834863425	0.26683547519675	0.20054011925925
3	0.5302689788945571	0.2673600869964321	0.2021931089293071

	A ₁	A ₂	A ₃	A ₄
1	0.19160490235475	0.43419768310825	0.1012078735	0.27288586847975
2	0.1655950248895063	0.5338253828485941	0.0669867959838	0.2332914547361563
3	0.29690905875	0.33225653625	0.1177775	0.25301968375
4	0.2297781724838	0.3583950006866	0.1513218988	0.2604252453838

	A ₁	A ₂	A ₃
1	0.53327500524866307995	0.2666288788461974197	0.19996599113894982724
2	0.53325775769239485947	0.26662679190492301733	0.19996932862420015432
3	0.53318931877289589309	0.26659910892178367237	0.19995346877080429382

	A ₁	A ₂	A ₃	A ₄
1	0.20136616536255553531	0.44640805760335066677	0.10169105442969946567	0.25025483272603616456
2	0.19362191680343891753	0.46273780740313024765	0.095710415611570641276	0.24742940460358688663
3	0.20501686780148653629	0.43609753200019428878	0.10446529375211237173	0.25422763607424866418
4	0.20814393285488904738	0.43470229452605623435	0.10449345056925829708	0.25242243464831215484

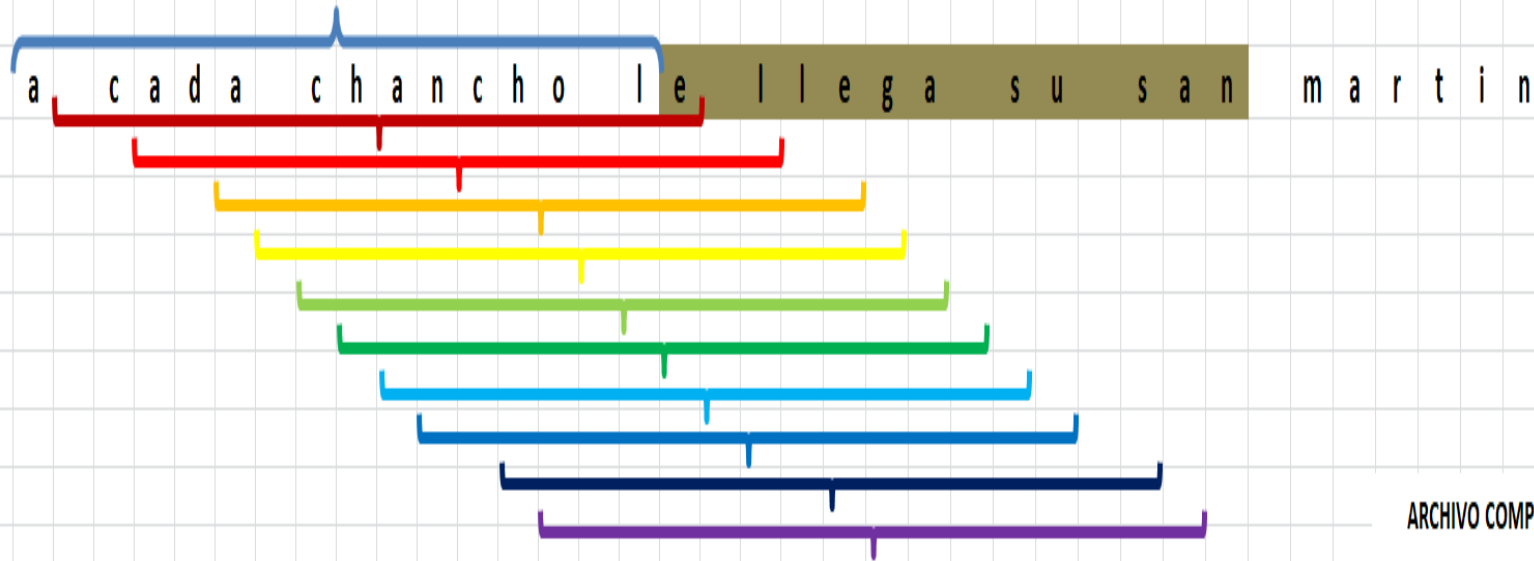
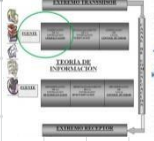
	A ₁	A ₂	A ₃
1	0.53318387988564567737	0.26658767441442439643	0.19993835643329982467
2	0.53317534882884879419	0.2665834090539706359	0.19993515752612316623
3	0.53311563359375072405	0.26655355201382744033	0.1999127650894997084

	A ₁	A ₂	A ₃	A ₄
1	0.19992012290462780969	0.44959493239452995142	0.09997621527389081731	0.24986994646930284922
2	0.19970826316646774181	0.44985786813923117551	0.099831613687507173946	0.24973872599729804194
3	0.20005458789486983688	0.44939039102125346805	0.10006557801970530156	0.24994053818472706193
4	0.20004415533744300066	0.44936826284468289743	0.10006434348793510143	0.24992936345118004134

MATRIZ ESTACIONARIA



CODIFICACIÓN LEMPEL ZIV



El objetivo de la estrategia de compresión Lempel Ziv es referenciar todo el conjunto de caracteres que se va recibiendo (generalmente texto) respecto de una ventana, que en tamaño (cantidad de caracteres que la conforman es potencia de dos 8,16,32, ...65536...) haciendo las veces de diccionario.

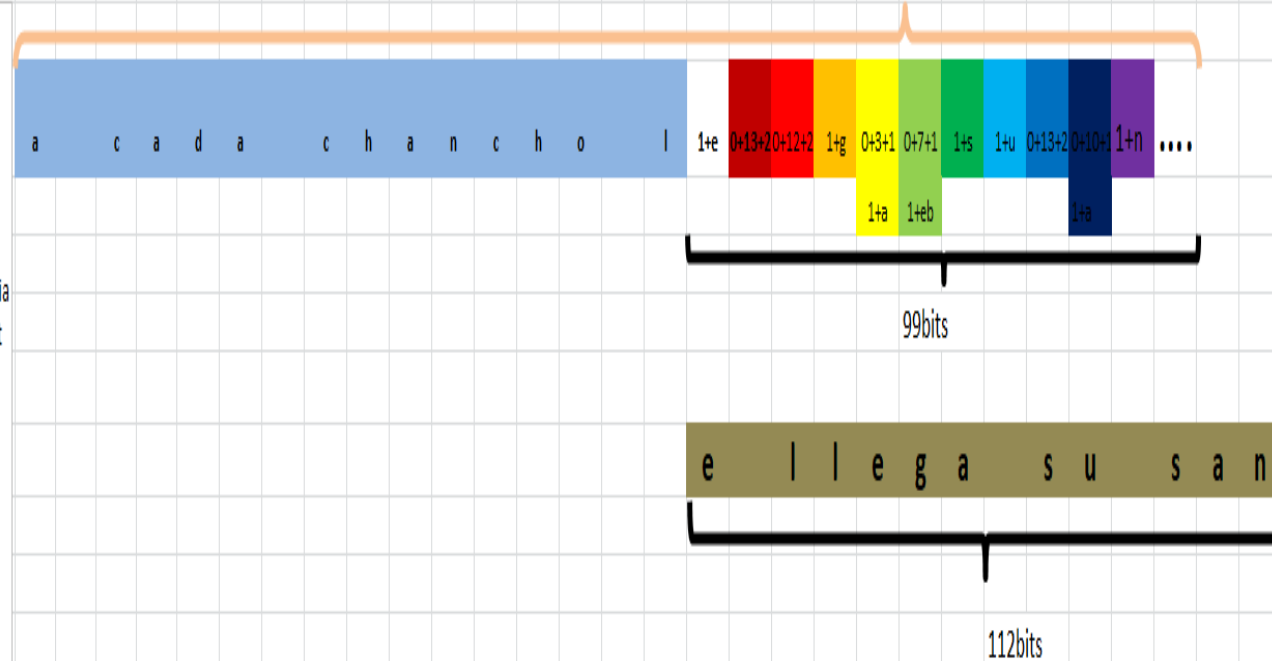
En general la ventana inicialmente la conforman los primeros caracteres de la fuente a comprimir (en nuestro caso los primeros 16 caracteres referenciada por la llave superior de color azul a cada chanco l). De esta manera todo caracter o cadena de caracteres nuevos que se lean en la secuencia se registrarán, inicialmente, con un flag que indica si tal cadena existe (flag, bit en 0) o no (Flag o bit en 1) en la ventana o diccionario.

Si la lectura realizada existe en el diccionario o ventana además del flag se indica desde donde empiezo la coincidencia y durante cuantos caracteres se extiende.

En caso de que el flag se coloque en 1 de modo que la lectura realizada no existe en la ventana, simplemente se adiciona el ascii del caracter leído.

Coincidencia: Flag=0+posición desde donde empieza la coincidencia+longitud de la misma.

No Coincidencia: Flag=1, Ascii del caracter leído



CODIFICACIÓN LEMPEL ZIV WELCH

a c a d a c h a n c h o l e l l e g a s u s a n m a r t i n

FRACCIÓN DE TEXTO ORIGINAL (240BITS) COMPRIMIDO EN (216)

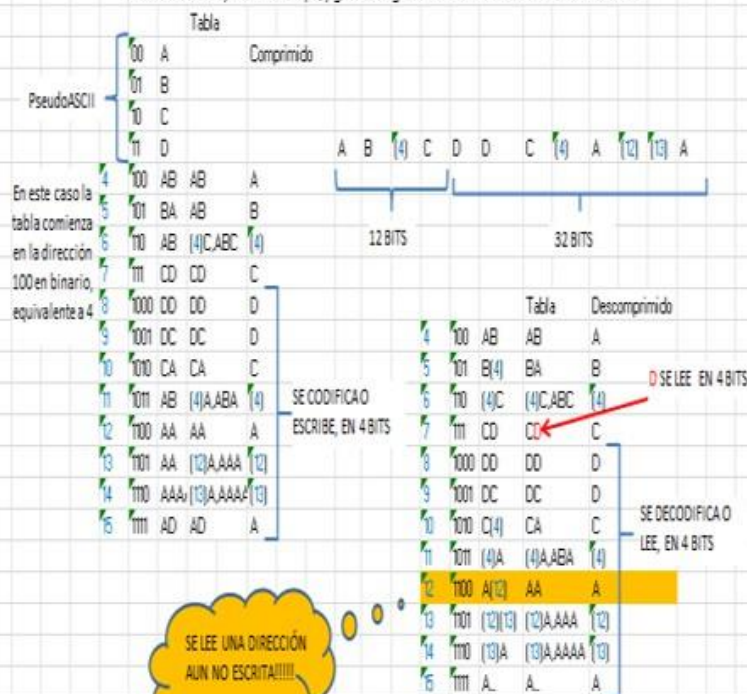
La mejora propuesta por la variante LZW de codificación consiste en asumir que los símbolos individuales ya están codificados en ASCII por lo que comienza desde un primer momento a codificar secuencias de símbolos de a par, triplas, cuádruplas conforme aparecen o no en una tabla que se va generando en RAM y que haría las veces del diccionario que se utilizaba antes en LZ. Entonces como la codificación ASCII para una aplicación real abarca desde las combinaciones de 8 bits que van desde el 0 al 255 la tabla propuesta por LZW arranca en 256 y desde ahí crece en profundidad y anchura en la medida que se van leyendo los símbolos.

Tabla	Comprimido
256 a(eb)	a(eb)
257 (eb)c	(eb)c
258 ca	ca
259 ad	ad
260 da	da
261 a(eb)	(256)c
262 ch	ch
263 ha	ha
264 an	an
265 nc	nc
266 ch	(262)o
267 o(eb)	o(eb)
268 (eb)l	(eb)l
269 le	le
270 e(eb)	e(eb)
271 (eb)l	(268)l
272 le	(268)g
273 ga	ga
274 a(eb)	(258)s
275 su	su
276 u(eb)	u(eb)
277 (eb)s	(eb)s
278 sa	sa
279 an	(264)(eb)
280	
281	

Tabla	Descomprimida
256 a(eb)	a(eb)
257 (eb)c	(eb)c
258 ca	ca
259 ad	ad
260 d(256)	da
261 (256)c	(256)c
262 ch	ch
263 ha	ha
264 an	an
265 n(262)	nc
266 (262)o	(262)o
267 o(eb)	o(eb)
268 (eb)l	(eb)l
269 le	le
270 e(268)	e(eb)
271 (268)(268)	(268)l
272 (268)g	(268)g
273 g(258)	ga
274 (258)s	(258)s
275 su	su
276 u(eb)	u(eb)
277 (eb)s	(eb)s
278 s(254)	sa
279 (264)an	(264)an
280	
281	

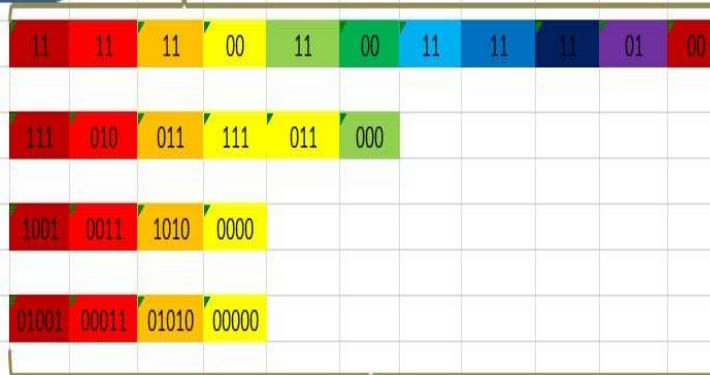
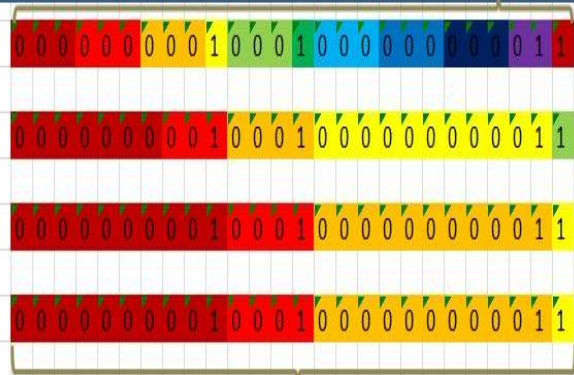
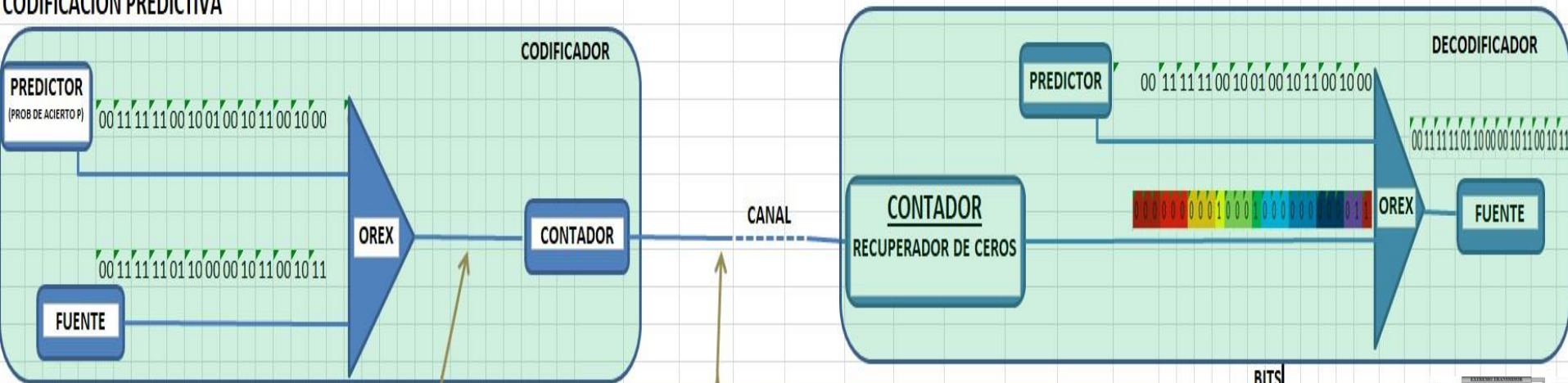
EJEMPLO DIDÁCTICO DE ACTUALIZACIÓN DE LONGITUDES DE CÓDIGO Y SINCRONIZACIÓN TRAS INTENTAR LEER UNA DIRECCIÓN AUN NO ESCRITA

Sea una fuente cuya variedad es $q=4$, y genera la siguiente secuencia: **ABABCDCCABAAAAAAD**



A B AB C D D C AB A AA AAA A

CODIFICACIÓN PREDICTIVA



SI EL PREDICTOR ES BUENO HABRÁ MAYORÍA DE CEROS QUE SERÁ LO QUE "CUENTE" EL CONTADOR LOGRANDO UN INTERESANTE NIVEL DE COMPRESIÓN

$$p(n) = p^n q \quad (n = 0, 1, \dots)$$

Summed over all lengths n , we have

$$\sum_{n=0}^{\infty} p^n q = q \sum_{n=0}^{\infty} p^n = \frac{q}{1-p} = \frac{q}{q} = 1$$

Length of Run	Number of Digits Sent
$0 \leq n \leq (2^k - 1) - 1$	k
$(2^k - 1) \leq n \leq 2(2^k - 1) - 1$	$2k$
$2(2^k - 1) \leq n \leq 3(2^k - 1) - 1$	$3k$
etc.	

The expected length is [setting $2^k - 1 = m$ and $p(n)$ as the probability of a run of exactly n 's]

$$\begin{aligned} & [p(0) + p(1) + \dots + p(m-1)]k \\ & + [p(m) + p(m+1) + \dots + p(2m-1)]2k \\ & + [p(2m) + p(2m+1) + \dots + p(3m-1)]3k \\ & + \dots \end{aligned}$$

$$p(n) = p^n q$$

$$\begin{aligned} & q[1 + p + \dots + p^{m-1}]k \\ & + qp^m[1 + p + \dots + p^{m-1}]2k \\ & + qp^{2m}[1 + p + \dots + p^{m-1}]3k \\ & + \dots \end{aligned}$$

Summing the terms in brackets, we arrange the expression in the form.

$$q \left[\frac{1-p^n}{q} \right] [1 + 2p^n + 3p^{2n} \dots] k$$

This sum is, in turn,

$$q \left[\frac{1-p^n}{q} \right] \left[\frac{1}{(1-p^n)^2} \right] k = \frac{k}{1-p^n}$$

We need a short table (Table 5.9-1) of the function ($k > 1$)

$$\frac{k}{1-p^{2^k-1}}$$

so that for a given $p \geq \frac{1}{2}$ we can pick the best k (Table 5.9-2).

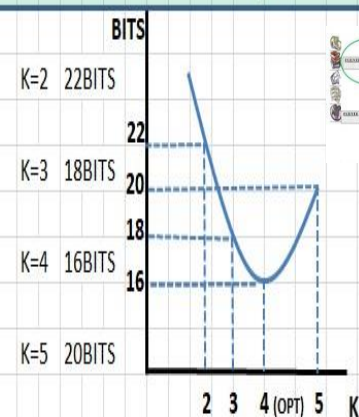


TABLE 5.9-1 Table of $\frac{k}{1-p^{2^k-1}}$

k	0.5	0.6	0.7	0.8	0.9	0.95	0.98	0.99
2	2.29	2.55	3.04	4.10	7.38	14.0	50.0	67.3
3			3.27	3.80	5.75	9.94	34.0	44.2
4				4.15	5.04	7.45	22.7	28.6
5					5.20	6.28	15.3	18.7
6						6.25	10.7	12.8
7							7.01	8.33
8								7.58
9								8.05
10								9.05

TABLE 5.9-2 Table of Efficiency of Encoding

p	Optimum k	Expected Number of Digits	Expected Length of Runs	Compression Ratio
0.7	2	3.04	3.33	0.91
0.8	3	3.80	5.00	0.76
0.9	4	5.04	10.00	0.50
0.95	6	6.25	20.00	0.31
0.98	7	7.58	50.00	0.16
0.99	8	8.67	100.00	0.087

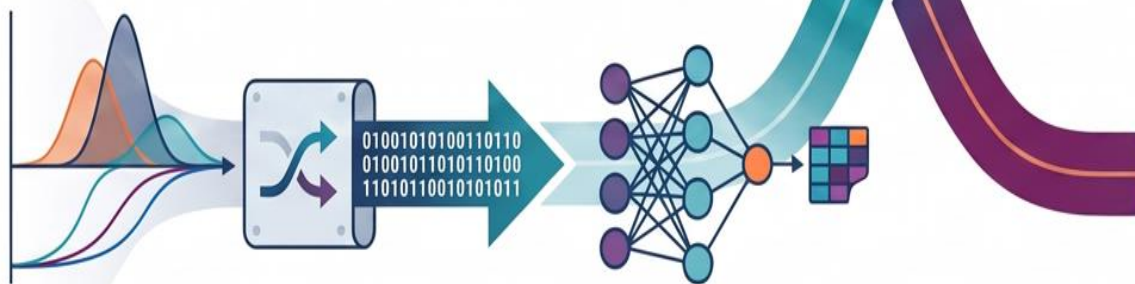
El Modelado de Lenguaje es Compresión: Los LLMs como Compresores Universales

La investigación establece que los modelos predictivos y los compresores sin pérdidas son “dos caras de la misma moneda”. Al aplicar codificación aritmética, los grandes modelos de lenguaje (LLM) no solo comprimen texto, sino que superan a compresores especializados en imágenes y audio, revelando nuevas leyes de escala sobre el tamaño de los datos y los modelos.

La Equivalencia Fundamental: Predicción = Compresión

Dos caras de la misma moneda

Maximizar la probabilidad de los datos es matemáticamente equivalente a minimizar los bits necesarios para almacenarlos.



El Puente de la Codificación Aritmética

Transforma las probabilidades de un modelo en un flujo de bits con una eficiencia casi óptima.

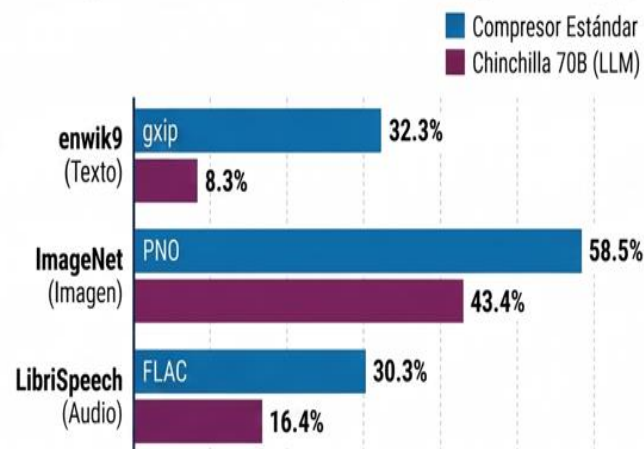
Compresión en Contexto (In-Context)

Los LLMs comprimen datos nuevos usando su capacidad de aprendizaje sin necesidad de reentrenamiento.

Rendimiento Superior en Múltiples Modalidades

Superando a los Estándares de la Industria

Comparativa de la Tasa de Compresión Bruta (Menor es Mejor)



El modelo Chinchilla 70B comprime mejor que PNG en imágenes y mejor que FLAC en audio.

El Dilema del Tamaño del Modelo

Para datasets pequeños, el peso de los parámetros del modelo puede superar el beneficio de compresión.



CÓDIGO GRAY

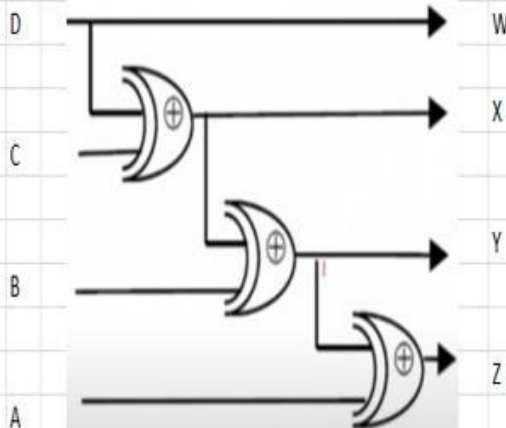
EL CÓDIGO GRAY TIENE LA PARTICULARIDAD DE SER UN CÓDIGO **ADYACENTE** (PALABRAS CÓDIGOS ASOCIADAS A REPRESENTACIONES CONTIGUAS DE LA BASE 10 DIFIEREN SOLAMENTE EN UN BIT) Y **CÍCLICO** (EL ÚLTIMO CÓDIGO ES ADYACENTE DEL PRIMERO!!)

W	X	Y	Z	DEC	D	C	B	A
BINARIO NATURAL					BINARIO GRAY			
0	0	0	0	0	0	0	0	0
0	0	0	1	1	0	0	0	1
0	0	1	0	2	0	0	1	1
0	0	1	1	3	0	0	1	0
0	1	0	0	4	0	1	1	0
0	1	0	1	5	0	1	1	1
0	1	1	0	6	0	1	0	1
0	1	1	1	7	0	1	0	0
1	0	0	0	8	1	1	0	0
1	0	0	1	9	1	1	0	1
1	0	1	0	10	1	1	1	1
1	0	1	1	11	1	1	1	0
1	1	0	0	12	1	0	1	0
1	1	0	1	13	1	0	1	1
1	1	1	0	14	1	0	0	1
1	1	1	1	15	1	0	0	0

ADYACENTES

ADYACENTES

CONVERSIÓN DE GRAY A BINARIO!!



W	BA	DC	11	10	01	00
11	1	1	1	1	1	1
10	1	1	1	1	1	1
01	0	0	0	0	0	0
00	0	0	0	0	0	0

W=D

X	BA	DC	11	10	01	00
11	0	0	0	0	0	0
10	1	1	1	1	1	1
01	1	1	1	1	1	1
00	0	0	0	0	0	0

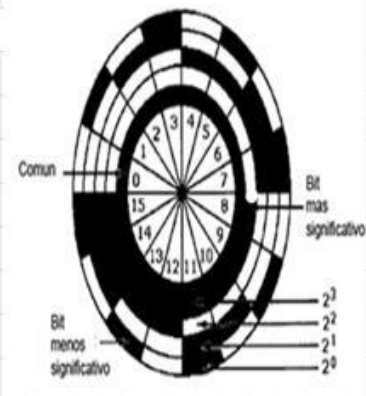
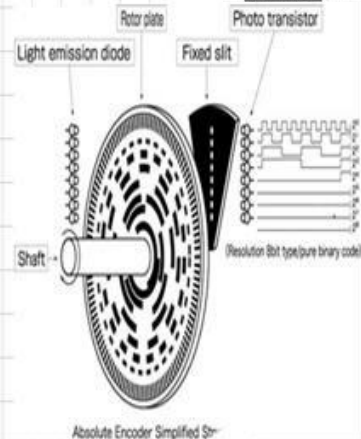
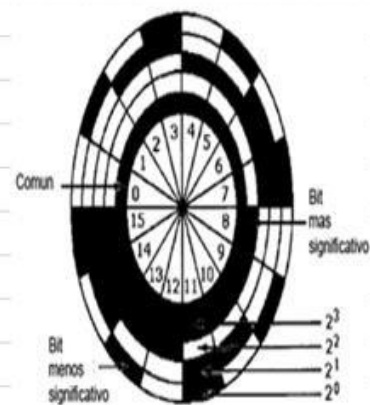
X = D xor C

Y	BA	DC	11	10	01	00
11	0	0	1	1	1	1
10	1	1	0	0	0	0
01	0	0	1	1	1	1
00	1	1	0	0	0	0

Y = (D xor C) xor B

Z	BA	DC	11	10	01	00
11	0	1	1	1	0	0
10	0	1	1	1	0	0
01	0	1	1	1	0	0
00	0	1	1	1	0	0

Z = ((D xor C) xor B) xor A



TRANSFORMADA BURROWS WHEELER

La BWT no comprime los datos por sí misma, sino que los reorganiza para que sean más fáciles de comprimir (por ejemplo, con un codificador como RLE o Huffman). El proceso consiste en tres pasos principales:

1. **Formar todas las rotaciones cíclicas** de la cadena.
2. **Ordenar lexicográficamente** (como en un diccionario) todas estas rotaciones.
3. **Tomar la última columna** de esta matriz ordenada. Esa es la transformada.

Vamos a aplicarlo a `ABRACADABRA`.

Paso 1: Crear todas las rotaciones cíclicas

Tomamos la cadena y creamos todas las versiones posibles rotando un carácter a la vez.

La cadena original es: **A B R A C A D A B R A** (11 caracteres, por lo que tendremos 11 rotaciones). Cada fila se genera tomando la fila anterior y desplazando todos los caracteres una posición a la izquierda, moviendo el primero al final.

Nº	Rotación
0	A B R A C A D A B R A
1	B R A C A D A B R A A
2	R A C A D A B R A A B
3	A C A D A B R A A B R
4	C A D A B R A A B R A
5	A D A B R A A B R A C
6	D A B R A A B R A C A
7	A B R A A B R A C A D
8	B R A A B R A C A D A
9	R A A B R A C A D A B
10	A A B R A C A D A B R

Paso 2: Ordenar las rotaciones lexicográficamente

Ahora, tratamos cada una de las filas de la tabla anterior como una cadena independiente y las ordenamos alfabéticamente, como si estuviéramos ordenando palabras en un diccionario.

Matriz sin ordenar (solo para referencia)

0: ABRACADABRA
1: BRACADABRAA
2: RACADABRAAB
3: ACADABRAABR
4: CADABRAABRA
5: ADABRAABRAC
6: DABRAABRACA
7: ABRAABRACAD
8: BRAABRACADA
9: RAABRACADAB
10: AABRACADABR

Matriz ordenada:

Nº	Rotación Ordenada
10	A A B R A C A D A B R
7	A B R A A B R A C A D
0	A B R A C A D A B R A
3	A C A D A B R A A B R
5	A D A B R A A B R A C
1	B R A C A D A B R A A
8	B R A A B R A C A D A
4	C A D A B R A A B R A
6	D A B R A A B R A C A
9	R A A B R A C A D A B
2	R A C A D A B R A A B

Nota importante: El símbolo '\$`' (o a veces '#`')

En la implementación real de la BWT, se suele añadir un carácter especial de fin de cadena (como '\$`') que se considera lexicográficamente menor que cualquier otro carácter. Esto es crucial para que la transformación sea reversible. Para 'ABRACADABRA\$', el resultado sería ligeramente diferente ('ARD\$RCAAAABB'). Sin embargo, como tu frase original no lo incluye, seguimos el proceso estándar sin él, asumiendo un orden alfabético simple donde 'A < B < C < D < R'.

Paso 3: Extraer la última columna

La Transformada de Burrows-Wheeler es simplemente la última columna de esta matriz ordenada. Veamos la matriz ordenada y fijémonos en el último carácter de cada fila:

Rotación Ordenada	Último Carácter	Resultado Final
:-----	-----	Si leemos la última columna de arriba a abajo, obtenemos la transformada: RDARCAAAABB
A A B R A C A D A B R	-> R	Normalmente, se escribe como una sola cadena: RDARCAAAABB
A B R A A B R A C A D	-> D	¿Por qué esto es útil?
A B R A C A D A B R A	-> A	La cadena original era: 'ABRACADABRA'
A C A D A B R A A B R	-> R	La cadena transformada es: 'RDARCAAAABB'
A D A B R A A B R A C	-> C	En la cadena transformada aparecen "corridas" (secuencias del mismo carácter). Tenemos "AAAA" y "BB". Agrupar caracteres repetidos de esta manera hace que la compresión posterior (por ejemplo, con Run-Length Encoding) sea mucho más eficiente. La BWT ha reorganizado la cadena para exponer estas secuencias.
B R A C A D A B R A A	-> A	Resumen en un solo vistazo
B R A A B R A C A D A	-> A	Paso 1: Crear rotaciones
C A D A B R A A B R A	-> A	Paso 2: Ordenar
D A B R A A B R A C A	-> A	Paso 3: Tomar última columna (BWT)
R A A B R A C A D A B	-> B	Entrada: ABRACADABRA
R A C A D A B R A A B	-> B	Salida BWT: RDARCAAAABB

PREDICTIVE PARTIAL MATCHING (PPM)

La codificación PPM significa Prediction by Partial Matching (Predicción por Coincidencia Parcial). Es una técnica de compresión de datos sin pérdida, muy usada en textos, que pertenece a la familia de los modelos estadísticos adaptativos.

1. Idea principal

El algoritmo intenta predecir el próximo símbolo (una letra, un byte, etc.) basándose en los símbolos anteriores. Usa modelos de contexto: por ejemplo, para predecir la próxima letra, puede fijarse en la palabra o en las últimas 3 o 4 letras ya vistas.

2. Coincidencia parcial

Si con el contexto largo (ej. 4 letras anteriores) no encuentra una predicción, baja a un contexto más corto (3 letras, luego 2, luego 1...). Esto lo hace hasta llegar al contexto de orden -1, que es una predicción uniforme (como decir: "si no sé nada, todos los símbolos son posibles con igual probabilidad").

3. Modelo probabilístico

Para cada contexto, el algoritmo lleva un conteo de cuántas veces apareció cada símbolo después de él. A partir de esas frecuencias construye una distribución de probabilidad para codificar el próximo símbolo.

4. Codificación

Generalmente, PPM se combina con codificación aritmética o rangos para representar los símbolos en muy pocos bits, de acuerdo con sus probabilidades.

5. Ventaja

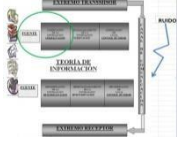
Al adaptarse continuamente al texto, suele lograr una compresión muy alta para datos con estructura predecible (lenguaje natural, código fuente, etc.).

En resumen:

La codificación PPM (Prediction by Partial Matching) es un método de compresión que predice el siguiente símbolo basándose en el contexto reciente, ajustando probabilidades según la experiencia previa. Es un modelo predictivo estadístico adaptativo y de los más potentes para compresión de texto.

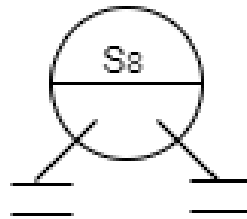
https://docs.google.com/spreadsheets/d/149EVQiMEG_E- EaytnztGGvWcEeR4vcl4x-Scf9w4pl/edit?usp=sharing

Huffman dinámico (I)



Premisa 1:

Transmisor y receptor, generan un árbol de codificación de Huffman, en el que llevan para cada símbolo del mensaje una hoja, con el símbolo propiamente dicho, y su peso (la cantidad de veces que apareció), éste peso se actualiza cada vez que el símbolo aparece en el mensaje.

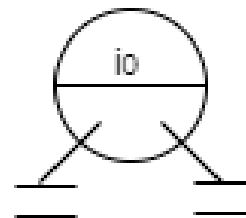


Premisa 2:

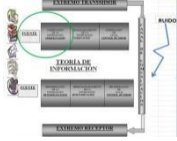
Al igual que la codificación de Huffman estático, los símbolos más probables deben estar más cerca de la raíz, para que sean más cortos (sibling property), es una estrategia de balanceo para que el árbol no degenera en una lista.

Premisa 3:

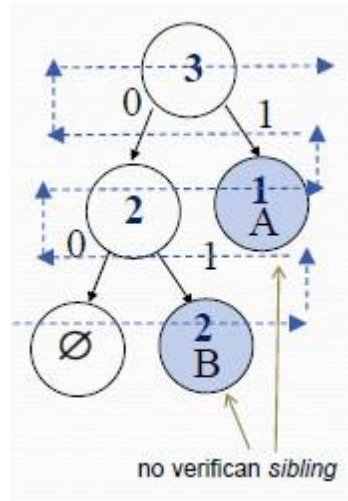
Transmisor y receptor inician un árbol con un símbolo, al que se denomina **símbolo imaginario**, representa a los símbolos que aún no han sido transmitidos, es decir, un símbolo del mensaje que aún no está en el árbol.



Sibling Property - Huffman dinámico (II)

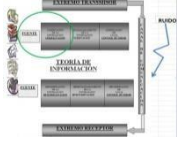


Sibling Property: busca mantener el árbol como árbol Huffman, esto es que los nodos de mayor peso, estén más cerca de la raíz.



Para mantener esta propiedad, se lleva una lista, que recorre el árbol desde lo profundo a la raíz, por nivel, de izquierda a derecha, iniciando el recorrido en el nodo imaginario. Si la lista está ordenada por pesos, cumple la propiedad, si no es así, necesita reordenarse, haciendo el intercambio más largo posible.

Algoritmo - Huffman dinámico (III)

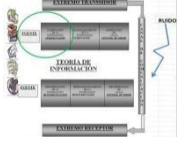


- 1) Inicializar el árbol con el símbolo imaginario, en transmisor y receptor, leer y transmitir el primer símbolo del mensaje, actualizar el árbol, generando a partir del símbolo imaginario, dos hijos, uno izquierdo, que será el nuevo símbolo imaginario, y otro derecho, que será el nuevo símbolo, con peso uno.
- 2) Leer el próximo símbolo del mensaje
- 3) Generar el código:
 - si el símbolo está en el árbol, emitir el código binario asociado a este símbolo
 - si no está en el árbol, emitir el código del nodo imaginario, seguido del símbolo literal nuevo (para el primer símbolo del mensaje no hay código para el nodo)
- 3) Actualizar el árbol:
 - Si el símbolo está en el árbol, incrementar la frecuencia del nodo, expandiendo la actualización a los nodos superiores.
 - Si no está, generar a partir del nodo imaginario, dos nuevos nodos: el nodo de la izquierda, será el nuevo nodo imaginario, y el de la derecha el nuevo símbolo con peso uno.
- 4) Verificar la propiedad de sibling :
 - Si no se cumple, reestructurar el árbol, intercambiando los 2 nodos en conflicto (con sus subárboles si los tuvieran): el nodo de peso mayor y más próximo al inicio del recorrido - con el nodo de peso menor y más cercano a la raíz
- 5) Si no terminó el mensaje volver al paso 1)

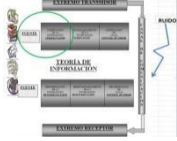
Compresión - Huffman dinámico (I)

Sea el mensaje, ABRACADABRA, que posee 5 símbolos distintos, se propone una codificación de longitud fija para representarlos, como lo indica la siguiente tabla:

Símbolo	Código
A	000
B	001
R	010
C	011
D	100



Compresión - Huffman dinámico (II)

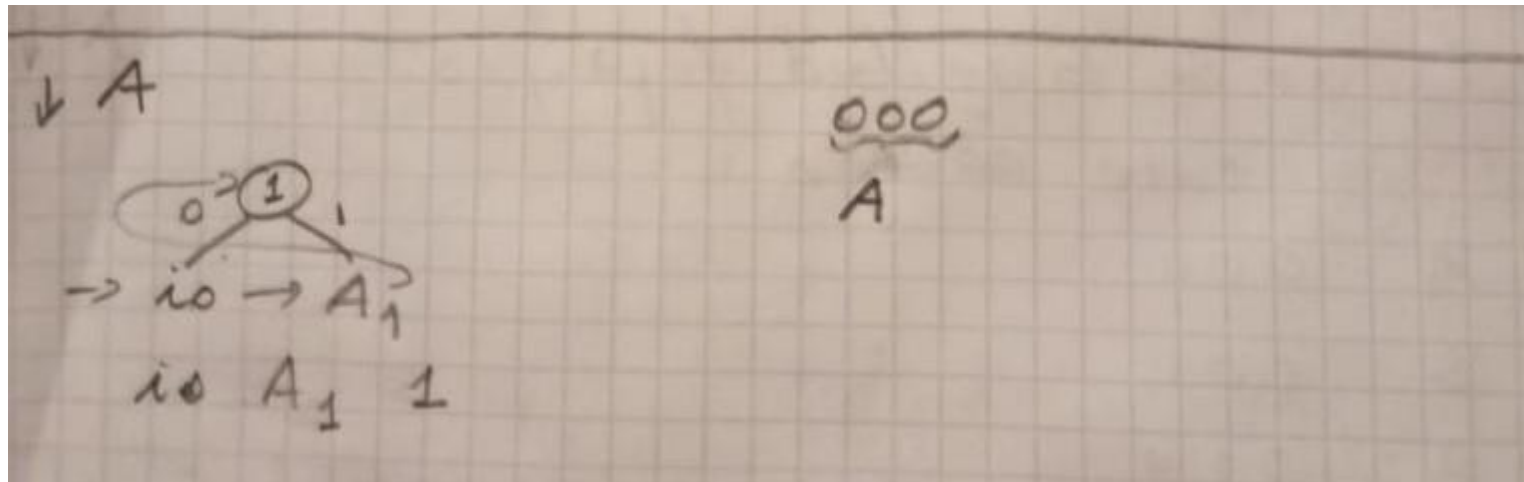


ABRACADABRA

io

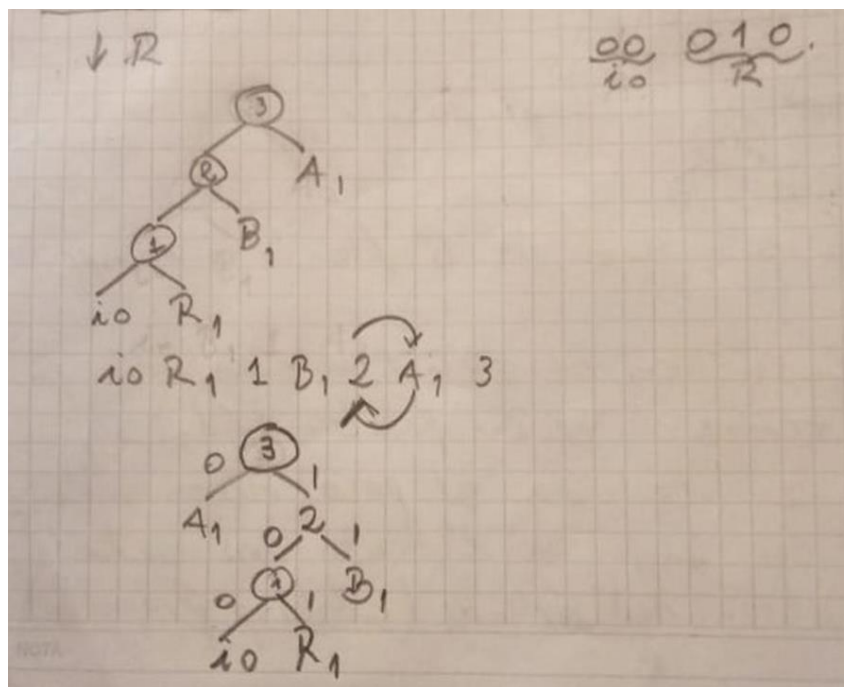
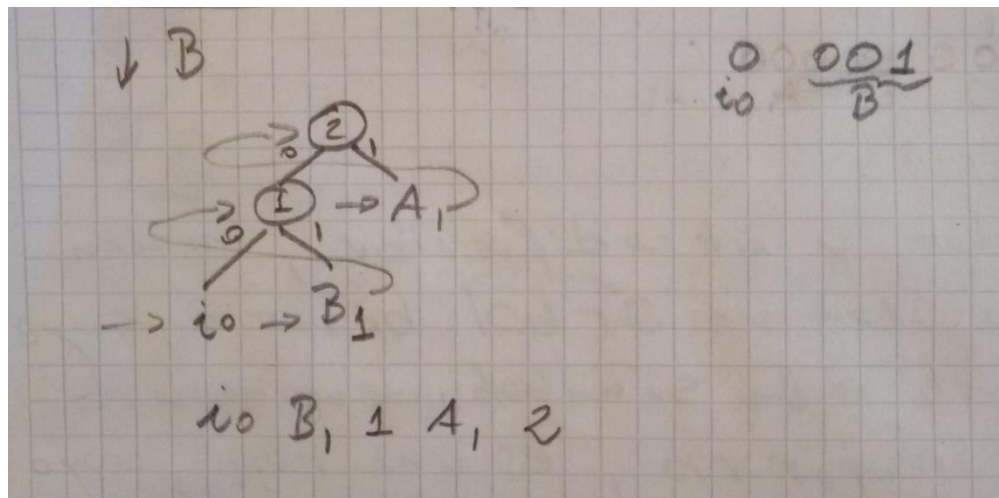
Archivo generado.

Símbolo	Código
A	000
B	001
R	010
C	011
D	100



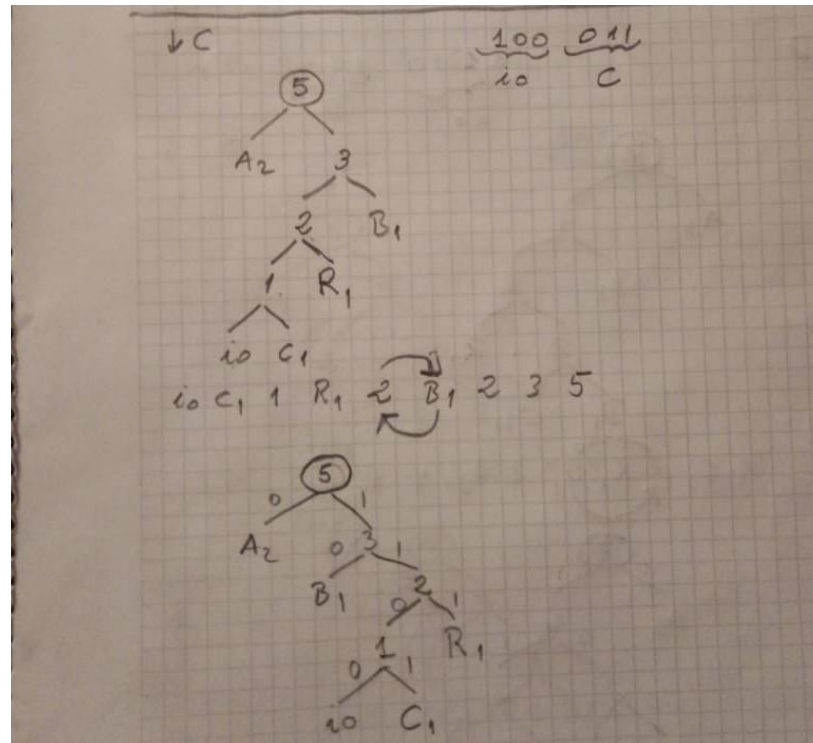
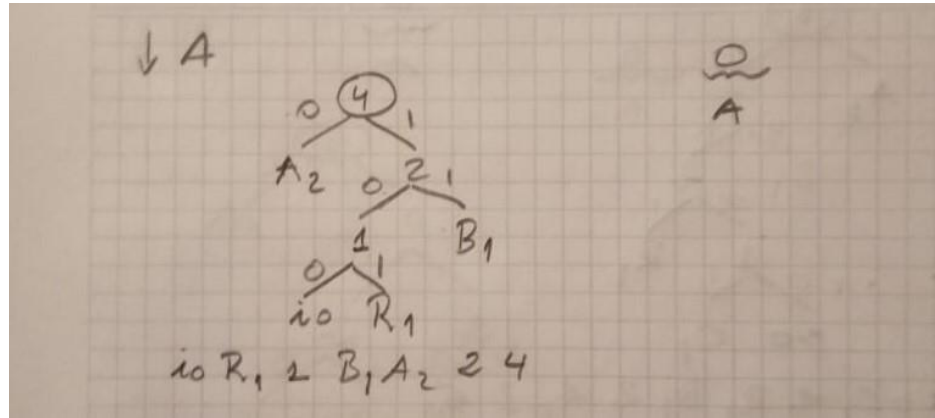
Compresión - Huffman dinámico (III)

Símbolo	Código
A	000
B	001
R	010
C	011
D	100



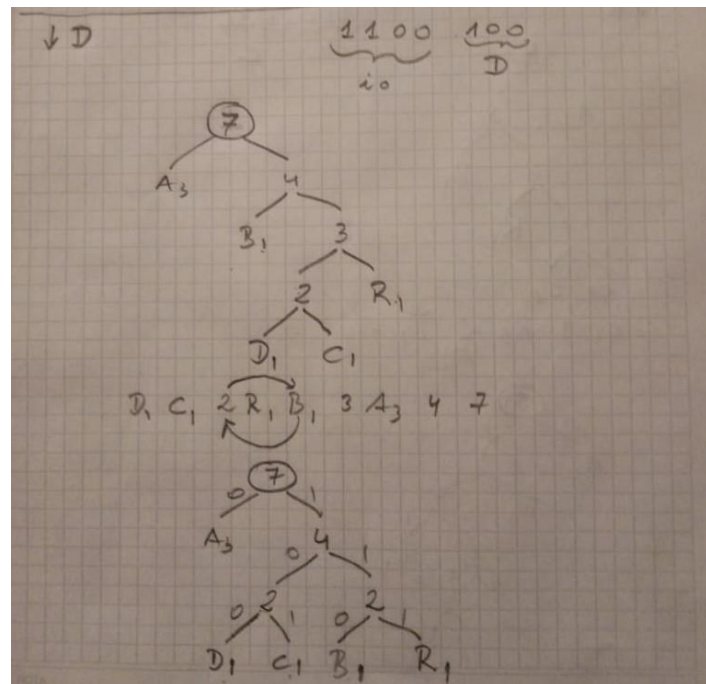
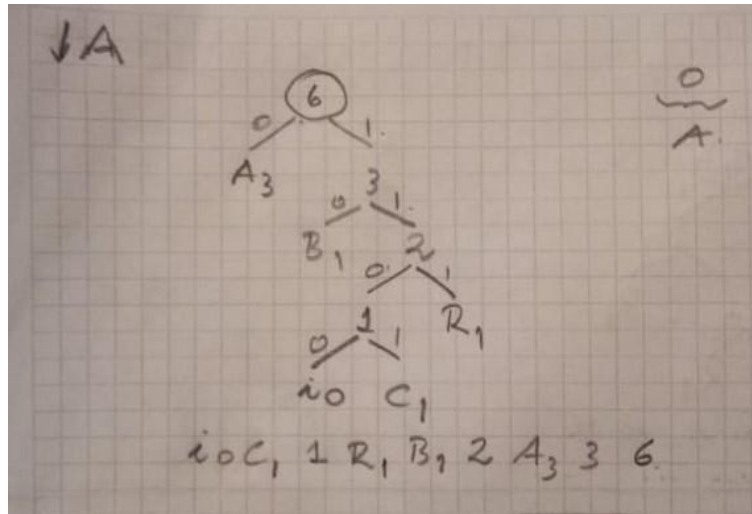
Compresión - Huffman dinámico (IV)

Símbolo	Código
A	000
B	001
R	010
C	011
D	100



Compresión - Huffman dinámico (V)

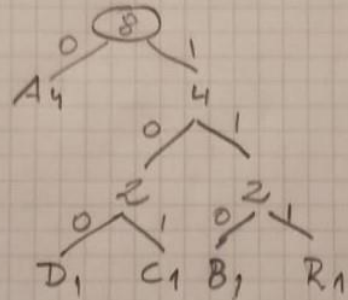
Símbolo	Código
A	000
B	001
R	010
C	011
D	100



Compresión - Huffman dinámico (VI)

↓ A.

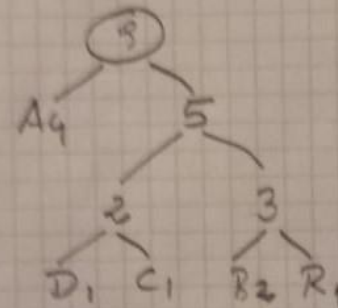
0
A



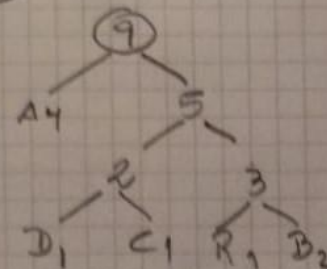
D₁ C₁ B₁ R₁ 2 2 A₄ 4 8

↓ B

110
B

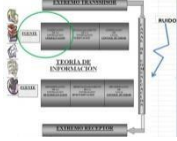


D₁ C₁ B₂ R₁ 2 3 A₄ 5 9

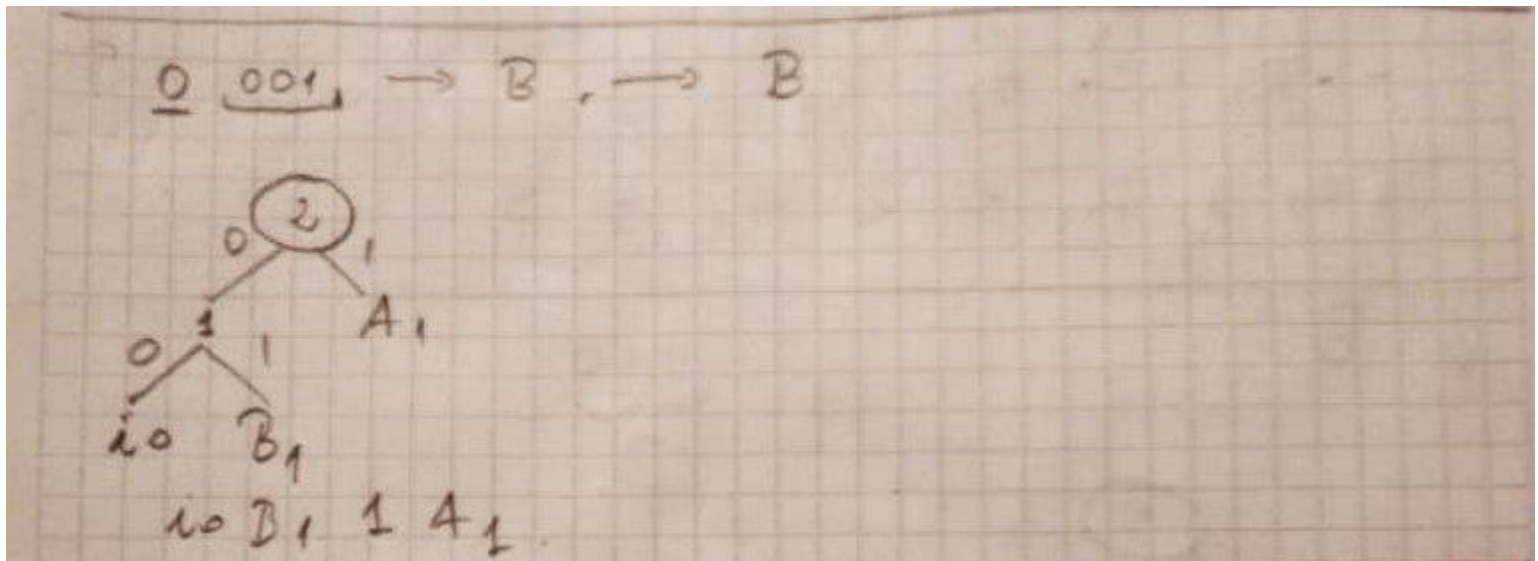
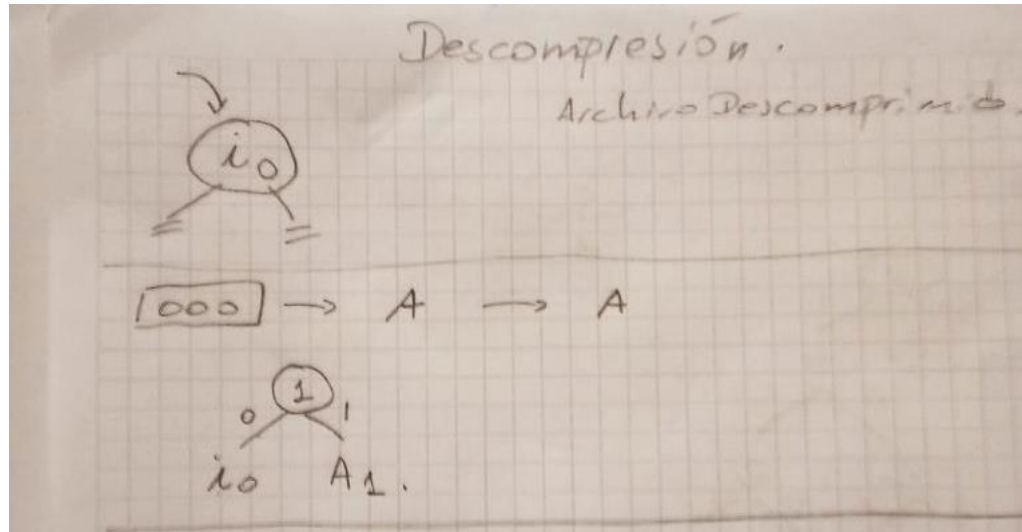


D₁ C₁ R₁ B₂ 2 3 A₄ 5 9

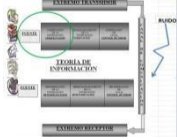
Descompresión - Huffman dinámico (I)



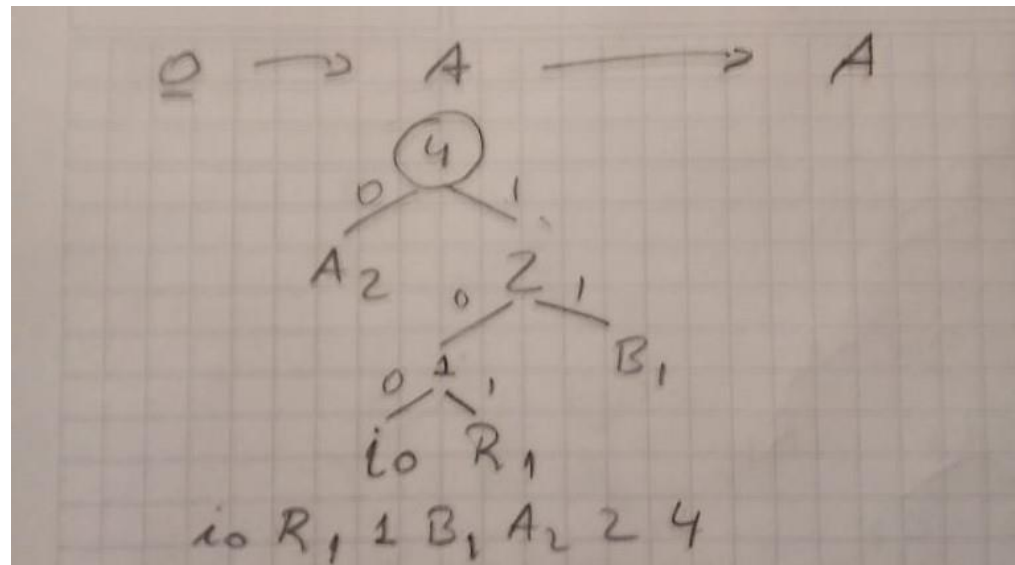
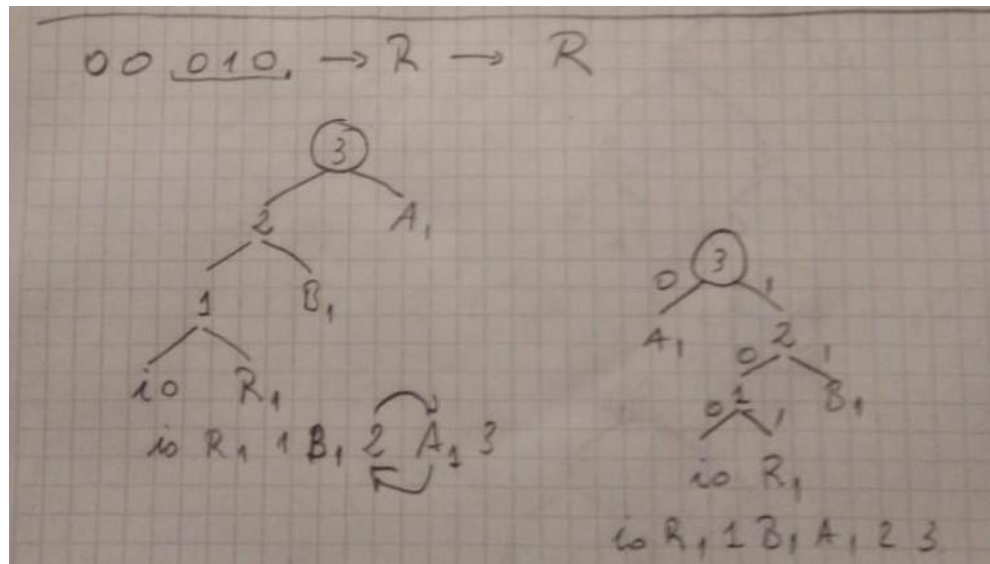
Símbolo	Código
A	000
B	001
R	010
C	011
D	100



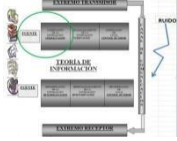
Descompresión - Huffman dinámico (II)



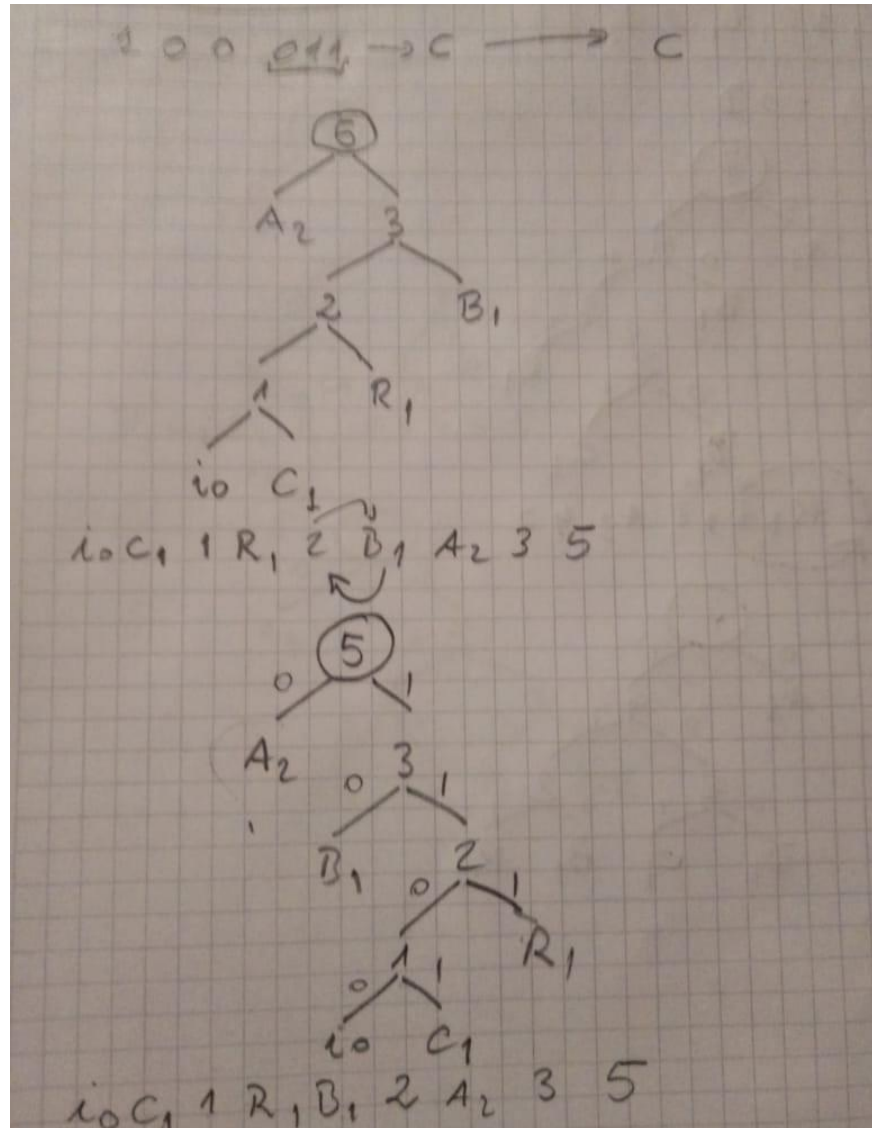
Símbolo	Código
A	000
B	001
R	010
C	011
D	100



Descompresión - Huffman dinámico (III)

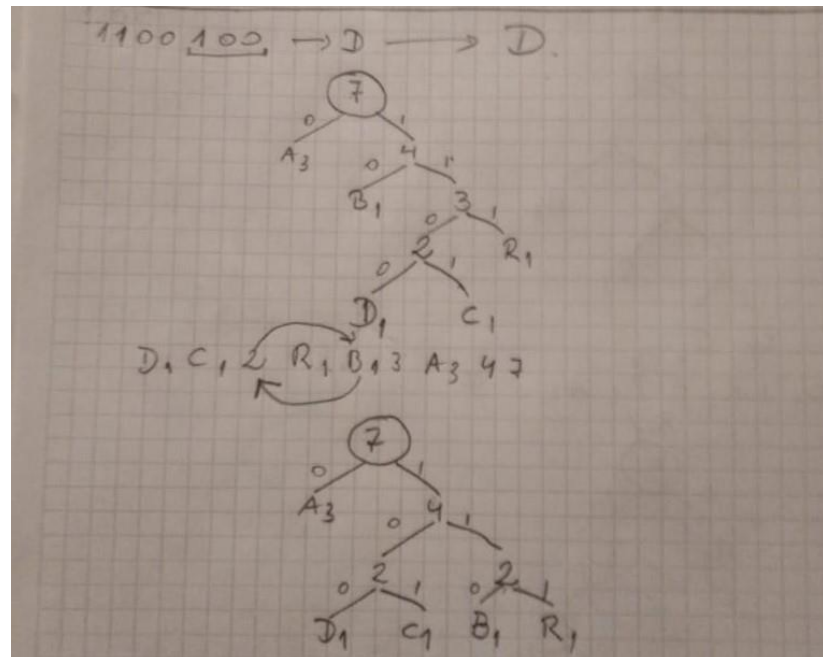
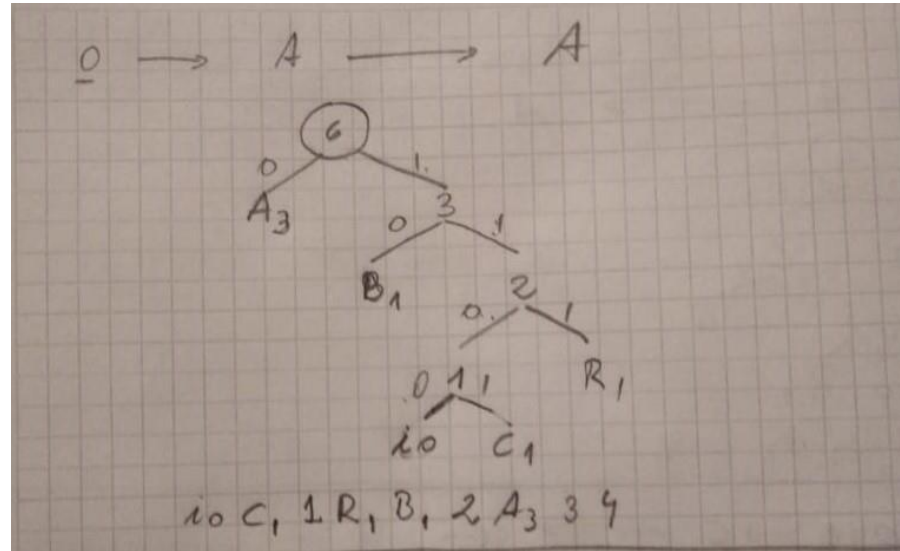


Símbolo	Código
A	000
B	001
R	010
C	011
D	100

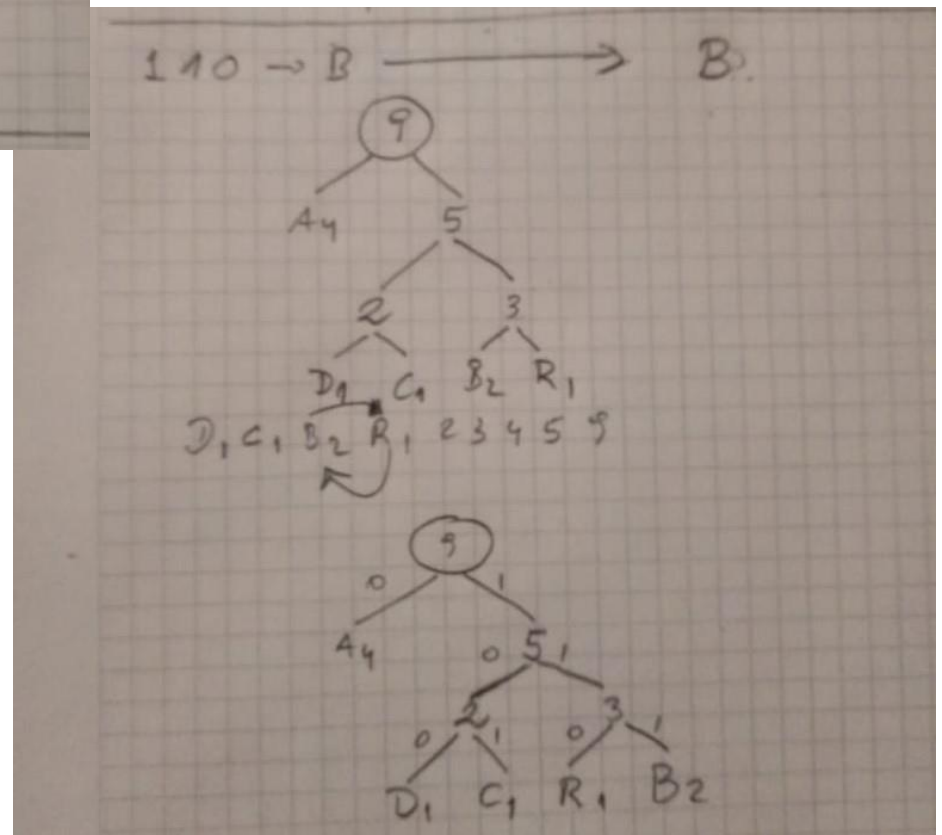
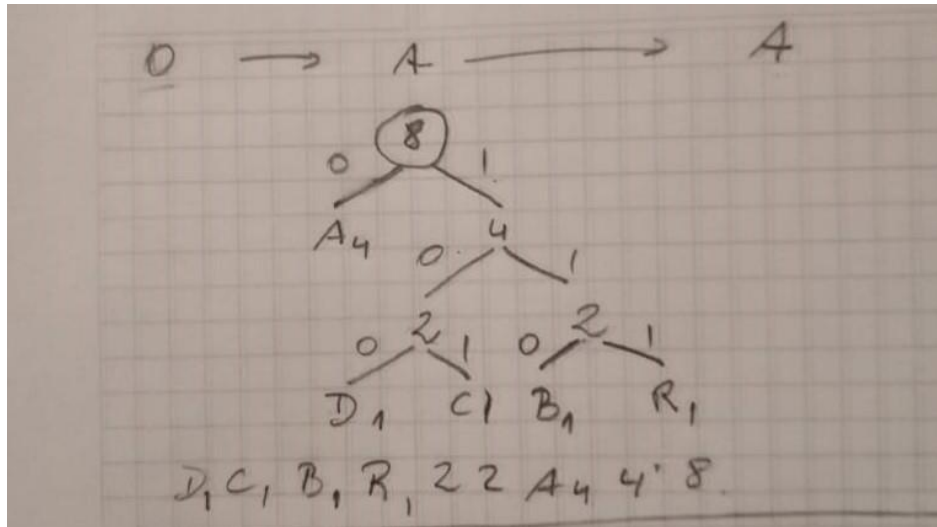
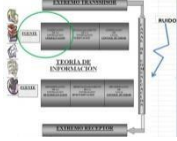


Descompresión - Huffman dinámico (IV)

Símbolo	Código
A	000
B	001
R	010
C	011
D	100



Descompresión - Huffman dinámico (V)



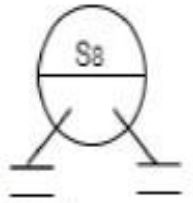
Huffman

Dinámico



09-08-1925 07-10-1999

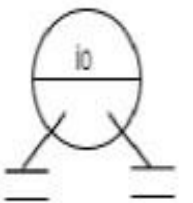
Premisas



Premisa 1: Transmisor y receptor, generan un árbol de codificación de Huffman , en el que llevan para cada símbolo del mensaje una hoja, con el símbolo propiamente dicho, y su peso (la cantidad de veces que apareció), éste peso se actualiza cada vez que el símbolo aparece en el mensaje.

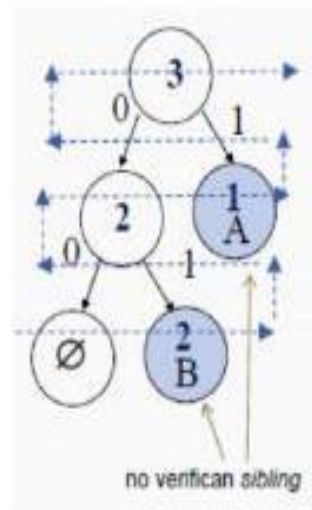
Premisa 2: Al igual que la codificación de Huffman estático, los símbolos más probables deben estar más cerca de la raíz, para que sean más cortos (sibling property), es una estrategia de balanceo para que el árbol no degenera en una lista.

Premisa 3: Transmisor y receptor inician un árbol con un símbolo, al que se denomina símbolo imaginario, representa a los símbolos que aún no han sido transmitidos, es decir, un símbolo del mensaje que aún no está en el árbol.

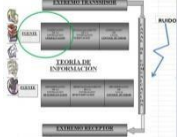


Sibling Property

Sibling Property: busca mantener el árbol como árbol Huffman, esto es que los nodos de mayor peso, estén más cerca de la raíz.



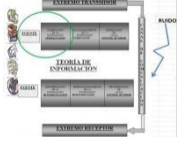
Para mantener esta propiedad, se lleva una lista, que recorre el árbol desde lo profundo a la raíz, por nivel, de izquierda a derecha, iniciando el recorrido en el nodo imaginario. Si la lista está ordenada por pesos, cumple la propiedad, si no es así, necesita reordenarse, haciendo el intercambio más largo posible.



Pasos

- 0) Inicializar el árbol con el símbolo imaginario, en transmisor y receptor, leer y transmitir el primer símbolo del mensaje, actualizar el árbol, generando a partir del símbolo imaginario, dos hijos, uno izquierdo, que será el nuevo símbolo imaginario, y otro derecho, que será el nuevo símbolo, con peso uno.
- 1) Leer el próximo símbolo del mensaje
- 2) Generar el código: –si el símbolo está en el árbol, emitir el código binario asociado a este símbolo –si no está en el árbol, emitir el código del nodo imaginario, seguido del símbolo literal nuevo (para el primer símbolo del mensaje no hay código para el nodo)
- 3) Actualizar el árbol: –Si el símbolo está en el árbol, incrementar la frecuencia del nodo, expandiendo la actualización a los nodos superiores. –Si no está , generar a partir del nodo imaginario, dos nuevos nodos: el nodo de la izquierda, será el nuevo nodo imaginario, y el de la derecha el nuevo símbolo con peso uno. Validar si el árbol contiene 255 símbolos, si posee todos menos uno (255) el próximo símbolo reemplazará al símbolo imaginario.
- 4) Verificar la propiedad de sibling : –Si no se cumple, reestructurar el árbol, intercambiando los 2 nodos en conflicto (con sus subárboles si los tuvieran): el nodo de peso mayor y más próximo al inicio del recorrido - con el nodo de peso menor y más cercano a la raíz
- 5) Si no terminó el mensaje volver al paso 1)

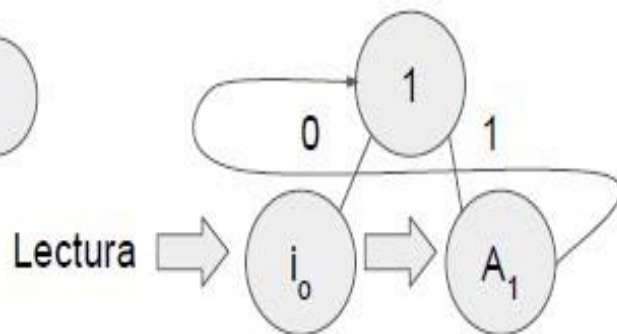
Compresión



INICIO

i_0

Leemos A



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

ABRACADABRA



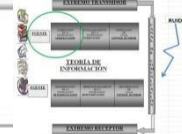
A = 000



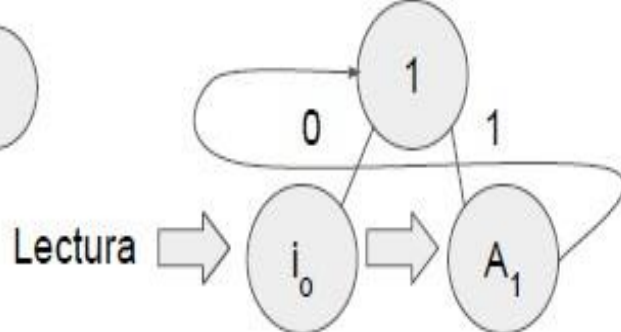
i_0 , A, 1

000

COMPRESIÓN



Leemos A



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

ABRACADABRA



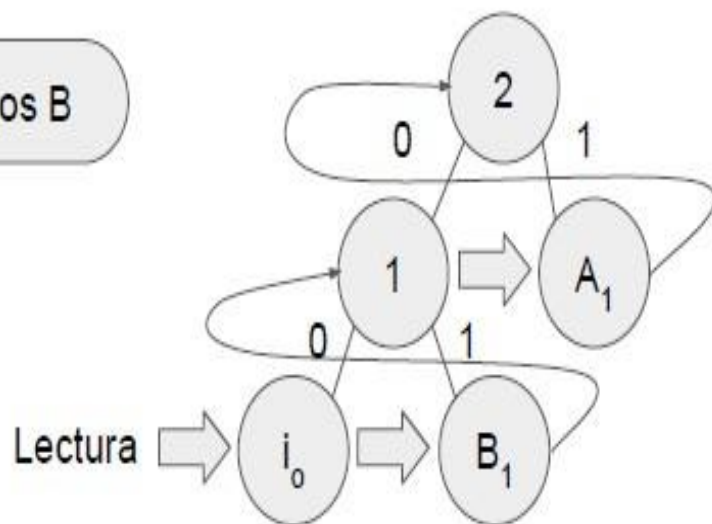
B = 0 001



i_0 B₁ 1 A₁ 2

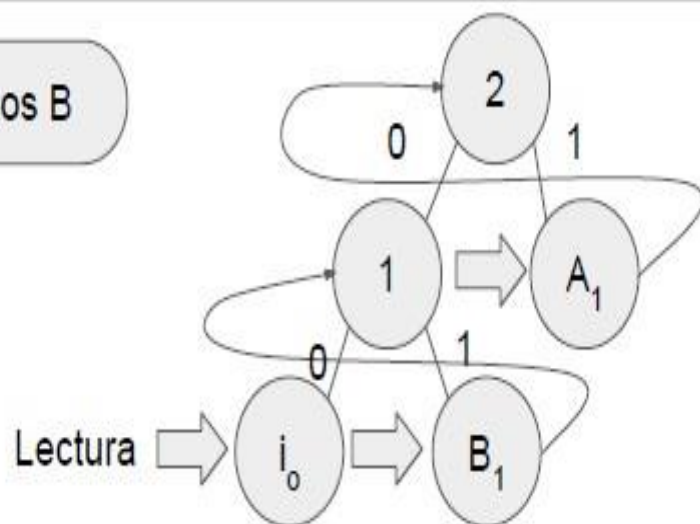
000 0 001

Leemos B



COMPRESIÓN

Leemos B



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

ABRACADABRA



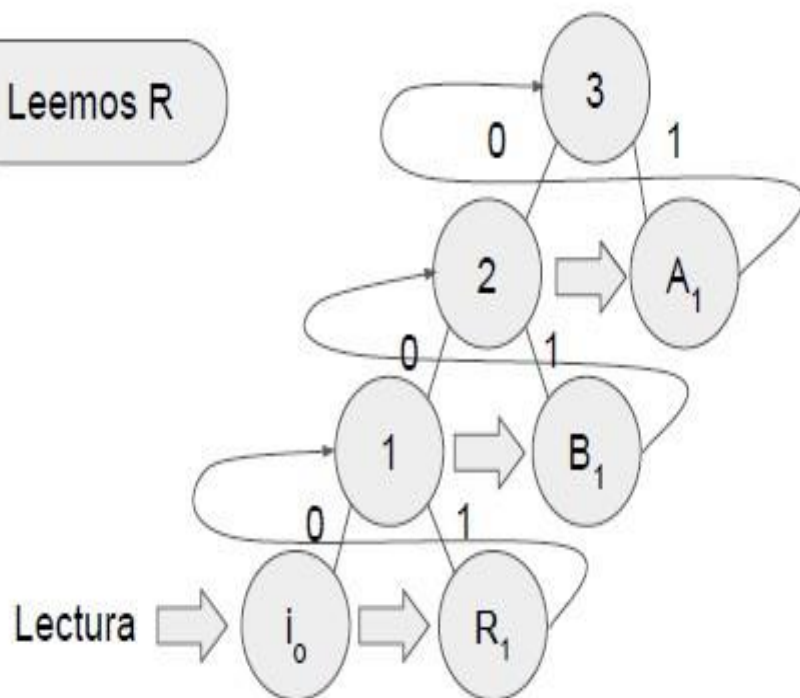
R = 00 011



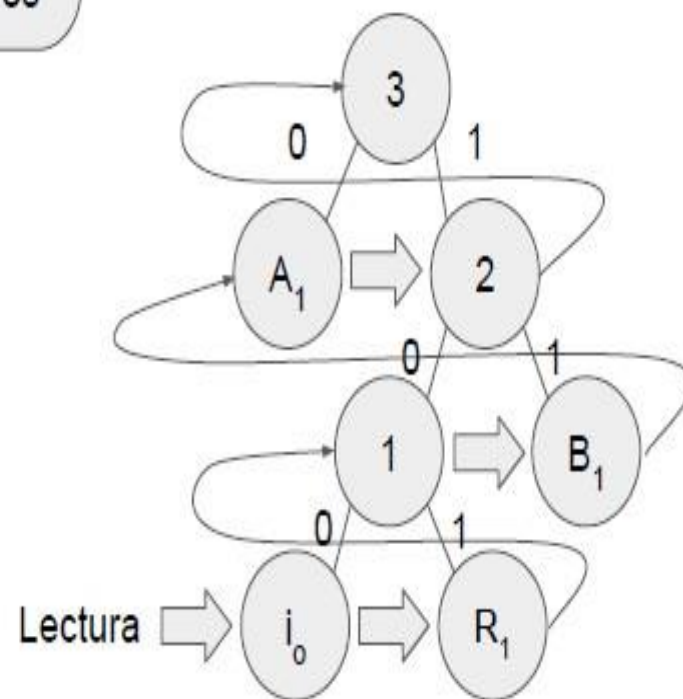
i_0 R₁ 1 B₁ **2** **A₁** 3

000 **0** 001 **00** 010

Leemos R



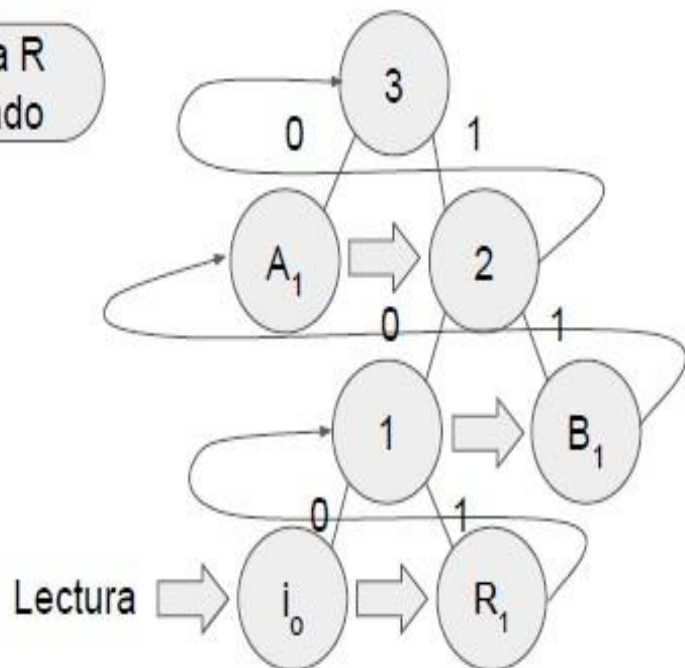
Ordenamos



COMPRESIÓN

i_0 R₁ 1 B₁ **A₁** **2** 3

Lectura R
ordenado



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

ABRACADABRA



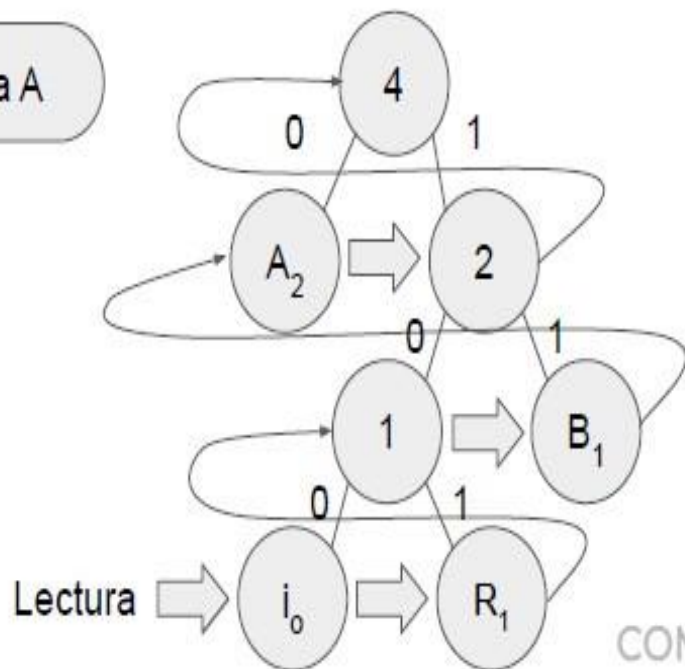
A = 0



i_0 R_1 1 B_1 A_2 2 4

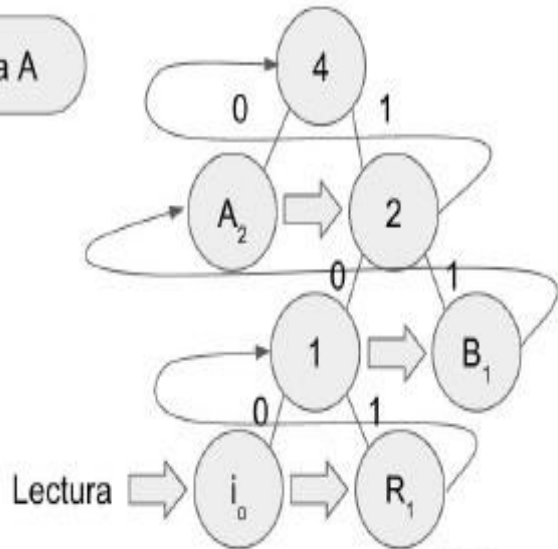
000 0 001 00 010 0

Lectura A



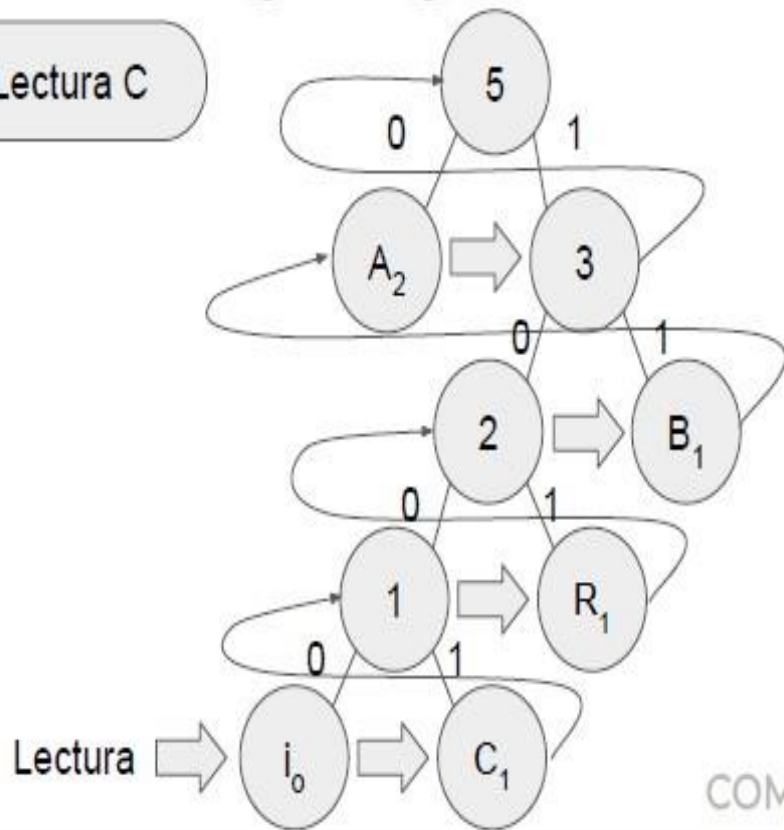
COMPRESIÓN

Lectura A



Lectura

Lectura C



Lectura

Símbolo	Código
A	000
B	001
R	010
C	011
D	100

ABRACADABRA



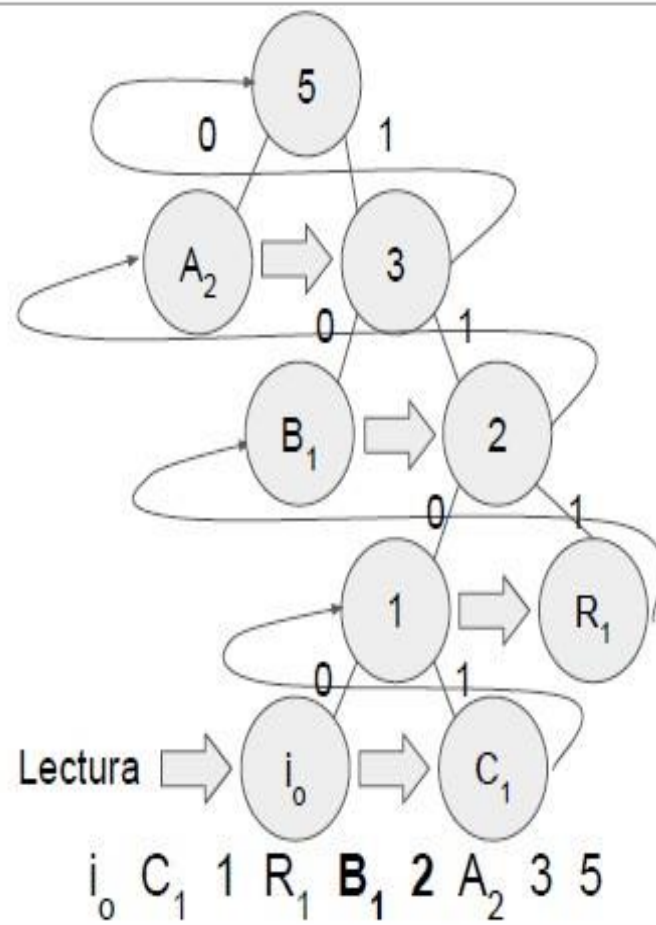
C = 100 011



i₀ C₁ 1 R₁ **2 B₁** A₂ 3 5

000 0 001 00 010 0 100 011

Ordenamos

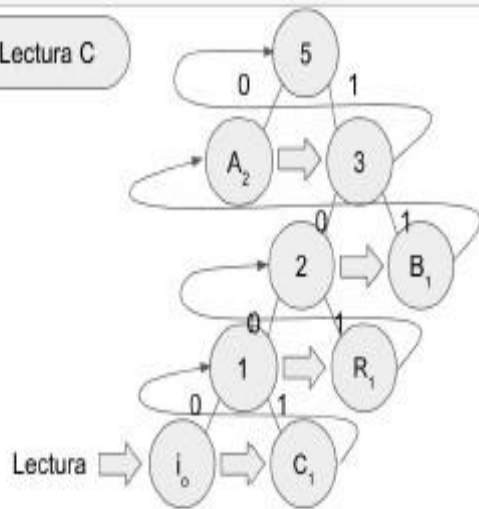


Lectura

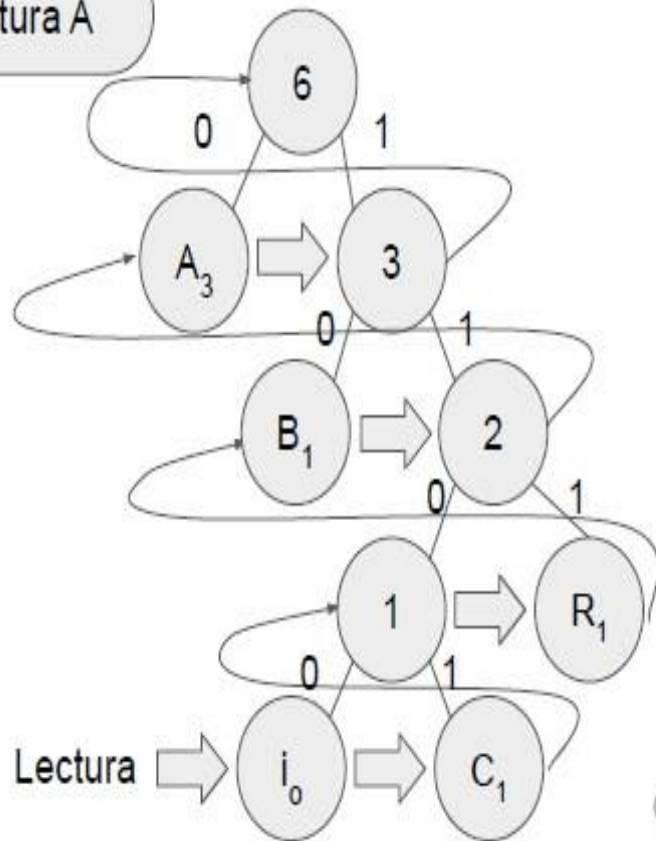
i₀ C₁ 1 R₁ **B₁ 2** A₂ 3 5

COMPRESIÓN

Lectura C



Lectura A



Símbolo Código

A 000

B 001

R 010

C 011

D 100

ABRACADABRA



A = 0

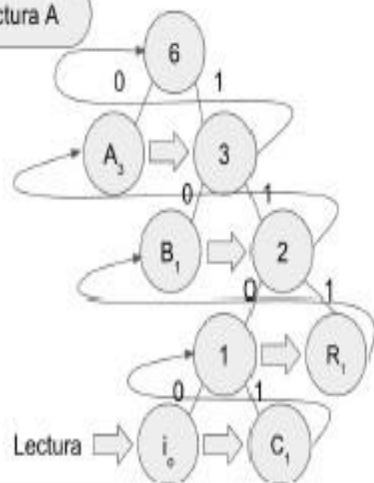


i_0 C_1 1 R_1 B_1 2 A_3 3 6

000 0 001 00 010 0 100 011 0

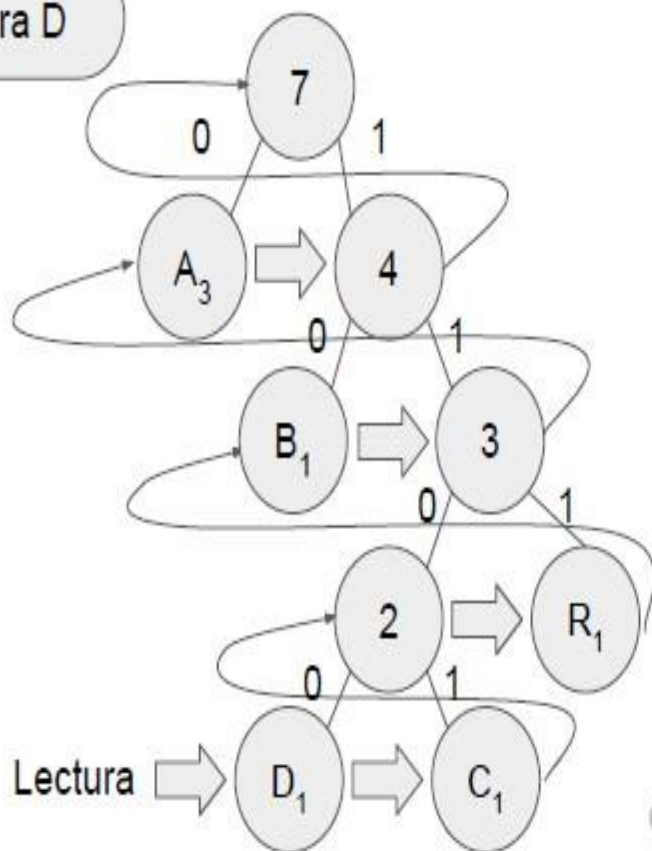
COMPRESIÓN

Lectura A



Lectura

Lectura D



Lectura

Símbolo Código

A	000
B	001
R	010
C	011
D	100

ABRACADABRA



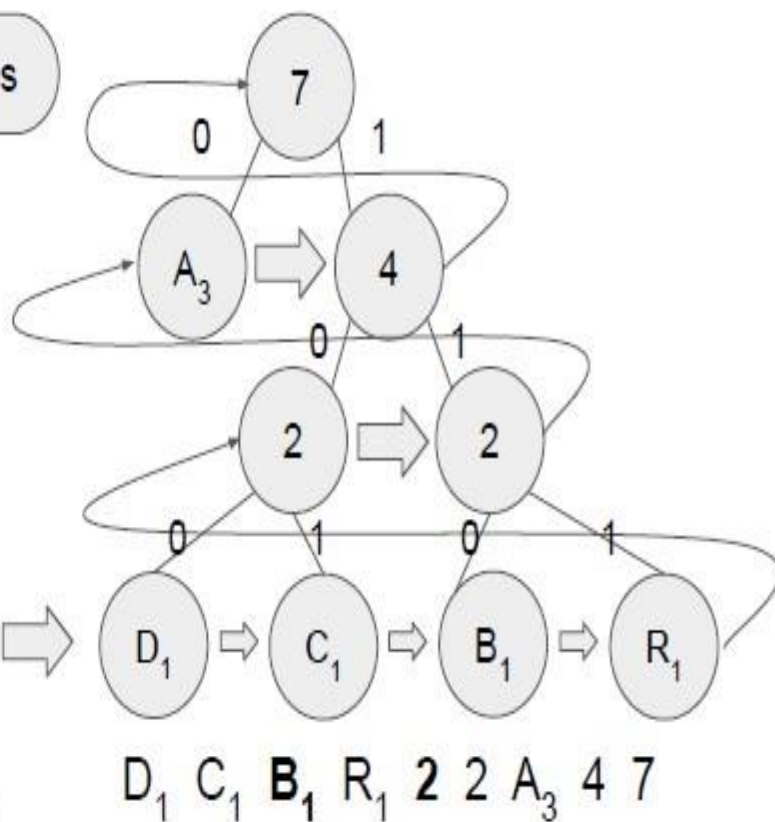
D = 1100 100



D₁ C₁ **2** R₁ **B₁** 3 A₃ 4 7

000 0 001 00 010 0 100 011 0 1100 100

Ordenamos

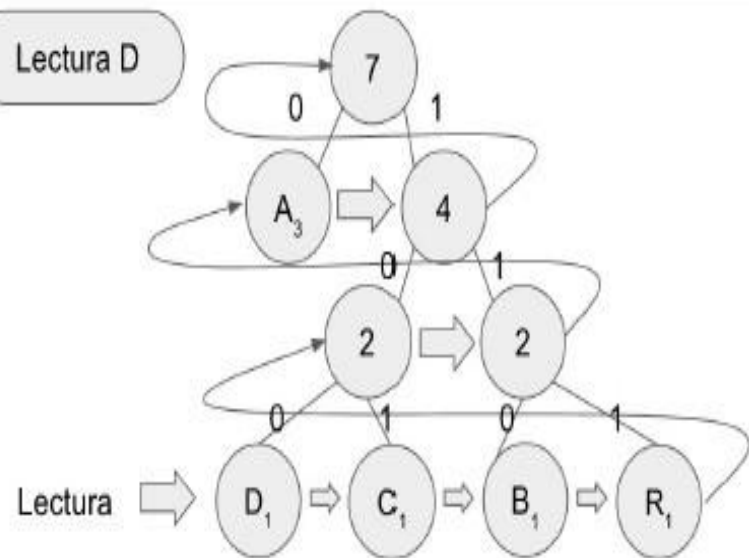


Lectura

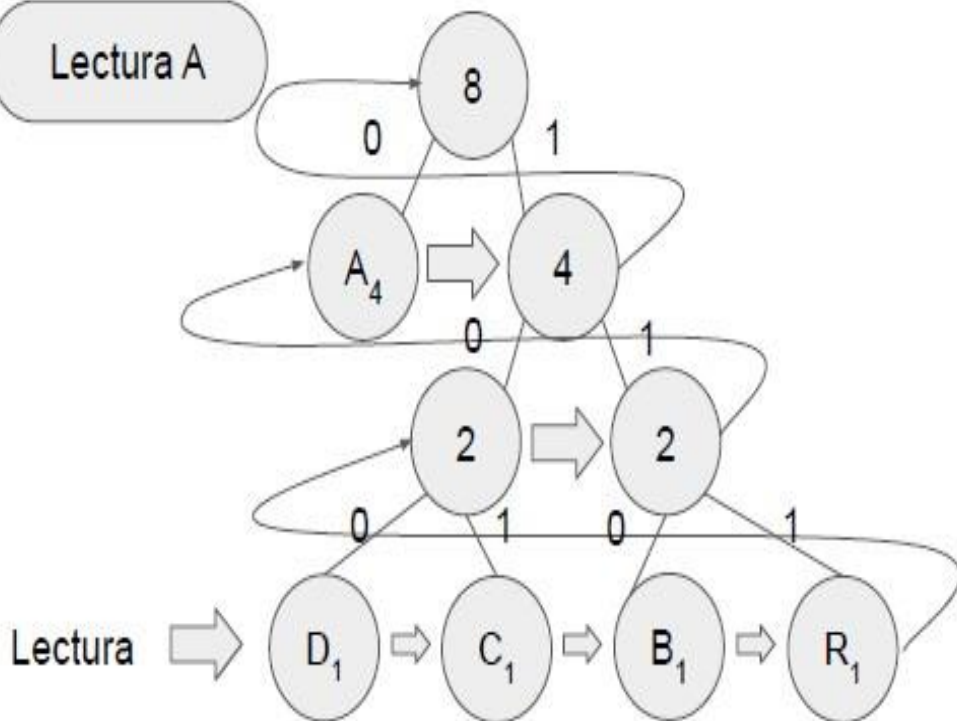
D₁ C₁ **B₁** R₁ **2 2** A₃ 4 7

COMPRESIÓN

Lectura D



Lectura A



COMPRESIÓN

ABRACADABRA



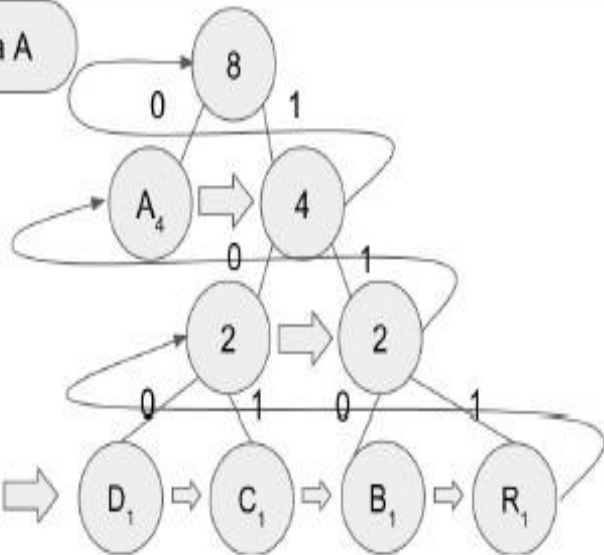
A = 0



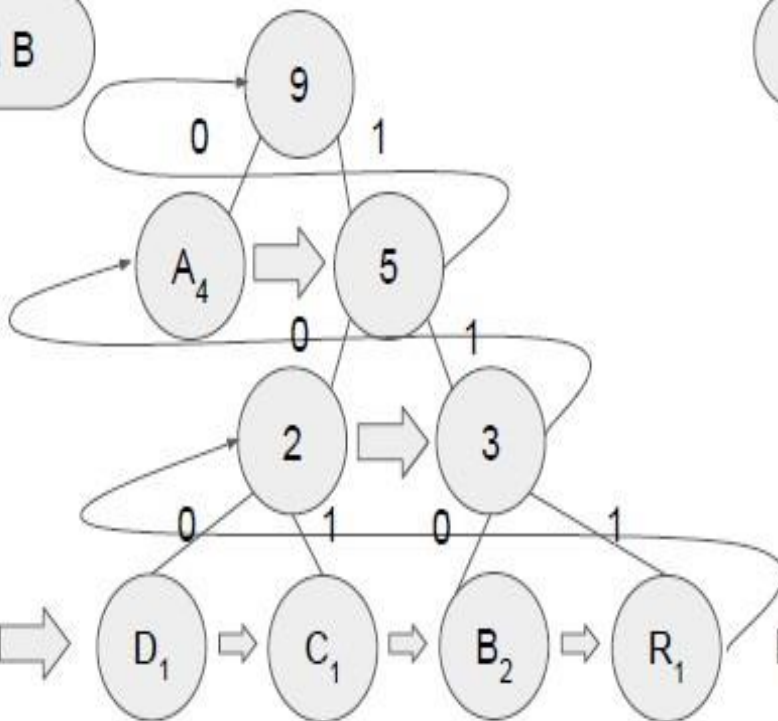
D₁ C₁ B₁ R₁ 2 2 A₄ 4 8

000 0 001 00 010 0 100 011 0 1100 100
0

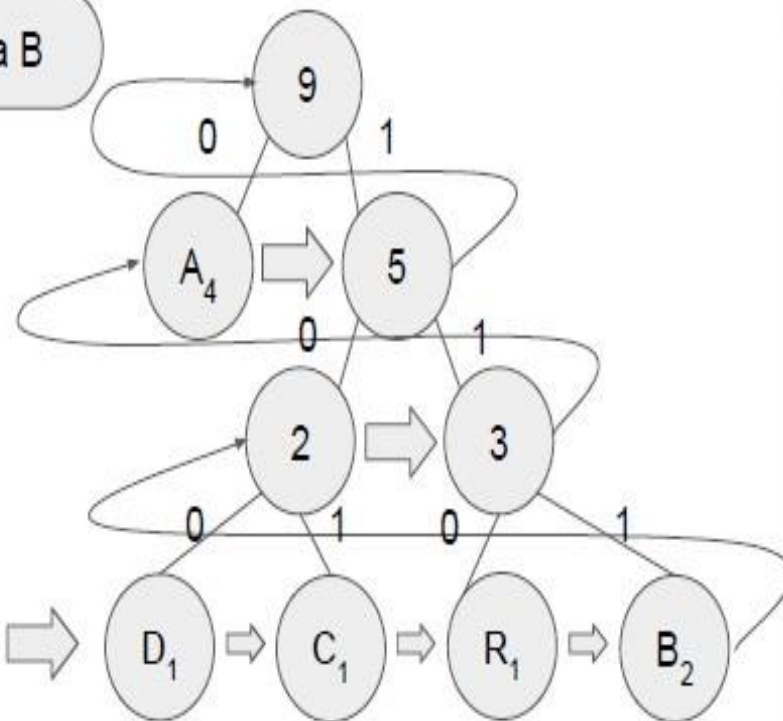
Lectura A



Lectura B



Lectura B



COMPRESIÓN

ABRACADABRA

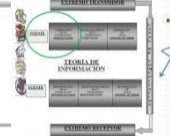


B = 110

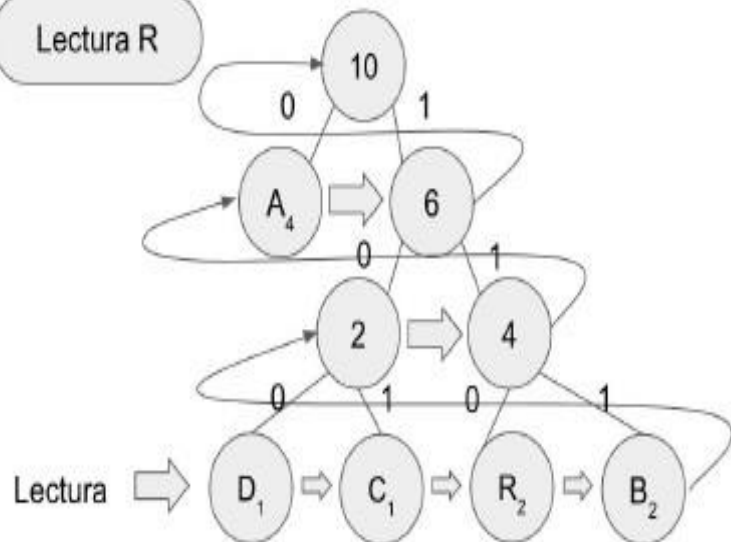


D₁ C₁ **B₂** **R₁** 2 2 A₃ 4 7

000 0 001 00 010 0 100 011 0 1100 100
0 110



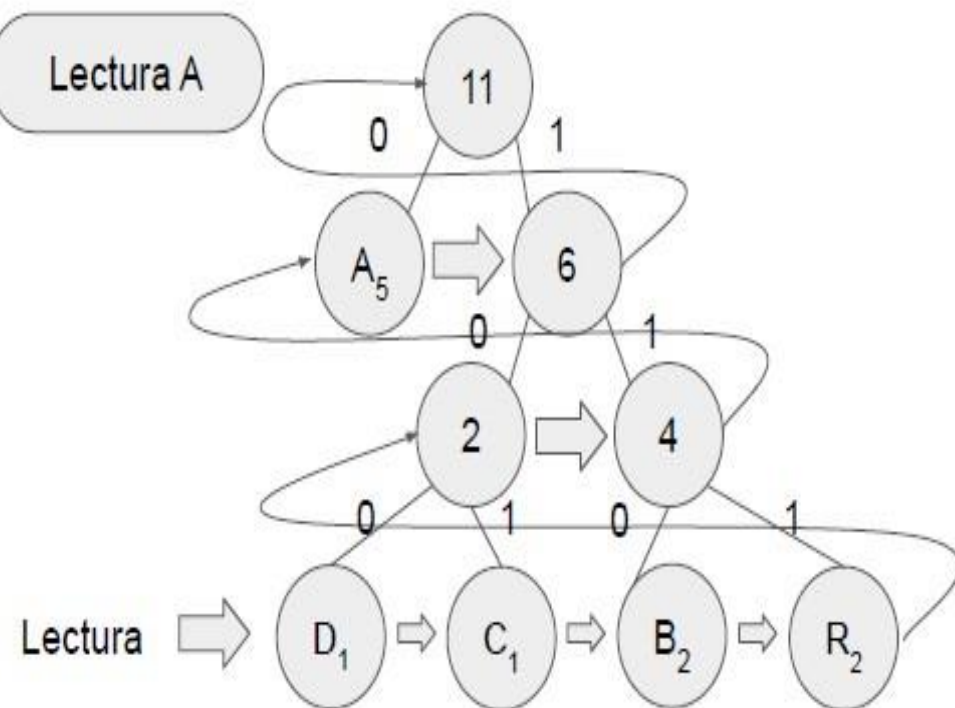
Lectura R



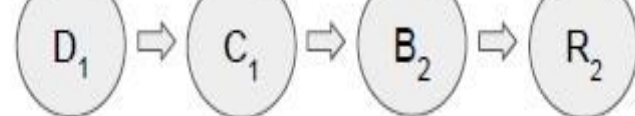
Lectura



Lectura A



Lectura



COMPRESIÓN

ABRACADABRA



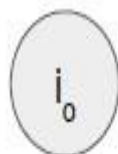
A = 0



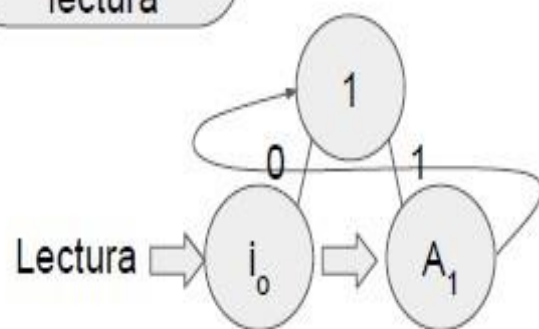
D₁ C₁ B₂ R₂ 2 4 A₅ 6 11

000 0 001 00 010 0 100 011 0 1100 100
0 110 110 0

Inicio



Primera
lectura



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



000 = A

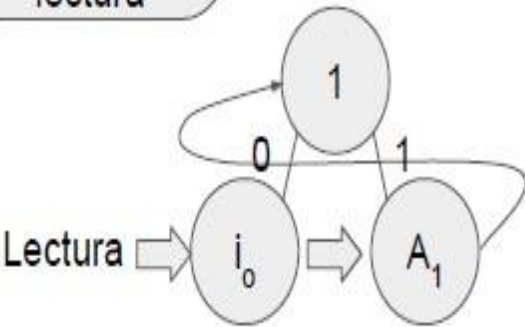


i_0 A_1 1

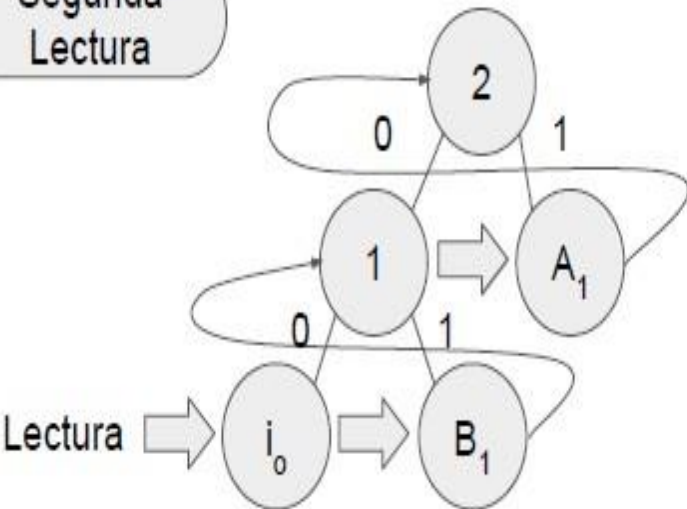
A

DESCOMPRESIÓN

Primera
lectura



Segunda
Lectura



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

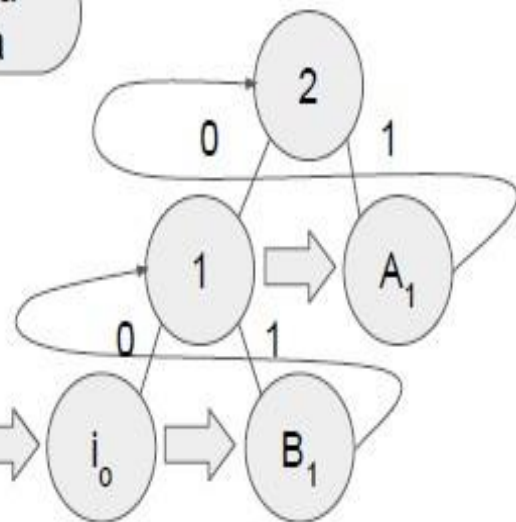
00000010001001000110110010001101100



0001 = B

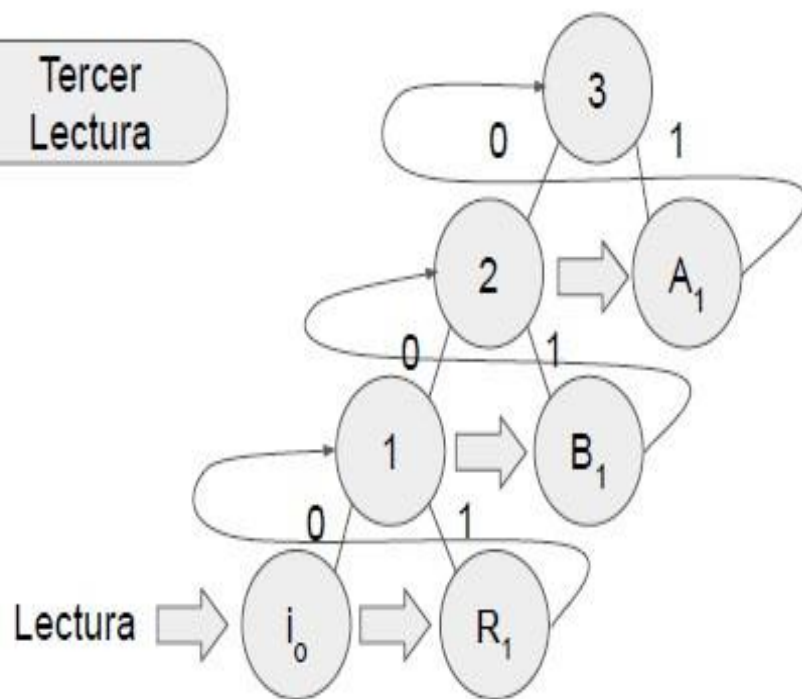

$$i_0 \quad B_1 \quad 1 \quad A_1 \quad 2$$
AB

Segunda Lectura



Lectura →

Tercer Lectura



Lectura →

Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



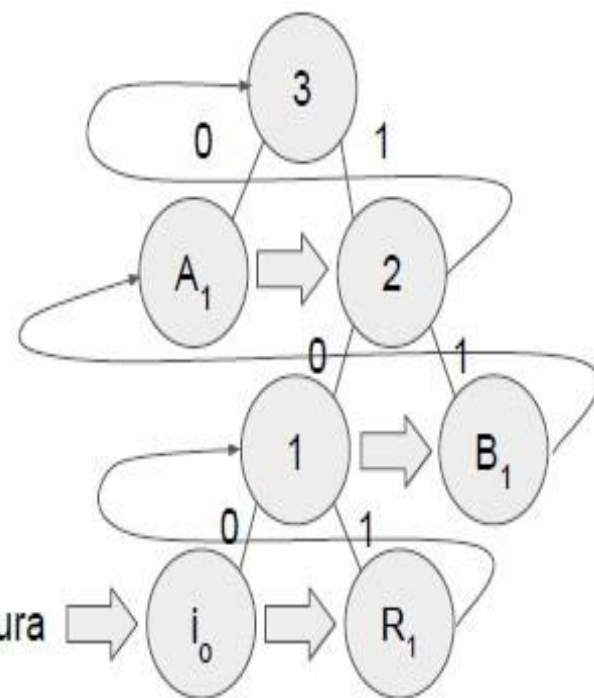
00010 = R



i_0 R₁ 1 B₁ 2 A₁ 3

A B R

Ordenamos

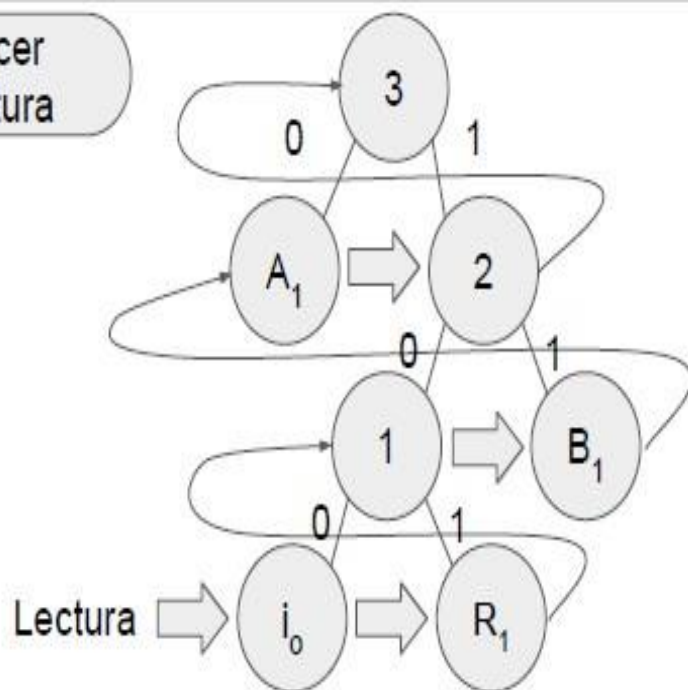


Lectura →

DESCOMPRESIÓN

i_0 R₁ 1 B₁ A₁ 2 3

Tercer
Lectura



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



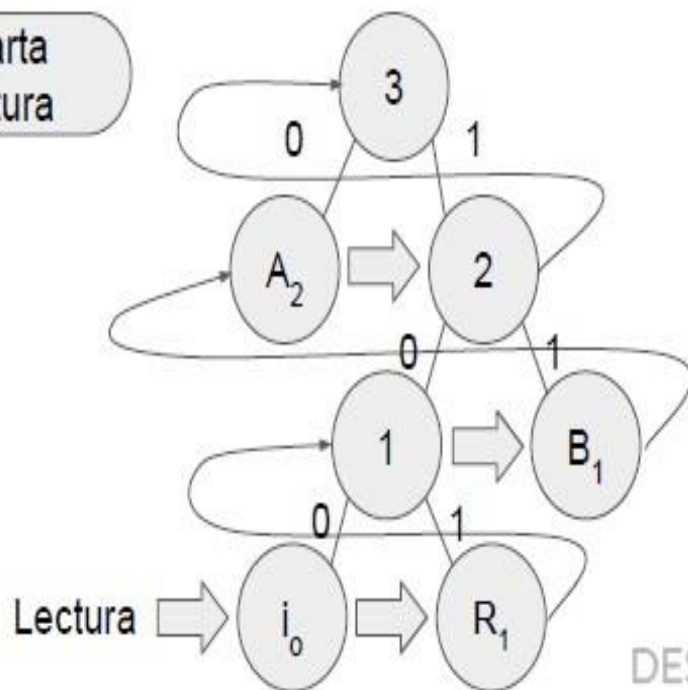
0 = A



i_0 R_1 1 B_1 A_2 2 3

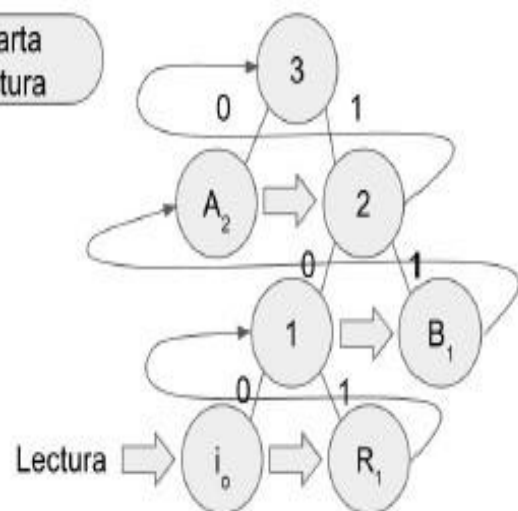
ABRA

Cuarta
Lectura



DESCOMPRESIÓN

Cuarta
Lectura



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



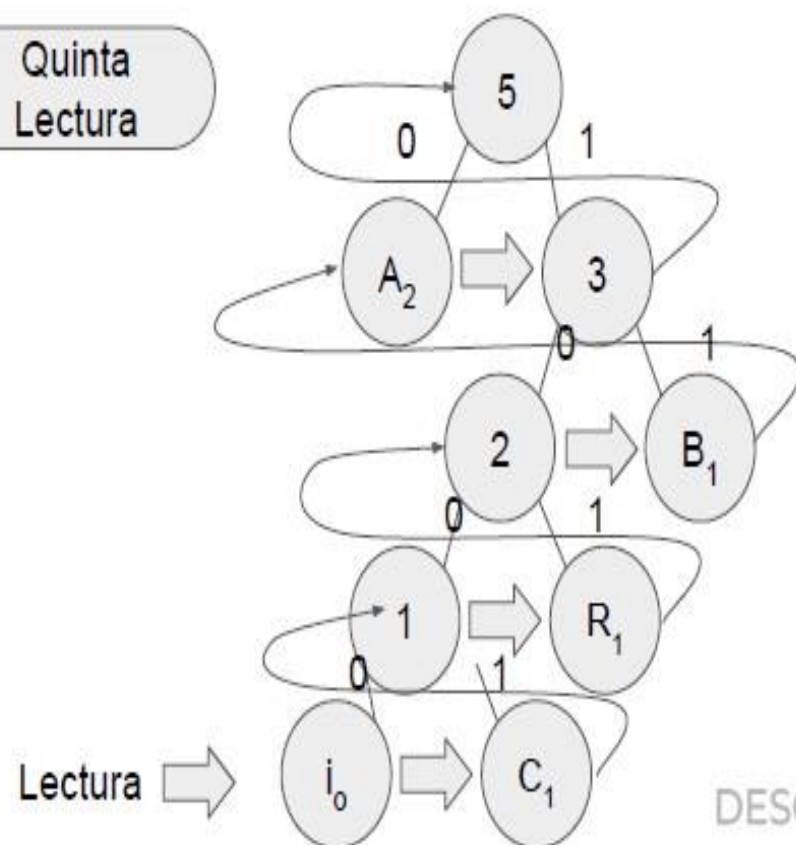
100011 = C



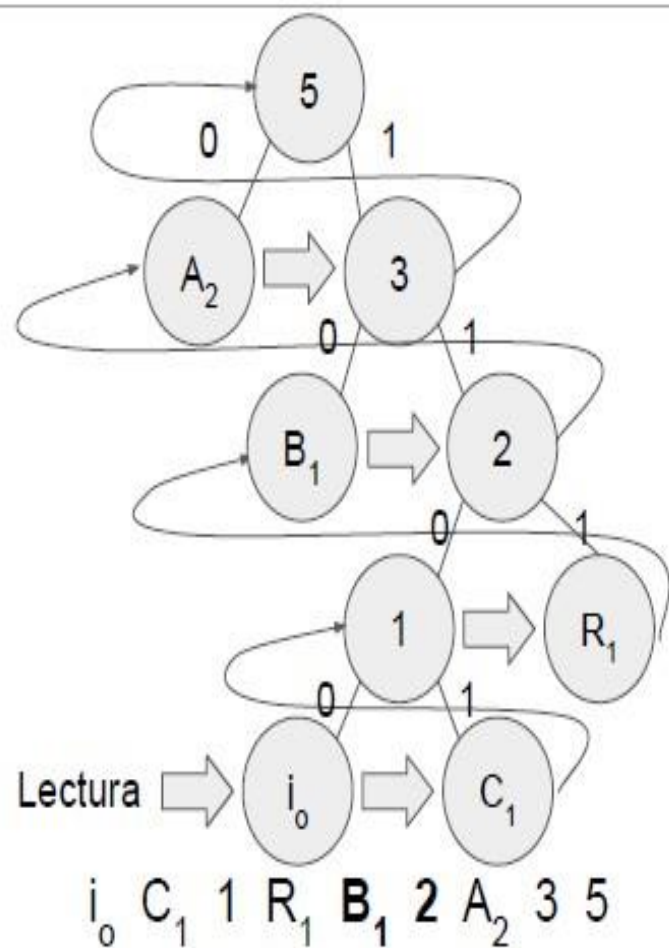
i_0 C₁ 1 R₁ **2 B₁** A₂ 3 5

ABRAC

Quinta
Lectura



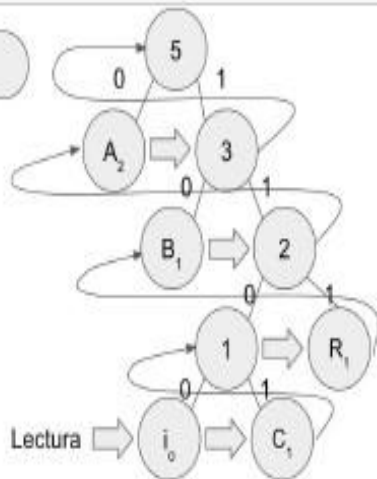
Ordenamos



DESCOMPRESIÓN

i_0 C₁ 1 R₁ **B₁ 2** A₂ 3 5

Quinta
Lectura



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



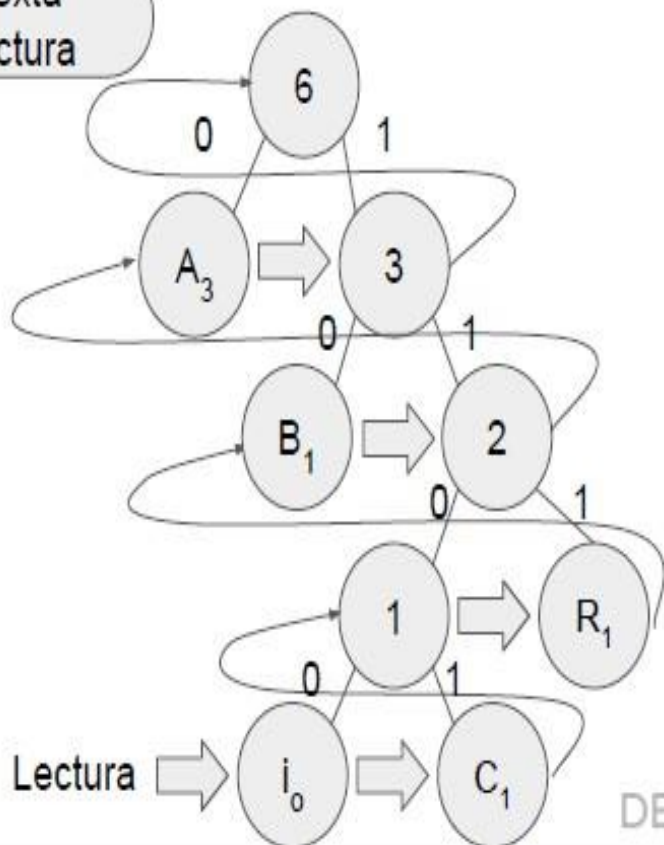
0 = A



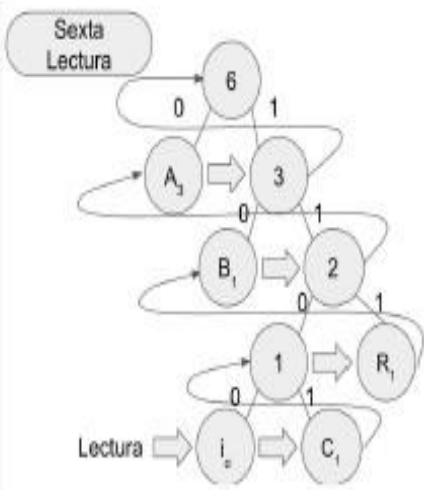
i_0 C_1 1 R_1 B_1 2 A_3 3 6

ABRACA

Sexta
Lectura



DESCOMPRESIÓN



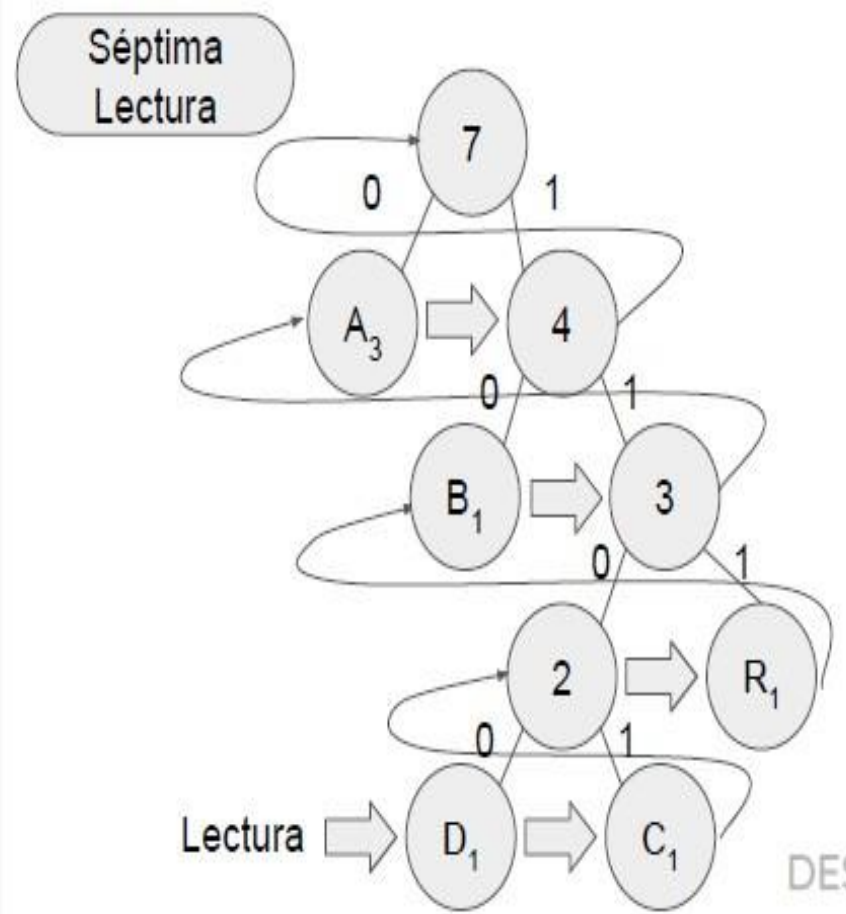
Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100

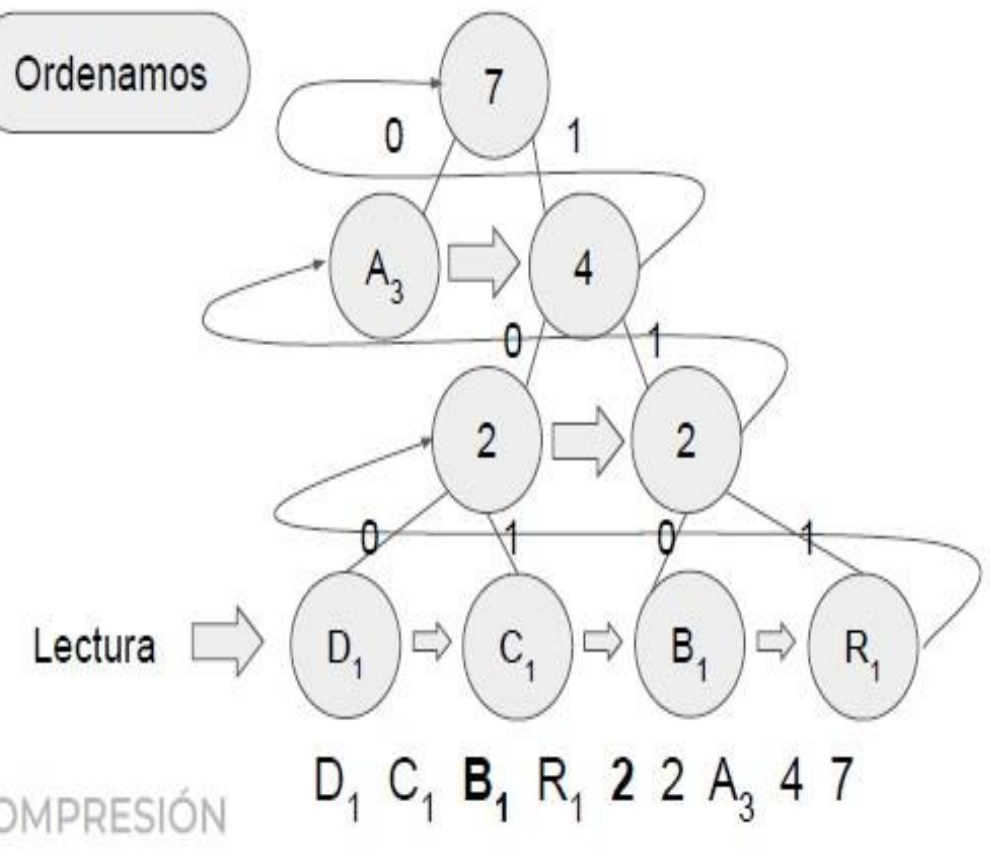
 1100100 = D

 $D_1 C_1 \mathbf{2} R_1 \mathbf{B}_1 3 A_3 4 7$

ABRACAD



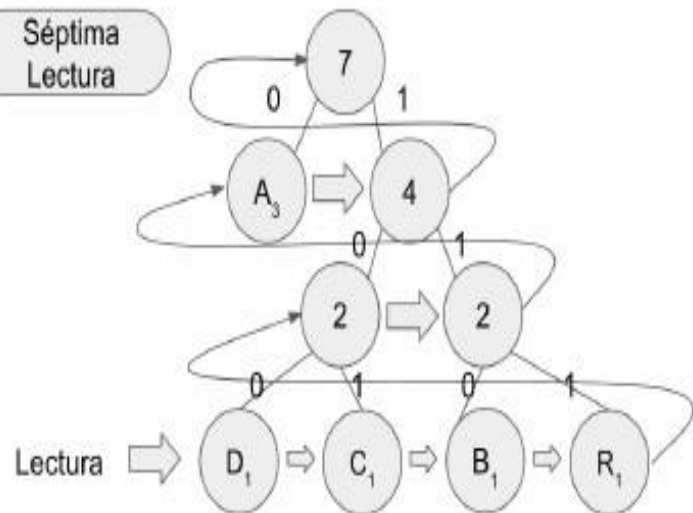
Ordenamos



DESCOMPRESIÓN

$D_1 C_1 \mathbf{B}_1 R_1 2 2 A_3 4 7$

Séptima
Lectura



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



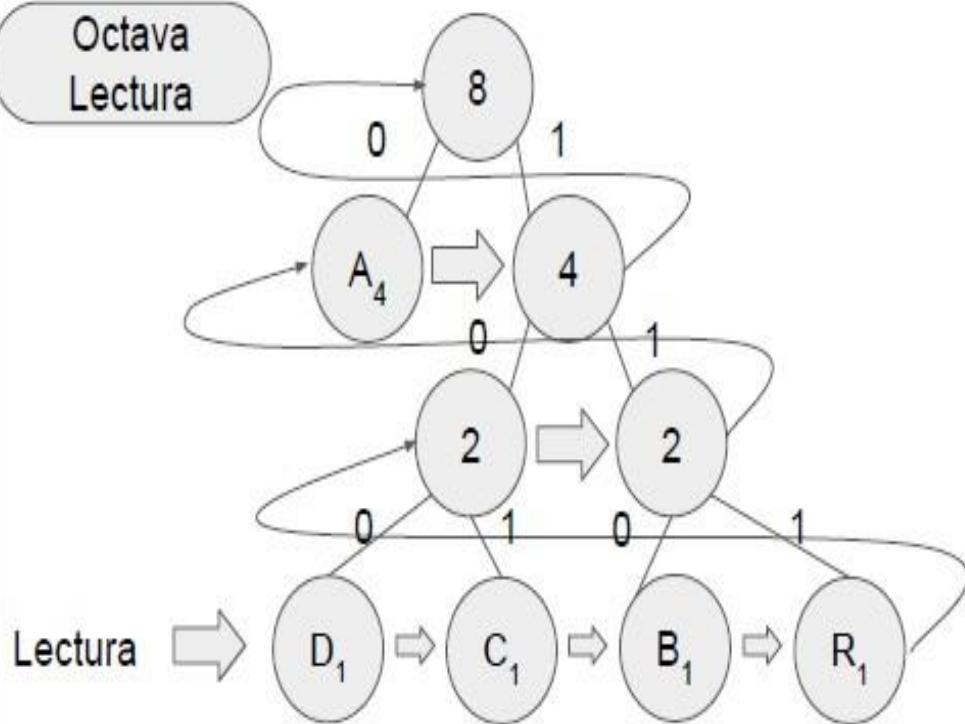
0 = A



D_1 C_1 **2** R_1 **B_1** 3 A_3 4 7

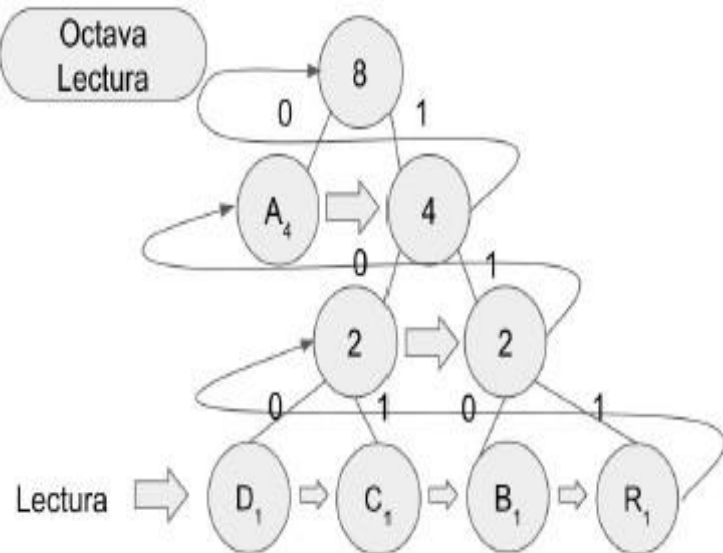
ABRACADA

Octava
Lectura



DESCOMPRESIÓN

Octava
Lectura



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



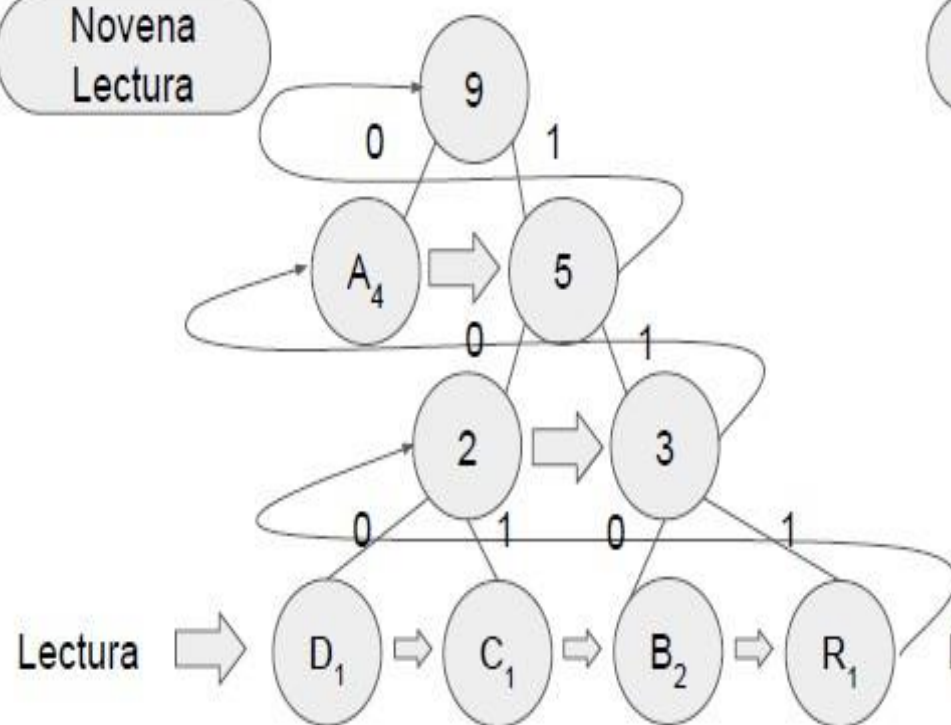
110 = B



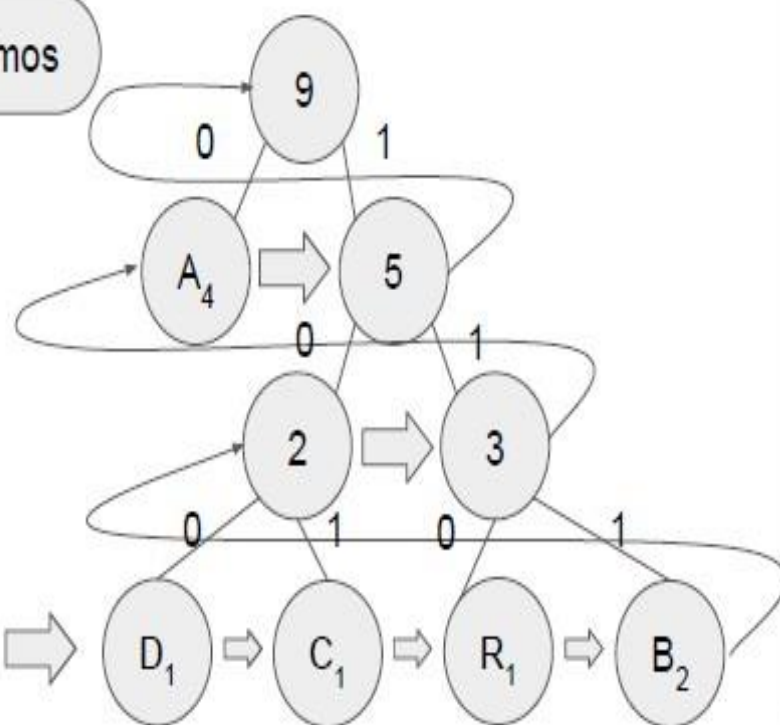
$D_1 C_1 B_2 R_1 2 3 A_4 5 9$

ABRACADAB

Novena
Lectura



Ordenamos

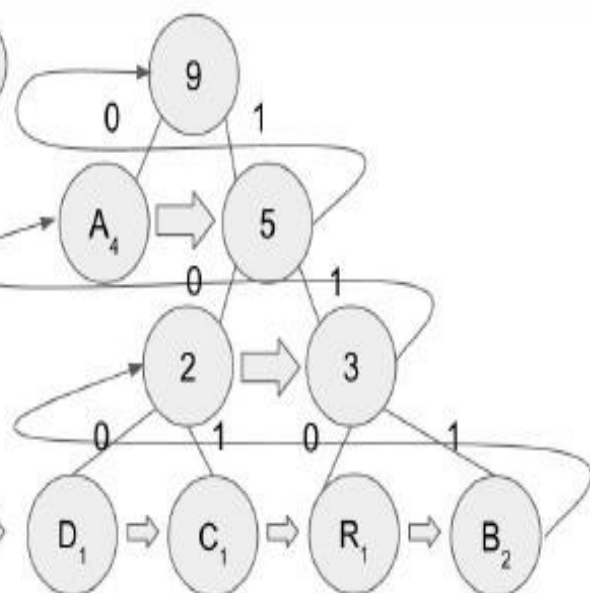


DESCOMPRESIÓN

$D_1 C_1 R_1 B_2 2 2 A_3 4 7$

Novena
Lectura

Lectura



Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



110 = R

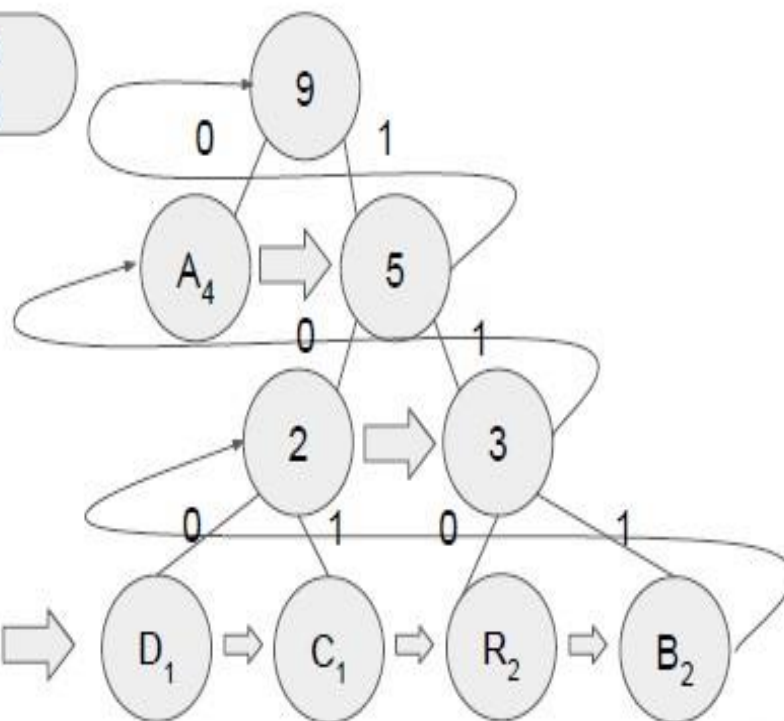


D₁ C₁ R₁ B₂ 2 3 A₄ 5 9

ABRACADABR

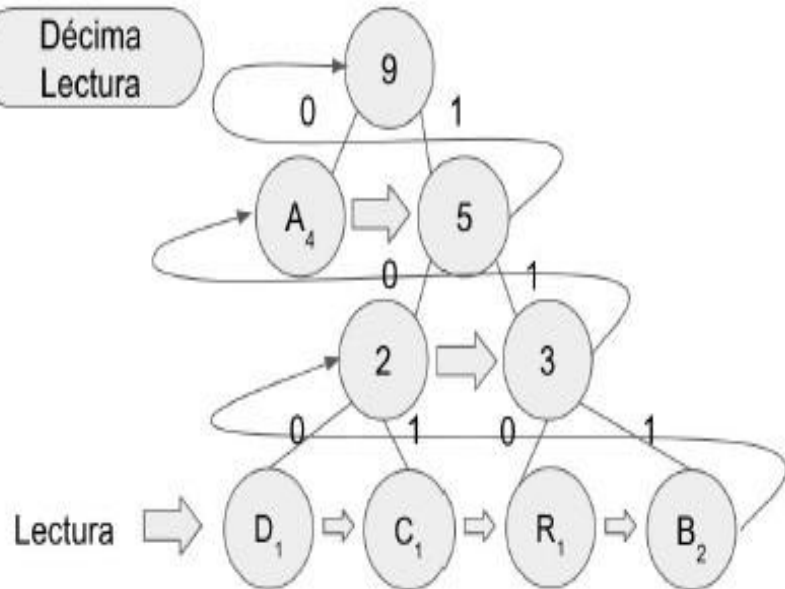
Décima
Lectura

Lectura

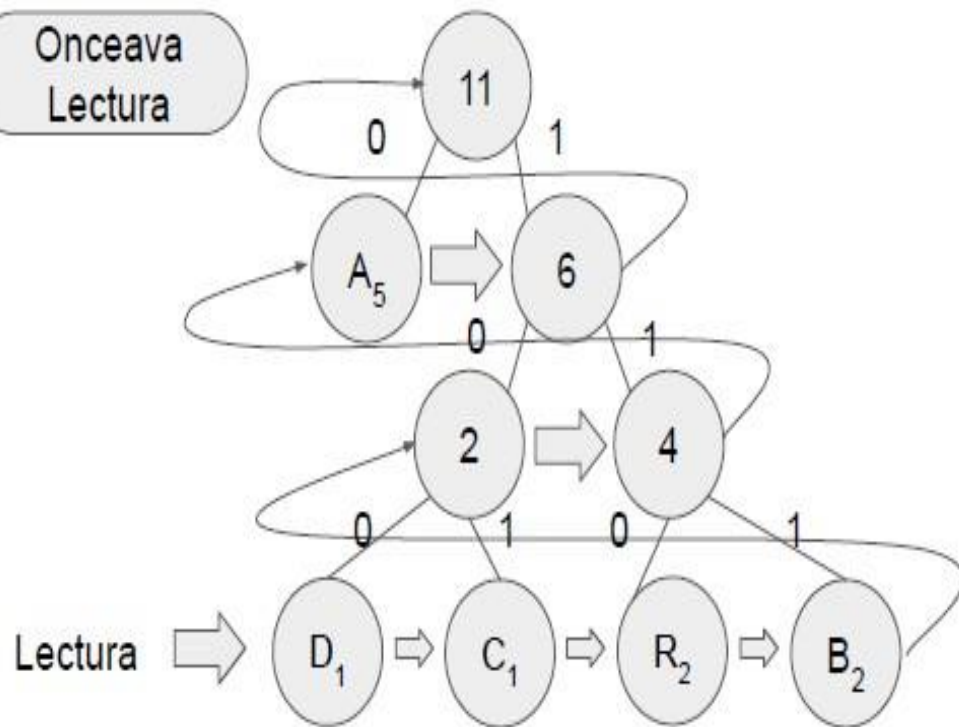


DESCOMPRESIÓN

Décima
Lectura



Onceava
Lectura



DESCOMPRESIÓN

Símbolo	Código
A	000
B	001
R	010
C	011
D	100

00000010001001000110110010001101100



0 = A



D₁ C₁ R₂ B₂ 2 4 A₅ 6 11

ABRACADABRA